Two-Time-Scale Transfer Learning for Market-by-Order Micro-Return Forecasting

Juan Diego Mejia Becerra[1], Gareth W. Peters[1], and Ruimeng Hu[1,2]

[1]Department of Statistics and Applied Probability, University of California, Santa Barbara
[2]Department of Mathematics, University of California, Santa Barbara

Last Modified: March 15, 2026

Abstract

Deep Learning models face a critical "simulation-to-reality" gap when applied to high-frequency financial data. Deep learning models often fail to capture the non-stationary dynamics of live markets requiring computationally expensive recalibrations. This work introduces the Two-Time-Scale Transfer Learning (TTSTL) framework, a hybrid model architecture that combines the flexibility of deep learning models with the parsimony and calibration ease of time series regression models. The TTSTL architecture mimics the hippocampus-neocortex interaction in the human brain by coupling a high-capacity deep learning "backbone" with a lightweight, adaptive time series regression "adapter." The backbone, utilizing a Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM) network, extracts complex, non-linear feature interactions from event-driven Limit Order Book (LOB) data, while the adapter—implemented as a Hybrid Autoregressive Moving Average with Exogenous variables-Generalized Autoregressive Conditional Heteroskedasticity (ARMAX-GARCH)-DeepLOB model—integrates these features into a statistical framework for micro-return forecasting. Experiments are conducted on synthetic and Intel (INTC) Market-by-Order (MBO) data. Results demonstrate that the TTSTL framework significantly outperforms standalone deep learning baselines in terms of accuracy (RelMAE), while also enabling volatility modeling and probabilistic forecasting.

Keywords: Market Microstructure, ARMAX-GARCH, Transfer Learning, Time Series Forecasting, High-Frequency Data, Limit Order Book

1 Introduction

Deep learning (DL) has emerged as a powerful tool across numerous fields, including finance, where its ability to handle large, complex datasets has proven invaluable. However, the efficacy of DL in finance is frequently counterbalanced by a significant "black box" problem. While neural networks excel at point imputation, they inherently lack the transparency and interpretability of parametric time series models. In regulatory and risk-management contexts, understanding the drivers of a forecast is often as critical as the point forecast accuracy itself. Pure DL architectures typically obscure these drivers, making it difficult to decompose a prediction into constituent market factors such as liquidity imbalances or volatility shocks. Furthermore, quantifying uncertainty—constructing reliable confidence intervals around imputations from DL models remains a computationally expensive and often heuristic endeavor in deep learning (see [3]), whereas it is a foundational feature of standard econometric inference.

In addition to these statistical limitations, the high-frequency environment imposes severe computational constraints. Today's electronic markets operate at a staggering scale and speed, with exchanges disseminating data at nanosecond resolution and liquid assets generating millions of Market-by-Order messages per session. This combination of velocity and volume creates a bottleneck for complex neural network structures, which often require hours or days for recalibration, rendering traditional model retraining mechanisms impractical. Moreover, the sheer data volume can destabilize numerical algorithms for calibration; likelihood functions can become too steep to allow estimation via smoothness-assuming algorithms such as Newton-Raphson or Fisher-Scoring. These issues are exacerbated by the non-stationary

behavior observed at the intra-day scale [17, 56]. Consequently, there is a need for a statistical modeling framework that not only generates reliable statistical forecasts but remains fast and adaptable.

Beyond computational hurdles, high-frequency financial modeling is constrained by the limited availability of high-quality datasets due to high costs [30] and the "simulation-to-reality" gap observed in older benchmarks like FI-2010 [47]. In this work, we treat this not as a primary motivation, but as a constraint on empirical validation. To mitigate this, we adapt existing tools to interface with Databento's pay-per-use service, lowering entry barriers for replicable research.

To address these statistical and computational challenges, the Two-Time-Scale Transfer Learning (TTSTL) framework is introduced. The primary objective of this work is to synthesize the feature-extraction power of deep learning with the interpretability of parametric econometric models. We propose an architecture where a deep neural network functions as a generator of non-linear interactions which are then fed into a parametric time series regression model—specifically, a Hybrid ARMAX-GARCH specification. Inspired by the complementary learning systems in neuroscience [45], TTSTL decouples the learning process into two distinct timescales: a slow-learning "backbone" that extracts stable, long-term features, and a fast-adapting "adapter" that recalibrates the parametric forecasting model online. This separation allows for computationally efficient updates and robust performance in the face of the non-stationary dynamics inherent to high-frequency finance.

1.1 Statistical Learning Models for the Limit Order Book

Statistical learning methods [26] can be used to uncover statistical regularities in financial data. Many empirical studies employ statistical learning methods to uncover stable patterns in high-frequency financial data and/or attempt to make predictions for the dynamics of the LOB. Outcomes range from the discovery of stylized facts such as a power-law distribution on the depth of incoming orders, and the gamma distribution of the available volume at the first level of the LOB [4], to the characterization of the behavior of trading algorithms [9], and the analysis of the factors driving the movement of the mid-price in short periods of time [14, 12, 11].

Statistical learning methods have been widely applied to generate short-term forecasts of LOB features using high-frequency data. For example, Easley et al. [19] use random forests to assess the predictive capabilities of three generations of microstructure measures, focusing on directional changes in the bid-ask spread, realized volatility, Jarque-Bera statistic, kurtosis, skewness, and sequential correlation of realized returns. More recently, an increasing body of research has turned to DL techniques for predicting price direction from LOB data [55, 61, 60, 31, 32, 7, 47], with architectures ranging from multi-layer perceptrons [55] to inception layers [61] and attention mechanisms [60]. Although these methods aim to automate feature discovery, questions persist regarding their effectiveness, robustness, and stability. In a recent study, Prata et al. [47] compare 15 distinct architectures in terms of accuracy and robustness, identifying the approach presented by Zhang et al. [60] as demonstrating improved classification accuracy under benchmark conditions.

The predictive capability of deep learning (DL) models is not without drawbacks. In addition to the considerable computational overhead required for training and inference, as well as interpretability challenges, these models rely on minimizing a loss function that may not reflect the true data-generating process. Referred to as the Empirical Risk Minimization (ERM) approach [6], this methodology can pose two main problems if applied indiscriminately. First, directly minimizing a loss function without careful input preprocessing may introduce spurious correlations [23, 46]. Second, within the DL context, the ERM approach calibrates a deterministic model which is then subsequently used to approximate the target process as a purely deterministic function of the model inputs via an *imputation* process rather than a formal statistical forecast, thereby complicating the ability to construct statistical predictive intervals which require conditional stochasticity to be present for a formal definition of the forecast distribution or its quantiles. Throughout this study, outputs from neural networks are therefore termed *imputations*, rather than predictions or forecasts.

The first of these problems was studied, at least indirectly, by Kolm, Turiel and Westray [31]. Their study compared the imputation accuracy of neural networks of varying complexity when provided with either stationary inputs (based on order flows [14]) or non-stationary inputs (raw LOB data). The findings indicate that models trained on stationary inputs outperform those relying on non-stationary data. Moreover, simpler network architectures with stationary inputs were shown to surpass more complex architectures operating on non-stationary inputs. This is the DL analog of the widely known results from the econometrics literature where it is typically advised to undertake time series regression models under stationarity assumptions, to avoid spurious regression outcomes, see [23, 46, 24].

A focus on statistical post-processing of DL imputations, rather than further architectural innovation, is motivated by the evidence above. In particular, the second issue in the ERM approach is addressed by refining imputations from DL models into a hybrid econometrics time series regression model. These imputations are embedded into an ARMAX-GARCH time series model that effectively approximates the stochastic dynamics of the micro-price process, thereby enabling reliable forecasting. Furthermore, a rigorous assessment of both the statistical and practical significance of DL-based imputations is facilitated by the proposed model. Additionally, solutions are provided to mitigate numerical challenges arising from steep likelihood functions, which can otherwise hinder likelihood-based inference.

1.2 Outline

The remainder of this paper is organized as follows. Section 2 outlines the primary methodological and empirical contributions of this work. Section 3 establishes the mathematical preliminaries for Market-by-Order (MBO) data and Limit Order Book (LOB) reconstruction. Section 4 details the feature generation and extraction process. The proposed Hybrid ARMAX-GARCH-DeepLOB model is formulated in Section 5, integrating generative features into a statistical time series framework. Section 6 introduces the Two-Time-Scale Transfer Learning (TTSTL) framework, describing the interaction between the backbone, interface, and adapter modules. Estimation procedures, including the recursive online combining rules, are presented in Section 7, followed by the forecasting methodology in Section 8. Section 9 details the performance metrics used to evaluate predictive accuracy. The experimental design, including data collection and preprocessing, is described in Section 10. Section 11 presents a comprehensive empirical evaluation of the framework, demonstrating its predictive efficacy and robustness through experiments on both synthetic LOB environments and real-world high-frequency INTC market-by-order data. Finally, Section 12 concludes the study with a summary of the main findings and a discussion of their implications.

2 Contributions

This study addresses the limitations of applying deep learning models directly to non-stationary high-frequency financial data by proposing a hybrid methodological framework. The primary contributions are as follows:

2.1 Methodological Contributions

1. **Methodological Framework (TTSTL):** A Two-Time-Scale Transfer Learning (TTSTL) framework is introduced. In this framework, the learning process is distinctively separated into two timescales: a slow-moving "backbone" wherein non-linear representations are learned from event-driven data, and a fast-moving "adapter" wherein typical econometric models are recalibrated to local market dynamics. This separation addresses the prohibitive computational cost of frequent retraining for deep neural networks while maintaining responsiveness to regime shifts. The TTSTL framework is validated against a Markovian MBO data simulator based on the framework proposed in [13].

2. **Hybrid Time Series Regression Model:** A Hybrid ARMAX-GARCH-DeepLOB model is formulated for the adapter module within the TTSTL framework. In contrast to standard deep learning approaches, where deterministic point imputations are produced, neural network outputs are treated here as generative covariates within a formal probabilistic framework. Through this integration, transparent and efficient construction of valid prediction intervals for micro-returns is enabled, bridging the gap between "black-box" predictions and statistical inference.

2.2 Empirical Contributions

1. **Accessible MBO Data Manipulation Framework:** The `meatpy` library, originally developed by Vincent Gregorie and Charles Martineau[1], was adapted to interface with Databento's pay-per-use data service[2]. By this approach, entry barriers for researchers are significantly lowered, enabling access to a more comprehensive set of assets, and fostering cost-effective, reproducible studies in

[1] `https://github.com/vgreg/MeatPy`
[2] `https://databento.com`

the high-frequency domain. The modified library and data manipulation implementations relating to this manuscript are made publicly available at `https://github.com/judmejiabe/MeatPy` and `https://github.com/judmejiabe/evolving-microstructural-toolkit-data.git` respectively.

2. **Comparative Analysis of TTSTL Configurations on INTC MBO Data:** Several TTSTL configurations were applied and contrasted using INTC MBO data from May and June 2024. All configurations yielded more accurate forecasts than raw CNN-LSTM imputations. Superior accuracy was observed in the configuration without external covariates, suggesting limits to the marginal benefits of complicated neural network architectures for low-latency micro-return imputations and highlighting the necessity for further comparisons between parsimonious and hybrid micro-return modeling approaches.

3 Background: Market-by-Order and Limit Order Book Data Structure

This section is devoted to establishing the data definitions and notation used throughout this manuscript. In Section 3.1 the limit order book (LOB) and Market-by-Order (MBO) processes are formally introduced, including the order flow sequence and its stochastic counterpart. In Section 3.2 event-driven and physical-time sampling schemes are presented, and their roles in discretizing continuous-time data streams are discussed. These preliminaries underpin the feature extraction and forecasting methodologies developed in later sections.

3.1 The Limit Order Book and Market-by-Order Data

Limit order books (LOBs) are a prevalent data structure in modern financial markets, serving as repositories for unexecuted limit orders. All visible orders are displayed and retained until they are either canceled or matched against a suitable counter-order. During trading hours, orders are submitted by buyers and sellers to a matching engine—dedicated hardware within the exchange. These orders typically fall into two broad categories: limit orders and market orders. When a limit order is submitted, both the volume and the maximum (for buys) or minimum (for sells) acceptable price are specified. Such orders remain in the LOB until a matching counterpart is found, or they are adjusted or canceled. In contrast, market orders require only a specified volume and side (buy or sell) at inception and are executed immediately against the best available order on the opposite side. While various matching rules are implemented by different exchanges, the common price-time priority mechanism is the focus of this work. Under this regime, orders are executed first by price competitiveness, and subsequently by arrival time when prices are equal. For a comprehensive treatment of limit order books and their stylized features, reference is made to [43, 22, 28, 49, 50].

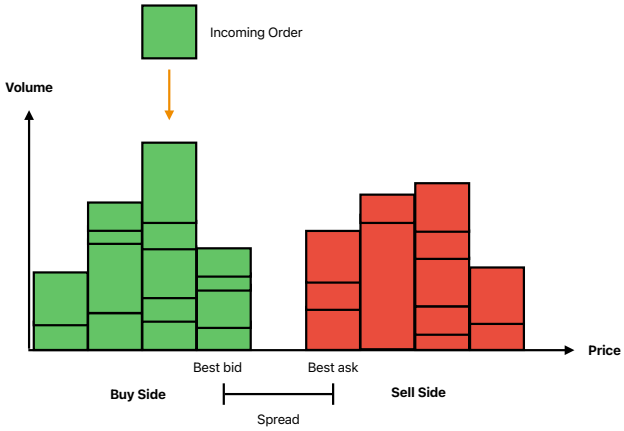


Figure 1: Limit order book snapshot illustration.

Market-by-order (MBO) data offers a more granular view than standard LOB snapshots. The full sequence of messages generated at the trading venue, including order additions, cancellations, and executions, is comprised within this data. Because the entire order flow and execution messages are captured by MBO data, it can be utilized not only to reconstruct the LOB but also to extract detailed microstructure measures. The mathematical framework underpinning these representations is provided in Section A of the supplementary material, drawing heavily on [13, 22, 42].

In the main text, only basic notation is introduced to preserve the flow of the discussion. Let the value of a given feature y at time $t \in [0, T]$ be represented by $y(t)$. As will be demonstrated in the next section, the LOB can be sampled in several ways. Two time scales are utilized: the first consists of the random times at which the LOB value is changed due to a new limit order addition, cancellation, or market order execution, by which an event-driven time series is produced. The second time scale consists of the regularly spaced observation times at which the LOB snapshot realization may be observed. A random partition [48, Ch. 2] is sufficient to distinguish between these two schemes. Random partitions are denoted as $\mathcal{P} = \{\tau_i\}_{i=0}^{N}$, where $0 = \tau_0 \leq \tau_1 \leq \tau_2 \leq \cdots \leq \tau_N = T$ are stopping times, and N is an almost surely finite integer-valued random variable. For any random partition, let $y_i^{\mathcal{P}} = y(\tau_i)$ represent the observation of the feature y at the corresponding stopping time τ_i. Whenever the random partition is clear from the context, the superscript $\mathcal{P}$ will be omitted. It is noted that if the stopping times are random, $\mathcal{P}$ models the event-driven scheme; otherwise, $\mathcal{P}$ models the time-driven scheme. This distinction is further explained in Section 3.2.

Consider a trading session time interval represented by $[0, T]$, during which a sequence of orders sent by participants is observed. This order flow sequence (OFS) consists of messages denoted by $\mathbf{o} = \{o_i\}_{i=1}^{N-1}$, where each message $o_i = (p_i, z_i, t_i, i_i, s_i, k_i)$ is submitted to the matching engine at time $t_i \in [0, T]$. Here, $p_i \in \mathbb{N}_{0:d}$ denotes the price level, $z_i \in \mathbb{N}$ the size, $i_i \in \mathbb{N}$ a unique message identifier, $s_i \in \{1, -1\}$ the side of the limit order book (with $s_i = 1$ indicating the bid side and $s_i = -1$ the ask side), and $k_i \in \{L, C, M\}$ the message type (limit, cancellation, or market order, respectively). For simplicity, buy (sell) market orders in the order flow sequence $\mathbf{o}$ are identified as limit orders with the maximum (minimum) possible price. That is,

$$(p_o, z_o, t_o, i_o, 1, M) \equiv (d, z_o, t_o, i_o, 1, L),$$
$$(p_o, z_o, t_o, i_o, -1, M) \equiv (0, z_o, t_o, i_o, -1, L).$$

A large number of participants interact in financial markets with a wide array of trading strategies; the aggregate process of such collections of orders can be modeled stochastically. Let $\boldsymbol{\omega} = (\omega_i : \tau_i \leq T)$ denote the Stochastic Order Flow Sequence (SOFS) where $\omega_i = (\pi_i, \zeta_i, \tau_i, \iota_i, \sigma_i, \kappa_i)$ is the random object whose realizations are given by o_i (explained above). The sequence $\mathbf{o}$ is designated as the Observed Order Flow Sequence (OOFS).

The OOFS $\mathbf{o}$, together with a specified matching rule (price-time in this case), produces MBO data. There are various ways to represent this information. Table 1 compares two such schemes: Nasdaq's ITCH 5.0 protocol [41] and the MBO data format provided by Databento[3]. A key distinction between the OFS and MBO data is that the OFS itself does not reflect matching, while the MBO data is already matched according to the given matching rule. As a result, reconstructing the limit order book (LOB) from MBO data is relatively straightforward; sufficiency is achieved by maintaining two lists—one for the buy side and one for the sell side—backed by hash tables keyed by order IDs. Conversely, reconstructing the LOB directly from the OFS is more complex and computationally intensive, as it requires continuously tracking order priority in addition to storing all outstanding orders. Furthermore, incomplete OFS data is often provided by some data vendors. Approximate LOB reconstruction when only incomplete OFS data is available is discussed in Chapter 3 of [49].

Finally, an array representation of the LOB is presented. The array representation is the one typically used in DL applications to the LOB (see for example [32, 61]). Let $p_l^b(t)$ ($p_l^a(t)$) denote the best bid (ask) price located l ticks below (above) the best bid (ask) at time t, and let $v_l^b(t)$ ($v_l^a(t)$) denote the corresponding volume at that price level. Define the row vector $\mathbf{l}^{(L)}(t)$ to include $L + 1$ levels of limit order book information arranged in the following way:

$$\mathbf{l}^{(L)}(t) = [p_0^a(t), v_0^a(t), p_0^b(t), v_0^b(t), \ldots, p_L^a(t), v_L^a(t), p_L^b(t), v_L^b(t)].$$

For multiple time points $t_1, t_2, \ldots, t_N$, these vectors are arranged into a matrix:

[3]See `https://databento.com/docs/venues-and-datasets/xnas-itch?historical=python&live=python&reference=python`

Table 1: Message mapping of Nasdaq's ITCH (NQ) and Databento (DB) messages. The T, F, and C indicators from Databento can be interpreted as Trade, Fill, and Cancel, respectively.

Message description	NQ indicator	DB indicator	Remarks
Add Order	A	A	Limit Order on Bid or Ask
Add Order with MPID Attribution	F	A	Attribution details are omitted in the DB dataset.
Order Executed Message	E	T, F, then C	All indicators share the same timestamp.
Order Executed with Price Message	C	T, F, then C	All indicators share the same timestamp. Indicator T is omitted if the message is marked as non-printable.
Order Cancel	X	C	The `quantity` column specifies the number of shares to be removed from the order.
Order Delete	D	C	The `quantity` column shows the current display size of the order.
Order Replace	U	C, then A	Both indicators share the same timestamp. Indicator C is issued with the original order ID, followed by A with a new ID.
Trade Message (Non-Cross)	P	T	Indicates execution against a non-displayable order (e.g. Iceberg orders). The transaction side is marked as N.
Cross Trade Message	Q	T	Indicates completion of a cross trade. The transaction side is marked as N.

$$\mathbf{l}^{(L)}(t_{1:n}) = \mathbf{l}(t_1, t_2, \ldots, t_n) = \begin{bmatrix} \mathbf{l}^{(L)}(t_1) \\ \mathbf{l}^{(L)}(t_2) \\ \vdots \\ \mathbf{l}^{(L)}(t_n) \end{bmatrix}.$$

3.2 Sampling LOB and MBO data

Determining the optimal sampling scheme for high-frequency financial data remains an open question [17]. Many researchers favor event-driven schemes—i.e., using random stopping times tied to market events—rather than physical time-based sampling. Indeed, some studies report that, in practice, event-driven and uniform-time sampling yield very similar results [19]. However, other work has indicated that trade time sampling, volume clock sampling or other approaches can affect the ability to discern certain statistical structure in LOB data, see discussion in [33]. Therefore, the fact that such sampling schemes are found to not materially affect the DL based modeling approaches may be indicative of the fact that such statistical structure is not being captured adequately by such DL methods in the first place, rather than evidence that such sampling method distinctions are inconsequential.

Multiple time scales interact when analyzing LOB and MBO data. These can be classified into event- and physical time-driven sampling schemes. Event-driven sampling schemes use market events such as trade times, order submission times, cancellation times, flash-crashes which can be effectively modeled as random stopping times. Physical time-driven sampling schemes use deterministic sampling schemes

which might or might not be uniform. The sampling schemes interacting in this study are detailed below

- **Event-Driven Sampling Schemes**
  - **All-Events Scheme**: Samples the quantities of interest at every market event, such as when an order is added, canceled, or executed. Denote the corresponding random partition as $\mathcal{P}^{(E)} = \{\tau_j^{(E)}\}_{j=0}^{N^{(E)}}$, where $\tau_j^{(E)}$ is the time of the j-th event. This partition will appear in the implementation of the CNN-LSTM neural network (see Section 4.1) and the order flow imbalance calculation (see Table 3 in Section 10.1).
  - **Trades Scheme**: Samples the quantities of interest each time a trade occurs. Denote the corresponding random partition as $\mathcal{P}^{(T)} = \{\tau_j^{(T)}\}_{j=0}^{N^{(T)}}$, where $\tau_j^{(T)}$ is the time of the j-th trade. This partition will appear in the definition of Roll's measure, Roll's impact measure, Kyle's λ, modified Amihud's measure, and volume-synchronized probability of informed trading (see Table 3 in Section 10 for detailed definitions and references).
- **Physical Time-Driven Sampling Schemes**
  - **Uniform Sampling:** Samples the quantities of interest on a uniform partition. Denote the corresponding deterministic partition as $\mathcal{P}_\delta^{(U)} = \{\delta j\}_{j=0}^{N^{(U)}}$, where $\delta = T/N^{(U)}$ corresponds to a fixed sampling period. This scheme is most readily applied within the time-series econometrics framework. The uniform sampling scheme will be adopted for the Hybrid ARMAX-GARCH-DeepLOB model (see Section 5). All of the endogenous and exogenous variables in the model will be sampled according to this scheme either by temporal aggregation (i.e., integrating a quantity of interest over the duration of the time bar) or point sampling (i.e., recording the quantity exactly at the right endpoint of the time bar), as detailed in Table 3.

In this context, the intervals defined by a partition $\mathcal{P}$ are called *bars*. While various sampling schemes such as volume bars can be defined (see Appendix B for detailed constructions), this work relies on two primary partitions. First, the *all-events* partition $\mathcal{P}^{(E)} = \{\tau_j^{(E)}\}_{j=0}^{N^{(E)}}$, where $\tau_j^{(E)}$ denotes the time of the j-th market event, is employed to generate inputs for the CNN-LSTM network. Second, the *time bars* partition $\mathcal{P}^{(U)}$ is used for the hybrid model's adapter module, defined by

$$\tau_j^{(U)} = \delta\, j, \quad j = 0, 1, \ldots,$$

so that each interval $(\tau_{j-1}^{(U)}, \tau_j^{(U)}] = (\delta(j-1), \delta j]$ is called a *time bar*.

To capture the local stationarity inherent in MBO and LOB data, different model components are updated on three nested time scales: *backbone* time bars, *adapter* time bars, and *sampling* time bars. First, each trading session is uniformly partitioned into coarse backbone bars, which govern the full retraining of the global model. Within each backbone bar, finer adapter bars are defined to schedule the adapter's online updates. Finally, each adapter bar is subdivided into sampling bars, on which data are ingested and one-step-ahead forecasts are generated.

Formally, let B_d^D, $d = 1, 2, \ldots, D$ denote the 9 : 45 am to 3 : 45 pm[4] time interval of the d-th trading session in the sample. This interval is then hierarchically partitioned as follows:

$$\mathcal{P}_{\Delta^B}^{(U)}\big(B_d^D\big) = \{\, B_{d,k}^B : k = 1, \ldots, K\},$$

where each backbone time bar $B_{d,k}^B$ has length Δ^B. Within each backbone time bar, the uniform partition

$$\mathcal{P}_{\Delta^A}^{(U)}\big(B_{d,k}^B\big) = \{\, B_{d,k,\ell}^A : \ell = 1, \ldots, L\},$$

is applied to produce adapter time bars of length Δ^A. Finally, each adapter time bar $B_{d,k,i}^A$ is partitioned by

$$\mathcal{P}_\delta^{(U)}\big(B_{d,k,\ell}^A\big) = \{\, B_{d,k,i,j}^S : j = 1, \ldots, J\},$$

yielding sampling time bars of length δ.

[4] All times in this work are EST

4 Generative Feature Learning via Deep Learning Methods: The DeepLOB and CNN-LSTM architectures

This section reviews the deep learning architectures used to extract nonlinear features from high-frequency LOB data and additional feature processing. In Section 4.1 the DeepLOB [60] and CNN-LSTM architectures [31] are described along with training procedures and input-output mapping. Section 4.2 explains the feature processing which supports the representations used by the hybrid econometric model proposed in this paper.

4.1 The CNN-LSTM Architecture

The CNN-LSTM architecture, originally presented in [31], is a refinement of the DeepLOB neural network architecture, introduced by Zhang et al. [61]. The DeepLOB architecture integrates convolutional, inception, and LSTM layers to impute midprice direction. Although its performance has been outpaced by more recent frameworks [47], DeepLOB is still considered a state-of-the-art model for price direction prediction in high-frequency financial markets [7, 31]. Several authors have investigated the factors contributing to the high accuracy of the DeepLOB architecture [31, 32, 7]. Particularly, Kolm et al. [31] empirically demonstrate that removing a composite convolutional block (see Section C.2.1 in the Supplementary Material) from the original DeepLOB architecture while feeding it with order flows as inputs (see Table 3 in Section 10.1; not to be confused with the order flow sequence defined in Section 3.1) tends to have a positive effect on accuracy, leading to the CNN-LSTM architecture illustrated in Figure S3 of Appendix C.

The CNN-LSTM architecture is employed as a high-capacity nonlinear operator mapping stationary order-flow sequences into a low-dimensional latent representation. This representation is treated as a stochastic covariate process whose realizations summarize nonlinear interactions in the limit order book. Detailed architectural specifications, including layer-by-layer descriptions, filter sizes, and optimizer hyperparameters, are provided in Appendix C.

In its original form, DeepLOB takes as input the top 10 levels of the LOB over a window of 100 events. Formally, for the all-events partition $\mathcal{P}^{(E)}$, the DeepLOB input at step i is:

$$\tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i-99:i} := \tilde{\mathbf{l}}^{(9)}\Big(\tau^{\mathcal{P}^{(E)}}_{i-99}, \tau^{\mathcal{P}^{(E)}}_{i-98}, \ldots, \tau^{\mathcal{P}^{(E)}}_{i}\Big) = \begin{bmatrix} \tilde{\mathbf{l}}^{(9)}\left(\tau^{\mathcal{P}^{(E)}}_{i-99}\right) \\ \tilde{\mathbf{l}}^{(9)}\left(\tau^{\mathcal{P}^{(E)}}_{i-98}\right) \\ \vdots \\ \tilde{\mathbf{l}}^{(9)}\left(\tau^{\mathcal{P}^{(E)}}_{i}\right) \end{bmatrix} \in \mathbb{R}^{100\times 40},$$

where $\tilde{\mathbf{l}}^{(L)}(\tau^{(E)}_x)$ is a whitened (normalized) version of the original LOB data $\mathbf{l}^{(L)}(\tau^{(E)}_x)$. As pointed out in [31], forecasting inputs should be stationary to avoid spurious relationships [23, 46]. Since the DeepLOB inputs include prices, which are inherently non-stationary, Kolm et al. [31] show that accuracy improves when employing stationary rather than non-stationary inputs. Constructing a stationary counterpart of the DeepLOB inputs is not trivial, as the data consist of both prices and volumes. Kolm et al. [31] achieves this by employing whitened order flows as inputs [14, 12]. These inputs can be represented as

$$\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i} = \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}\big(\tau^{\mathcal{P}^{(E)}}_{i-99}, \tau^{\mathcal{P}^{(E)}}_{i-98}, \ldots, \tau^{\mathcal{P}^{(E)}}_{i}\big) = \begin{bmatrix} \widetilde{\mathbf{OF}}^{(9)}_{i-99} \\ \widetilde{\mathbf{OF}}^{(9)}_{i-98} \\ \vdots \\ \widetilde{\mathbf{OF}}^{(9)}_{i} \end{bmatrix} \in \mathbb{R}^{100\times 20},$$

where $\widetilde{\mathbf{OF}}^{(L)}_i$ is a prewhitened version of the order flows (as defined in Table 3 and the comments below in Section 10.1)

$$\mathbf{OF}^{(L)}_i = \big[\, OF^a_{0,i},\ OF^b_{0,i},\ OF^a_{1,i},\ OF^b_{1,i},\ \ldots,\ OF^a_{L,i},\ OF^b_{L,i}\big] \in \mathbb{R}^{2(L+1)}.$$

The original DeepLOB target variable is a ternary classification of future price movements, derived from a filtered version of the midprice. Let $m_i = \big(p^{a,\mathcal{P}^{(E)}}_{0,i} + p^{b,\mathcal{P}^{(E)}}_{0,i}\big)/2$ denote the midprice at the i-th

event. For a chosen horizon $h \in \mathbb{N}$,

$$m_i^- = \frac{1}{h}\sum_{j=0}^{h} m_{i-j}, \qquad m_i^+ = \frac{1}{h}\sum_{j=1}^{h} m_{i+j}.$$

Two direction scores are considered:

$$d_i^{(1)} := \frac{m_i^+ - m_i^-}{m_i^-}, \qquad d_i^{(2)} := \frac{m_i^+ - m_i}{m_i}.$$

A ternary quantization of one of these scores is then employed as a target variable in the DeepLOB architecture:

$$y_i = \begin{cases} -1, & d_i^{(j)} < -\theta, \\ 0, & -\theta \le d_i^{(j)} \le \theta, , \\ 1, & d_i^{(j)} > \theta, \end{cases}$$

for a fixed threshold $\theta \in (0,1)$. While this ternary labeling is straightforward, it introduces two tuning parameters—namely the prediction horizon h used to compute $d_i^{(j)}$ and the threshold θ. In practice, selecting values that remain robust across varying market regimes can be challenging and may lead to instability in predictive performance. Consequently, the CNN-LSTM architecture proposed in [31] adopts a physical-time-driven midprice return with a latency correction $\lambda \in (0, h_1)$ at multiple horizons $h_1, h_2, \ldots, h_K \in (0, \infty)$. In that setup, the target is

$$\mathbf{r}_i^{mid} = \big(r_{i,1}^{mid},\, r_{i,2}^{mid},\, \ldots,\, r_{i,K}^{mid}\big),$$

where

$$r_{i,k}^{mid} = \frac{m\big(\tau_i^{\mathcal{P}^{(E)}} + h_k\big) - m\big(\tau_i^{\mathcal{P}^{(E)}} + \lambda\big)}{m\big(\tau_i^{\mathcal{P}^{(E)}} + \lambda\big)}.$$

In the present application, only one forecasting horizon is used, no latency correction is applied, and the return is computed based on the micro-price [21]. That is, the target variable is given by the micro-return as follows

$$\tilde{r}_i^{w,\delta} = \frac{p^w\big(\tau_i^{\mathcal{P}^{(E)}} + \delta\big) - p^w\big(\tau_i^{\mathcal{P}^{(E)}}\big)}{p^w\big(\tau_i^{\mathcal{P}^{(E)}}\big)}, \tag{1}$$

where $\delta > 0$ denotes the sampling period for the overarching hybrid model (see Section 10.1), and

$$p^w(t) = \frac{v_0^a(t)\, p_0^b(t) + v_0^b(t)\, p_0^a(t)}{v_0^a(t) + v_0^b(t)}.$$

This construction yields a continuous-valued micro-return that is directly compatible with time-series regression and volatility modeling.

As explained in Appendix C, the CNN-LSTM network takes whitened order flow arrays $\mathbf{OF}_{i-99:i}^{\mathcal{P}^{(E)}}$ as inputs and micro-returns $\tilde{r}_i^{w,\delta}$, as defined in expression (1), as outputs. The training loop aims to minimize the mean squared error loss

$$\text{Loss}(\text{CNNLSTM}, \tilde{\mathbf{r}}^{w,\delta}) = \frac{1}{N^{(E)}} \sum_{i=1}^{N^{(E)}} \left(\tilde{r}_i^{w,\delta} - \text{CNNLSTM}\left(\widetilde{\mathbf{OF}}_{i-99:i}^{\mathcal{P}^{(E)}} \right) \right)^2.$$

Approximate minimization of this objective function is carried out through the ADAM optimizer [29] with a learning rate of 10^{-4}, exponential decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$, a denominator offset of 10^{-8}, and a batch size of 32. The final weights are those yielding the smallest validation loss.

At each sampling time bar, the CNN-LSTM output is measurable with respect to the σ-algebra generated by past order-flow events. Consequently, when included as a regressor in the Hybrid AR-MAX–GARCH model, it satisfies the standard exogeneity conditions required for Conditional Maximum Likelihood Estimation (CMLE) under appropriate independence assumptions.

4.2 Feature Processing: The Backbone Module

The feature processing is conducted in three steps. First, the CNN-LSTM network is trained as described in Section 4.1. The estimated weights $\hat{\mathbf{W}}$ of the network are stored to define the embedding $\phi_{\hat{\mathbf{W}}}$, which maps the stationary order flow sequences to the latent representation of the network's penultimate layer. Second, all covariates are tested for stationarity to avoid spurious relationships [23, 46]. This step applies the Augmented Dickey–Fuller (ADF) [52, 24, 1], Phillips–Perron (PP) [46, 24, 1], and Kwiatkowski–Phillips–Schmidt–Shin (KPSS) [34] tests on every non-overlapping window, summarizing rejections to guide transformations or variable eliminations. Section D in the supplementary material provides further details on the assumptions, hypotheses, test statistics, and critical regions of the stationarity tests. Third, the correlation matrix of the remaining covariates is examined to identify and mitigate multicollinearity by removing variables whose pairwise correlations exceed a user-defined absolute threshold. Randomization is employed to avoid systematic bias in variable retention when strong correlation clusters are present. The highly correlated covariates are eliminated by means of a randomized greedy graph-based algorithm. These steps ensure that all covariates entering the adapter model satisfy the regularity conditions typically required to avoid spurious regression and to justify likelihood-based inference.

5 Proposed Hybrid ARMAX-GARCH-DeepLOB Model

In this work, a hybrid method is developed to incorporate generative features from the DeepLOB imputations into an econometric regression model for micro-price. This section details how this is achieved, highlighting the benefits of this approach across three factors: computational efficiency in the high-frequency settings of two-time-scale transfer learning; statistical estimation accuracy and analysis for uncertainty quantification of forecasts; and the interpretability of regression result structures.

Autoregressive Moving Average with Exogenous variables-Generalized Autoregressive Conditional Heteroskedasticity models, or ARMAX-GARCH models, are common in financial time series analysis as their structure allows one to approximate complex dynamics by implementing a dynamic dependence of endogenous values combined with exogenous variables while allowing for a conditionally heteroskedastic volatility structure which can capture volatility clustering and leverage effects. The process $\{r_j\}_{j\in\mathbb{Z}}$ is said to follow an ARMAX-GARCH process if it satisfies the following equation

$$\phi(B)\left(r_j - c - \mathbf{b}^T\mathbf{x}_j\right) = \theta(B)\varepsilon_j, \quad j \in \mathbb{Z}. \tag{2}$$

Here, $\phi(z) = 1-\phi_1 z-\phi_2 z^2-\cdots-\phi_p z^p$ and $\theta(z) = 1+\theta_1 z+\theta_2 z^2+\cdots+\theta_q z^q$ are the autoregressive and moving average polynomials, B is the backshift operator, and $\{\mathbf{x}_j\}_{j\in\mathbb{Z}}$ is a covariate strongly stationary process such that the innovations are independent of the history of the process. That is, ε_j is independent of $\{\mathbf{x}_j\}_{j=-\infty:j}$ for all $j \in \mathbb{Z}$. On its simplest form, the noise term follows a GARCH(P, Q) volatility structure. Precisely, let $\{\varepsilon_j\}_{j\in\mathbb{Z}}$ be a stochastic process satisfying

$$\begin{cases} \varepsilon_j = \sigma_j\,\eta_j, \\ \sigma_j^2 = \omega + \sum_{k=1}^{Q}\alpha_k\varepsilon_{j-k}^2 + \sum_{k=1}^{P}\beta_k\sigma_{j-k}^2, \end{cases} \tag{3}$$

where $\omega > 0$, $\alpha_k, \beta_k \geq 0$, and $\{\eta_j\}_{j\in\mathbb{Z}}$, is a sequence of i.i.d. random variables with zero mean and unit variance. The last equation can be rewritten as $\theta(B)\sigma_j^2 = \omega + \alpha(B)\varepsilon_j^2$ where $\alpha(z) := \sum_{k=1}^{Q}\alpha_k z^k$ and $\beta(z) := 1 - \sum_{k=1}^{P}\beta_k z^k$.

To ensure the stability of the processes in Eq. (2) and (3), and to guarantee the asymptotic efficiency of the CMLE estimators, further restrictions must be imposed on the roots of the polynomials $\alpha(\cdot)$, $\beta(\cdot)$, $\phi(\cdot)$, and $\theta(\cdot)$ (for a detailed technical discussion, see [40]). The dynamics of equation (3) can be re-expressed as

$$\mathbf{z}_j = \boldsymbol{\eta}_j + \mathbf{A}_j\mathbf{z}_{j-1}, \tag{4}$$

where[5]

$$\mathbf{z}_j = \begin{pmatrix} \varepsilon_j^2 & \varepsilon_{j-1}^2 & \cdots & \varepsilon_{j-Q+2}^2 & \varepsilon_{j-Q+1}^2 & \sigma_j^2 & \sigma_{j-1}^2 & \cdots & \sigma_{j-P+2}^2 & \sigma_{j-P+1}^2 \end{pmatrix}^T,$$

[5]For the sake of precision, note that $\boldsymbol{\eta}_j$ is a vector of length $P+Q$ whose first entry is $\omega\eta_j^2$, its $(Q+1)$-th entry is ω, and all other entries are zero.

$$\eta_j = \begin{pmatrix} \omega\eta_j^2 & 0 & \cdots & 0 & 0 & \omega & 0 & \cdots & 0 & 0 \end{pmatrix}^T,$$

$$\mathbf{A}_j = \begin{pmatrix} \alpha_1\eta_j^2 & \alpha_2\eta_j^2 & \cdots & \alpha_{Q-1}\eta_j^2 & \alpha_Q\eta_j^2 & \beta_1\eta_j^2 & \beta_2\eta_j^2 & \cdots & \beta_{P-1}\eta_j^2 & \beta_P\eta_j^2 \\ 1 & 0 & \cdots & 0 & 0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 & 0 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & 0 & 0 & \cdots & 0 & 0 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_{Q-1} & \alpha_Q & \beta_1 & \beta_2 & \cdots & \beta_{P-1} & \beta_P \\ 0 & 0 & \cdots & 0 & 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cdots & 0 & 0 & 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

This representation for the GARCH dynamic is useful as it allows one to define via the eigen structure of the transition matrix, the stability of the time series evolution. Consider the top Lyapunov exponent, which is defined as $\rho(\mathbf{A}) = \lim_{j\to\infty} \frac{1}{j}\mathbb{E}\left\{\log\|\mathbf{A}_j\mathbf{A}_{j-1}\cdots\mathbf{A}_1\|\right\}$ with $\|\cdot\|$ any matrix norm in the space of $(p+q)\times(p+q)$ matrices. Then, it can be shown that a necessary and sufficient condition for the existence of a strictly stationary solution to (3) is given by $\rho(\mathbf{A}) < 0$ [20, thm. 2.4.]. For $P = Q = 1$, the top Lyapunov exponent condition $\rho(\mathbf{A}) < 0$ reduces to $\mathbb{E}\left\{\log\left(\alpha\eta_1^2 + \beta\right)\right\} < 0$. In contrast, the condition $\alpha + \beta < 1$ is necessary and sufficient for second-order stationarity [20, thm. 2.5.].

Ergodicity is an important property that helps ensuring desirable properties of statistical estimators [20, see p. 367]. Determining ergodicity in a general GARCH(P, Q) process is often more complicated than establishing its stationarity. Early studies addressed mixing properties [5, 8]. A straightforward application of Theorem 2.8 from [53] demonstrates that $\rho(\mathbf{A}) < 0$ is sufficient to guarantee the ergodicity of the process $\{\varepsilon_j\}_{j\in\mathbb{Z}}$.

Now, assuming that $\phi(z)$ is invertible, the solution to (2) is given by

$$r_j = c + \mathbf{b}^T\mathbf{x}_j + \frac{\theta(B)}{\phi(B)}\varepsilon_j = f(\ldots,(\varepsilon_{j-1},\mathbf{x}_{j-1}),(\varepsilon_j,\mathbf{x}_j),(\varepsilon_{j+1},\mathbf{x}_{j+1}),\ldots).$$

If the joint process $\{(\varepsilon_j,\mathbf{x}_j)\}_{j\in\mathbb{Z}}$ is ergodic, then so is $\{r_j\}_{j\in\mathbb{Z}}$. Unfortunately, confirming the joint ergodicity of the process $\{(\varepsilon_j,\mathbf{x}_j)\}_{j\in\mathbb{Z}}$ does not simply follow from marginal ergodicity of $\{\varepsilon_j\}_{j\in\mathbb{Z}}$ and $\{\mathbf{x}_j\}_{j\in\mathbb{Z}}$. Although marginal ergodicity is a necessary condition, it is not sufficient for establishing joint ergodicity. A deeper investigation into the additional requirements for joint ergodicity lies beyond the scope of this article.

The Hybrid ARMAX-GARCH-DeepLOB model shares the structure of the ARMAX-GARCH model whilst incorporating a generative feature embedding using the CNN-LSTM DL model architecture to produce interaction time series covariates from the LOB structures which effectively produces a special construction of the covariate process to combine information from the OF and the LOB.

As explained in Section 3.2, the trading session will be partitioned into nested backbone, adapter, and sampling time bars denoted $B^B_{d,k}$, $B^A_{d,k,\ell}$, and $B^S_{d,k,\ell,j}$ respectively ($B^S_{d,k,\ell,j} \subseteq B^A_{d,k,\ell} \subseteq B^B_{d,k}$). Let the micro-return in the sampling bar $B^S_{d,k,\ell,j}$ be defined as

$$r^w\left(B^S_{d,k,\ell,j}\right) = \frac{p^w\left(\inf B^S_{d,k,\ell,j} + \delta\right) - p^w\left(\inf B^S_{d,k,\ell,j}\right)}{p^w\left(\inf B^S_{d,k,\ell,j}\right)},$$

where $\inf B^S_{d,k,\ell,j}$ denotes the start time (infimum) of the sampling bar interval.

Similarly, denote the set of covariates listed in Section 10.1 available after observing the sampling time bar $B^S_{d,k,\ell,j}$ as $\mathbf{x}_{d,k,\ell,j}$. The Hybrid ARMAX-GARCH-DeepLOB model assumes that the micro-return process follows an ARMAX-GARCH stochastic process, with the covariate term provided by the embedding $\phi_{\mathbf{W}}$ (where $\mathbf{W}$ denotes the generic network weights, as opposed to the estimated weights $\hat{\mathbf{W}}$ used during inference), encoding the features generated by the last layer of the CNN-LSTM neural network as explicitly defined in the Section C.3.1 of the Appendix. That is, for $j \in \mathbb{Z}$

$$\begin{cases} \phi(B)\left(r^w\left(B^S_{d,k,\ell,j}\right) - c - \mathbf{b}^T_{\boldsymbol{\omega}}\mathbf{x}_{d,k,\ell,j-1} - \mathbf{b}^T_{\mathbf{X}}\phi_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right)\right) = \theta(B)\varepsilon_{d,k,\ell,j}, \\ \varepsilon_{d,k,\ell,j} = \sigma_{d,k,\ell,j}\,\eta_{d,k,\ell,j}, \\ \beta(B)\sigma^2_{d,k,\ell,j} = \omega + \alpha(B)\varepsilon^2_{d,k,\ell,j}. \end{cases} \tag{5}$$

where $\phi(z)$ and $\theta(z)$ have roots lying outside the unit circle, $\{\eta_{d,k,\ell,j}\}$ is an i.i.d. sequence of random variables, and $i_j = \max\left\{i : \tau^{(E)}_i \in B^S_{d,k,\ell,1:j}\right\}$ is the index of the last message that took place before sampling time bar $B^S_{d,k,\ell,1:j}$. Let $\varepsilon_{d,k,\ell,j}$ be independent of the history of the joint covariate process $\left\{\left(\mathbf{x}_t, \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_t-99:i_t}\right)\right\}_{t=-\infty:j-1}$ to avoid biased estimators [24, Ch. 9]. A second option, called the parsimonious Hybrid ARMAX-GARCH-DeepLOB model, replaces the generative embedding $\phi_{\mathbf{W}}$ in Eq. (5) entirely with the raw CNN-LSTM output. This output is denoted by $\text{CNNLSTM}_{\mathbf{W}}$ to indicate its dependence on the neural network weights.

6 Two-Time-Scale Transfer Learning for Hybrid Models Construction

The proposed framework incorporates information from the OFS and the LOB to obtain micro-return forecasts. Let $\mathbb{F}^{\boldsymbol{\omega}} = \left(\mathcal{F}^{\boldsymbol{\omega}}(t)\right)_{t\geq 0}$ and $\mathbb{F}^X = \left(\mathcal{F}^X(t)\right)_{t\geq 0}$ be the completed filtrations generated by the OFS and LOB process, respectively. The optimal forecast of the micro-return given all the information in the OFS up to the sampling time bar $B^S_{d,k,\ell,j}$ is $\mathbb{E}\left\{r^w\left(B^S_{d,k,\ell,j-1}\right)|\mathcal{F}^{\boldsymbol{\omega}}\left(\sup B^S_{d,k,\ell,j-1}\right)\right\}$, where $\sup B^S_{d,k,\ell,j-1}$ denotes the end time (supremum) of the sampling bar interval. This forecast is typically infeasible due to the complexity of $\mathcal{F}^{\boldsymbol{\omega}}\left(\sup B^S_{d,k,\ell,j-1}\right)$. The optimal forecast can be approximated by combining information from the filtrations $\mathbb{F}^{\boldsymbol{\omega}}$ and $\mathbb{F}^X$ additively. Moreover, a correction can be applied to account for micro-return under- and over-reactions. Specifically

$$\mathbb{E}\left\{r^w\left(B^S_{d,k,\ell,j-1}\right)|\mathcal{F}^{\boldsymbol{\omega}}\left(\sup B^S_{d,k,\ell,j-1}\right)\right\} \approx c + \mathbf{b}^T_{\boldsymbol{\omega}}\mathbf{x}_{d,k,\ell,j-1} + \mathbf{b}^T_{\mathbf{X}}\phi_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right) + \sum_{k=1}^{\infty}\psi_k\varepsilon_{d,k,\ell,j-k}.$$

This approximation is heuristic and motivated by conditional projection arguments; no claim of optimality is made beyond first-order approximation under the assumed information reduction. This expression motivates the use of the Hybrid ARMAX-GARCH-DeepLOB model as given in Equation (5).

The Hybrid ARMAX-GARCH-DeepLOB employs features learned by the CNN-LSTM network to produce statistical forecasts for micro-returns. Though seemingly simple, several considerations must be addressed due to the context at hand. First, the extensive set of covariates generates multicollinearity which hampers the model stability if left unresolved. Second, the large sample size induces ill-posedness in the likelihood function which causes the failure of numerical algorithms used to compute the estimator of the Hybrid ARMAX-GARCH-DeepLOB model. Third, news affect the covariate structure which can inhibit the model's forecasting power. These issues motivate the Two-Time-Scale Transfer Learning (TTSTL) Framework for Hybrid Models construction.

Inspired by the complementary learning systems (CLS) theory—where the hippocampus enables rapid adaptation and the neocortex consolidates slow, stable structure—TTSTL couples a high-capacity *backbone* with a lightweight *adapter*, linked by an *interface* [45]. The backbone module (neocortex analogue) learns durable representations; the adapter module (hippocampus analogue) turns these representations into forecasts using Statistical Estimation Theory with online updates to react quickly to changing conditions. To maintain robustness, the interface enforces parsimony by learning a stable covariate set that governs the adapter's inputs.

To formalize the transfer learning perspective, we define the learning problem on each adapter time bar as a distinct local statistical task:

$$\mathcal{T}_{d,k,\ell} := \text{estimation of } \boldsymbol{\varphi} \text{ under local stationarity on } B^A_{d,k,\ell},$$

where $\boldsymbol{\varphi}$ is the parameter vector of the adapter module. The backbone learns a shared representation that is reused across these tasks. The transfer occurs through parameter combination and covariate restriction, not through shared loss functions. The remainder of this section proceeds as follows: Section 6.1 details the TTSTL architecture, and Section 6.2 describes the update scheme.

6.1 The TTSTL Model

Event driven Order Flow data are encoded by the backbone module via expert-selected features (see Section 10.1) and a deep representation of LOB dynamics extracted through the CNN-LSTM network (Section 4.1). Because retraining of the backbone is computationally intensive, prompt adaptation to market shifts cannot be achieved. To address this limitation, the adapter is implemented as a Hybrid ARMAX-GARCH–DeepLOB model designed for rapid online updates (Section 5). The high dimensionality of the backbone's output, however, can impair the adapter's performance, necessitating an interface for model subspace selection. Accordingly, the variable-selection technique described in Section 7.3 is applied to transform the backbone's representation into a reduced covariate set, thereby ensuring computational efficiency and forecasting precision.

Accurately capturing arbitrary non-stationary relationships within a single model is infeasible, as each trading session presents a continuum of heterogeneous dynamics that intensify over longer time bars. Adapter time bars, being much shorter, tend to exhibit more stable local patterns than backbone time bars. Framing the micro-return dynamics within each adapter time bar as a distinct estimation task naturally places the method within a transfer-learning paradigm.

The TTSTL framework captures locally stationary relationships by leveraging representations learned by a backbone module (see Figure 2) which comprises microstructural features (see Section 10.1), a generative embedding (see Section 4.1), stationarity testing, and covariance matrix analysis (see Section 4.2). The adapter model (Section 5) is retrained at the adapter time-bar frequency using Conditional Maximum Likelihood Estimation (CMLE; Section 7.1), with model parsimony controlled via model subspace selection (Section 7.3). Incoming MBO data are preprocessed into Event-Driven LOB data (denoted as EDLOB in Figure 2) and Physical Time-Driven Microstructural Features (denoted as TDMF in Figure 2), then stored and broadcast to all components. The backbone ingests EDLOB to train the CNN-LSTM network, while stationarity of the TDMF series is assessed to avoid spurious relationships[6]. After computing the covariance matrix and eliminating redundant variables, the backbone exports both the CNN-LSTM weights (W in Figure 2) and the full covariate set (F in Figure 2) (comprising the CNN-LSTM's final-layer features and the variables listed in Table S2 which pass the stationarity tests declared in Section 4.2). The interface then applies the procedure of Section 7.3 to reduce F to a parsimonious subset S (Figure 2), continuously monitoring S for regime shifts. Finally, S is passed to the adapter, which generates one-step-ahead forecasts.

[6] As noted in Section 4.1, CNN-LSTM inputs are order flow measures that remain stationary at the adapter frequency; hence, their learned features are not tested for stationarity.

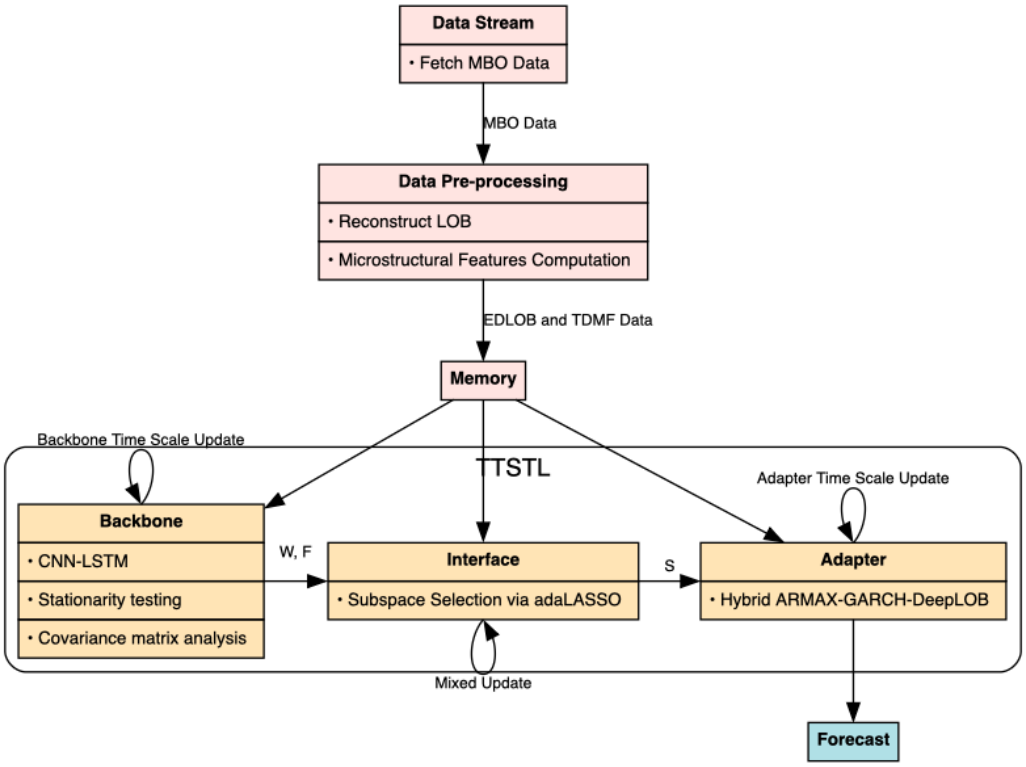


Figure 2: Two-Time-Scale Transfer Learning (TTSTL) framework. The diagram illustrates the interaction between the high-capacity deep learning backbone (slow-timescale feature learning), the interface module (feature embedding transfer and subspace selection via adaLASSO), and the econometric adapter module (fast-timescale calibration and parsimonious online forecasting).

Two operational modes are defined to manage computational cost: **Global Retrain** mode and **Local Adapt** mode (Table 2). In Global Retrain mode, all components—backbone, interface, and adapter—are fully updated overnight at the end of each backbone time bar. Specifically, the CNN-LSTM weights are re-estimated, stationarity tests and covariance analyses are performed, and interface hyperparameters are tuned. The updated backbone parameters and interface settings are then carried forward into the corresponding backbone bar on the following trading day. During Local Adapt mode, the backbone weights and selected covariate subspace remain fixed; the adapter executes recursive CMLE updates at each adapter time bar to refine forecasts, while the interface uses its fixed hyperparameters solely to monitor for shifts in regression structure.

Table 2: TTSTL Modes: Global Retrain vs. Local Adapt

Module	Global Retrain mode	Local Adapt mode
Backbone	Full update	Transfers Learned Weights Transfers Full Set of Covariates
Interface	Full update	Transfers Learned Hyperparameters
Adapter	Incremental Updates	Incremental Updates and Forecasts

6.2 The TTSTL Updating Scheme

This section describes the updating scheme. In Global Retrain mode, the weights of the CNN-LSTM update according to the scheme given by

$$\hat{\mathbf{W}}(B^B_{1:d,k}) = \text{Update}\left(\hat{\mathbf{W}}(B^B_{1:d-1,k}), B^B_{d,k}\right).$$

The update operator is intentionally left abstract, as the TTSTL framework is agnostic to the specific backbone optimization method. Training is performed with the Adam optimizer (Section 4.1). Within

each backbone time bar $B^B_{d,k}$, a contiguous validation interval is selected whose length is a fixed percentage of the bar; the remaining observations constitute the training set. To prevent temporal leakage, buffer windows are inserted immediately before and after the validation block and exclude them from both training and validation.

The interface module contains four hyperparameters of interest (See Sections 7.3 in the text and E in the appendix for further details)[7]: λ_1, λ_2, α and τ. In Global Retrain mode, λ_1 and α are selected via a cross-validation approach; λ_2 and τ are updated via a BIC approach. BIC is used heuristically to control model complexity within each backbone bar, rather than as a global consistency criterion. The resulting hyperparameters are frozen during the Local Adapt mode to detect shifts in regression structure. This procedure is carried out independently of previous time bars and thus the updating scheme can ignore information from other backbone time bars.

$$(\hat{\lambda}_1, \hat{\lambda}_2, \hat{\alpha}, \hat{\tau})(B^B_{1:d,k}) = (\hat{\lambda}_1, \hat{\lambda}_2, \hat{\alpha}, \hat{\tau})(B^B_{d,k}).$$

The model subspace learned by the interface as described in Section 7.3 goes along with the same scheme

$$S(B^B_{1:d,k}) = S(B^B_{d,k}).$$

The estimated CNN-LSTM weights and selected covariates give way to the reduced Hybrid ARMAX-GARCH-DeepLOB model

$$\begin{aligned}\phi(B)\Big(r^w\big(B^S_{d,k,\ell,j}\big) - c - \mathbf{b}^T_{\boldsymbol{\omega},S(B^B_{d-1,k})}\,\mathbf{x}_{d,k,\ell,j-1} \\ - \mathbf{b}^T_{\mathbf{X},S(B^B_{d-1,k})}\,\phi_{\hat{\mathbf{W}}(B^B_{1:d-1,k})}\Big(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\Big)\Big) = \theta(B)\,\varepsilon_{d,k,\ell,j}.\end{aligned}$$

Here, $r^w(B^S_{d,k,\ell,j})$ denotes the micro-return in the sampling time bar in parentheses, $\mathbf{b}_{\boldsymbol{\omega},\boldsymbol{S}}$ and $\mathbf{b}_{\mathbf{X},S}$ are the regression coefficients with a zero entry in the inactive covariates (i.e. not in S). The reduced Hybrid ARMAX-GARCH-DeepLOB model parameters $\boldsymbol{\varphi}_S = (c, \mathbf{b}_{\boldsymbol{\omega},\boldsymbol{S}}, \mathbf{b}_{\mathbf{X},S}, \boldsymbol{\phi}, \boldsymbol{\theta}, \omega, \boldsymbol{\alpha}, \boldsymbol{\beta})$ are updated at the adapter time bar frequency according to the scheme

$$\begin{aligned}\hat{\boldsymbol{\varphi}}_{S(B^B_{d-1,k})}\left(B^A_{d,k,1:\ell}\right) &= \text{Update}\left(\hat{\boldsymbol{\varphi}}_{S(B^B_{d-1,k})}\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right), \\ \hat{\boldsymbol{\varphi}}_{S(B^B_{d-1,k})}\left(B^A_{d,k,1}\right) &= \hat{\boldsymbol{\varphi}}_{S(B^B_{d-1,k})}\left(B^A_{d-1,k,1:L}\right),\end{aligned}$$

where the Update mechanism is described in Section 7.2. The hyperparameters learned by the interface module in the Global Update are employed in the Local Adapt mode to heuristically assess structural changes ex post.

7 Estimation

This section is devoted to the estimation procedures for the Hybrid ARMAX-GARCH-DeepLOB model. In Section 7.1 the conditional maximum likelihood estimator (CMLE) is formulated. Section 7.2 introduces recursive online combining rules designed to mitigate large-sample numerical instabilities, and Section 7.3 describes the interface module's model subspace selection and structural break detection via adaptive LASSO. Together, these estimation components complete the methodological framework for real-time forecasting under non-stationary market dynamics.

7.1 Conditional Maximum Likelihood Estimation in the Hybrid ARMAX-GARCH-DeepLOB Model

The ARMAX-GARCH model is well suited for the Conditional Maximum Likelihood Estimation (CMLE) method which in the notation of Equation (2) is defined as

$$\hat{\boldsymbol{\varphi}}_{\text{CMLE},T} = \arg\max_{\boldsymbol{\varphi}\in\Phi} \tilde{\mathcal{L}}_T(\boldsymbol{\varphi}).$$

[7] Appendix Section E is also available in repository form at `https://github.com/judmejiabe/case-study-adaLASSO.git`.

Here, $\tilde{\mathcal{L}}_T$ denotes the Conditional Log-Likelihood Function which is an approximation to the Log-Likelihood Function. The formal definition is given below

$$
\begin{aligned}
&\tilde{\mathcal{L}}_T(\boldsymbol{\varphi}) = \sum_{j=1}^{T} \log \tilde{f}_{\boldsymbol{\varphi}}\left(r_j | r_{1:j-1}, \mathbf{x}_{0:j-1}\right) = \sum_{j=1}^{T} \tilde{\ell}_j(\boldsymbol{\varphi}),\\
&\tilde{\ell}_j(\boldsymbol{\varphi}) = \log \tilde{f}_{\boldsymbol{\varphi}}\left(r_j | r_{1:j-1}, \mathbf{x}_{0:j-1}\right),\\
&\tilde{f}_{\boldsymbol{\varphi}}\left(r_j | r_{1:j-1}, \mathbf{x}_{0:j-1}\right) = \frac{1}{\tilde{\sigma}_j(\boldsymbol{\varphi})} g\left(\frac{\tilde{\varepsilon}_j(\boldsymbol{\varphi})}{\tilde{\sigma}_j(\boldsymbol{\varphi})}\right),\\
&\tilde{\varepsilon}_j(\boldsymbol{\varphi}) = r_j - \left(c + \mathbf{b}^T \mathbf{x}_{j-1} + \sum_{k=1}^{p} \phi_k \left(r_{j-k} - c - \mathbf{b}^T \mathbf{x}_{j-k-1}\right) + \sum_{j=1}^{q} \theta_k \tilde{\varepsilon}_{j-k}(\boldsymbol{\varphi})\right), \quad j \geq 1,\\
&\tilde{\varepsilon}_j(\boldsymbol{\varphi}) = c_j \in \mathbb{R}, \quad j = 0, -1, \ldots, -\max(q, Q),\\
&\tilde{\sigma}_j^2(\boldsymbol{\varphi}) = \omega + \sum_{k=1}^{Q} \alpha_k \tilde{\varepsilon}_{j-k}^2(\boldsymbol{\varphi}) + \sum_{k=1}^{P} \beta_k \tilde{\sigma}_{j-k}^2(\boldsymbol{\varphi}), \quad j \geq 1,\\
&\tilde{\sigma}_j^2(\boldsymbol{\varphi}) = c_j' \in [0, \infty), \quad j = 0, -1, \ldots, -P.
\end{aligned}
$$

Here g is the probability density function of η_j. Standard results imply that the influence of initialization constants c_j and c_j' vanishes asymptotically and is negligible for sufficiently long adapter windows. Several properties of $\hat{\boldsymbol{\varphi}}_{\mathrm{CMLE},T}$ are given in [40]. For instance, it is shown that the residuals $\tilde{\varepsilon}_j(\cdot)$ and $\tilde{\sigma}_j(\cdot)$ uniformly converge to their *infinite history* counterparts. This validates the approach taken in Section 8 to approximate the residuals. Some regularity conditions are sufficient to guarantee the asymptotic efficiency of the CMLE. These regularity conditions include, but are not limited to the polynomials $\phi(\cdot)$ and $\theta(\cdot)$ in equation (2) not having roots in common and such roots being outside the complex unit disk, $\alpha(\cdot)$ and $\beta(\cdot)$ in equation (3) not having roots in common, the top Lyapunov exponent being negative, and the joint process $\left\{\left(\varepsilon_j, \mathbf{x}_j\right)\right\}_{j \in \mathbb{Z}}$ being ergodic and strictly stationary. The reader is invited to consult [40] for the technical details. The main results in [40] are paraphrased below

Theorem 1. *Under proper conditions, it follows that*

$$\hat{\boldsymbol{\varphi}}_{\mathrm{CMLE},T} \xrightarrow[T \to \infty]{a.s.} \boldsymbol{\varphi}_0.$$

Theorem 2. *Under proper conditions, it follows that*

$$\sqrt{T}(\hat{\boldsymbol{\varphi}}_{\mathrm{CMLE},T} - \boldsymbol{\varphi}_0) \xrightarrow[T \to \infty]{\mathcal{L}} N\left(0, I_{\boldsymbol{\varphi}_0}^{-1}\right).$$

Moreover, the following outer product approximation holds

$$\frac{1}{T} \sum_{j=1}^{T} \nabla_{\boldsymbol{\varphi}}^T \tilde{\ell}_j(\boldsymbol{\varphi}_0) \nabla_{\boldsymbol{\varphi}} \tilde{\ell}_j(\boldsymbol{\varphi}_0) \xrightarrow[T \to \infty]{a.s.} I_{\boldsymbol{\varphi}_0}.$$

Although consistency and asymptotic normality are attractive estimator properties, they can become problematic in very large samples. By consistency, the probability that $\hat{\boldsymbol{\varphi}}_{CMLE,T}$ lies outside any arbitrarily small neighborhood of the true parameter $\boldsymbol{\varphi}_0$ vanishes as $T \to \infty$. Consequently, the likelihood surface concentrates ever more sharply around $\boldsymbol{\varphi}_0$, resembling a Dirac delta in the limit. In practical terms, this extreme peakiness renders the log-likelihood ill-conditioned and numerically unstable for standard maximization algorithms. Thus, paradoxically, the large-sample regime that guarantees optimal asymptotic behavior also undermines the feasibility of directly applying CMLE. This challenge is addressed in the next section.

7.2 Recursively Online Combining of CMLEs

To help overcome the instability of the numerical algorithms used for estimation present in the large sample setting and manage dynamic relationships arising from the underlying local stationarity of the process, the present work proposes to apply a recursive local estimation procedure. This procedure entails refining a global estimate with a local estimator according to an online update rule of the form

$$\hat{\boldsymbol{\varphi}}_S\left(B^A_{d,k,1:\ell}\right) = \text{Update}\left(\hat{\boldsymbol{\varphi}}_S\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right)$$

Inspired by the Bayesian model-averaging framework of [37], the update rules examined in this paper construct a weighted average of recent CMLE fits—hereafter referred to as combining formulas. By averaging fresh local estimates, these methods circumvent the instability often associated with recursive estimators. The specific combining rules utilized in this work are summarized below. These update rules are heuristic and not claimed to preserve asymptotic efficiency; rather, their primary role is to ensure numerical stabilization and rapid adaptation to local market dynamics.

- **Myopic Update:** If one believes that the stationarity of the process breaks beyond one adapter time bar, a reasonable alternative is to update the estimated parameter at the end of every adapter time bar
$$\text{Update}_{\text{M}}\left(\hat{\boldsymbol{\varphi}}_S\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right) = \hat{\boldsymbol{\varphi}}_S\left(B^A_{d,k,\ell}\right)$$

- **Simple Averaging:** If the sequence of estimators is close to i.i.d., an optimal choice would be to average the estimator sequence. That is,
$$\text{Update}_{\text{SA}}\left(\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right) = \frac{\ell-1}{\ell}\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right) + \frac{1}{\ell}\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,\ell}\right).$$

- **Power-Law Recency Weighting:** The myopic update and simple averaging are two extremes of the local stationarity spectrum. In the middle, the LOB dynamics shift at varying rates. In this situation, one possibility is to assign a larger weight to the estimation in the most recent windows. Motivated by [59], the following updating rule is proposed
$$\text{Update}_{\text{PL}}\left(\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right) = \frac{\sum_{\tilde{\ell}=1}^{\ell-1}\tilde{\ell}^{\alpha}}{\sum_{\tilde{\ell}=1}^{\ell}\tilde{\ell}^{\alpha}}\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right) + \frac{\ell^{\alpha}}{\sum_{\tilde{\ell}=1}^{\ell}\tilde{\ell}^{\alpha}}\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,\ell}\right).$$

- **Exponential Smoothing:** Another alternative to capture varying degrees of local stationarity is to forget the past at a geometric rate, see [15],
$$\text{Update}_{\text{ES}}\left(\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right), B^A_{d,k,\ell}\right) = (1-\alpha)\,\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right) + \alpha\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,\ell}\right).$$

7.3 Model Subspace Selection: The Interface Module

Including all potential covariates in the ARMAX–GARCH-DeepLOB specification risks overfitting and severe multicollinearity. To address this, a two-stage variable-selection algorithm is adopted. The technique is based on the Flexible adaLASSO of Medeiros and Mendes [38], which retains the oracle property and model-selection consistency even with non-Gaussian, heteroskedastic errors[8]. Appendix E presents a simulation study to illustrate the effectiveness of adaLASSO.

Within the TTSTL framework, the Interface module is designed to address both of these issues. The variable selection procedure is carried out in two stages: The first stage entails defining a surrogate model with similar dynamics to that of the Hybrid ARMAX-GARCH-DeepLOB and conduct variable selection at the adapter's time bar frequency. The adaLASSO procedure is applied under the assumption of approximate local stationarity within each adapter bar. In the second stage, the statistical significance of the selected variables is assessed by means of t-tests with heteroskedasticity correction. In this last step, the variables with a relatively higher statistical significance rate are subsequently passed to the adapter for iterative re-estimation and forecasting.

[8] Under some regularity conditions, of course.

Stage 1: Flexible adaLASSO. In the adapter time bar $B^A_{d,k,\ell}$, consider the following model which is a surrogate of the Hybrid ARMAX-GARCH-DeepLOB model defined in Equation (5).

$$\begin{cases} r^w\left(B^S_{d,k,\ell,j}\right) = c + \sum_{\nu=1}^{p^*} \psi_\nu r^w\left(B^S_{d,k,\ell,j-\nu}\right) + \mathbf{b}_{\boldsymbol{\omega}}^T \mathbf{x}_{d,k,\ell,j-1} + \mathbf{b}_{\mathbf{X}}^T \phi_{\hat{\mathbf{W}}\left(B^B_{1:d-1,k}\right)}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right) + \varepsilon_{d,k,\ell,j}, \\ \varepsilon_{d,k,\ell,j} = \sigma_{d,k,\ell,j}\,\eta_{d,k,\ell,j}, \\ \sigma^2_{d,k,\ell,j} = \omega + \sum_{\nu=1}^{Q} \alpha_\nu \varepsilon^2_{d,k,\ell,j-\nu} + \sum_{\nu=1}^{P} \beta_\nu \sigma^2_{d,k,\ell,j-\nu}. \end{cases} \tag{6}$$

Now focus on a penalization of the mean equation. Denote

$$\tilde{\mathbf{x}}_{d,k,\ell,j-1} = \begin{pmatrix} r^w\left(B^S_{d,k,\ell,j-1}\right) \\ \vdots \\ r^w\left(B^S_{d,k,\ell,j-p^*}\right) \\ \mathbf{x}_{d,k,\ell,j-1}, \\ \phi_{\hat{\mathbf{W}}\left(B^B_{1:d-1,k}\right)}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right) \end{pmatrix},$$

and collect the corresponding coefficients into $\tilde{\mathbf{b}} = \left(\psi_1, \ldots, \psi_{p^*}, \mathbf{b}_{\boldsymbol{\omega}}^T, \mathbf{b}_{\mathbf{X}}^T\right)^T$, so that $r^w\left(B^S_{d,k,\ell,j}\right) = c + \tilde{\mathbf{b}}^T \tilde{\mathbf{x}}_{d,k,\ell,j-1} + \varepsilon_{d,k,\ell,j}$. Collecting the observations and covariates into

$$\begin{aligned} \mathbf{r}\left(B^A_{d,k,\ell}\right) &= \left(r^w\left(B^S_{d,k,\ell,1}\right), r^w\left(B^S_{d,k,\ell,2}\right), \ldots, r^w\left(B^S_{d,k,\ell,J}\right)\right)^T, \\ \tilde{\mathbf{X}}\left(B^A_{d,k,\ell}\right) &= \left(\tilde{\mathbf{x}}_{d,k,\ell,0}, \tilde{\mathbf{x}}_{d,k,\ell,1}, \ldots, \tilde{\mathbf{x}}_{d,k,\ell,J}\right)^T \end{aligned}$$

such that

$$\mathbf{r}\left(B^A_{d,k,\ell}\right) = c\mathbf{1} + \tilde{\mathbf{X}}\left(B^A_{d,k,\ell}\right)\tilde{\mathbf{b}} + \boldsymbol{\varepsilon}_{d,k,\ell}$$

The Flexible adaLASSO operates by shrinking or expanding a variable-specific ℓ_1 penalization term. This dilation is given by the relevance of the variables as determined by an initial elastic-net fit. Formally, the flexible adaLASSO is implemented in two steps as follows.

(i) *Initial elastic-net fit.* Obtain a preliminary estimate via

$$\hat{\tilde{\mathbf{b}}}^{(I)}\left(B^A_{d,k,\ell}\right) = \arg\min_{c,\tilde{\mathbf{b}}} \left\{ \left\| \mathbf{r} - c\mathbf{1} - \tilde{\mathbf{X}}\left(B^A_{d,k,\ell}\right)\tilde{\mathbf{b}} \right\|_2^2 + \lambda_1 \left(\alpha \|\tilde{\mathbf{b}}\|_1 + \frac{1-\alpha}{2} \|\tilde{\mathbf{b}}\|_2^2 \right) \right\}.$$

The elastic-net penalty mitigates multicollinearity by combining ℓ_1 and ℓ_2 shrinkage.

(ii) *Adaptive re-weighting.* Define the weights $w_k := \left| \left[\hat{\tilde{\mathbf{b}}}^{(I)}_k\left(B^A_{d,k,\ell,j}\right) \right]_k \right|^{-\tau}$, with $\tau > 0$, and solve the weighted LASSO

$$\hat{\tilde{\mathbf{b}}}_{\mathrm{FaL}}\left(B^A_{d,k,\ell}\right) = \arg\min_{c,\tilde{\mathbf{b}}} \left\{ \left\| \mathbf{r} - c\mathbf{1} - \tilde{\mathbf{X}}\left(B^A_{d,k,\ell}\right)\tilde{\mathbf{b}} \right\|_2^2 + \lambda_2 \sum_k w_k \, |\tilde{b}_k| \right\}.$$

Under the conditions described in [38], this two-step procedure consistently identifies the true nonzero coefficients and achieves the oracle property [38].

As seen in Appendix E the adaLASSO effectiveness depends on the ability of the initial fit to discriminate relevant variables from irrelevant variables. Therefore the initial elastic-net fit parameters λ_1 and α must be selected carefully –as well as the adaptive re-weighting fit ones. In [38], the authors recommend optimizing the BIC on a hyperparameter grid. Given the sheer amount of data, conducting adaLASSO

on a fine four dimensional grid of $(\lambda_1, \lambda_2, \alpha, \tau)$ becomes prohibitively expensive. To mitigate this, a train/validation partition is created to measure the quality of the initial elastic-net fit hyperparameters, yielding $\hat{\lambda}_1\left(B^B_{d,k}\right)$ and $\hat{\alpha}\left(B^B_{d,k}\right)$. Fixing these hyperparameters, select $\hat{\lambda}_2\left(B^B_{d,k}\right)$ and $\hat{\tau}\left(B^B_{d,k}\right)$ from a grid by minimizing the average BIC across the adapter time bars contained in $B^B_{d,k}$. This process is only carried out in Global Retrain mode. In the Local Adapt mode, transfer the learned hyperparameters from the aligned backbone time bar of the previous day. Regime shifts are flagged when the selected covariate set or coefficient significance profile changes persistently across adapter bars.

As seen in Appendix E the adaLASSO effectiveness depends on the ability of the initial fit to discriminate relevant variables from irrelevant variables. Therefore the initial elastic-net fit parameters λ_1 and α must be selected carefully –as well as the adaptive re-weighting fit ones. In [38], the authors recommend optimizing the BIC on a hyperparameter grid. Given the sheer amount of data, conducting adaLASSO on a fine four dimensional grid of $(\lambda_1, \lambda_2, \alpha, \tau)$ becomes prohibitively expensive. To mitigate this, a train/validation partition is created to measure the quality of the initial elastic-net fit hyperparameters, yielding $\hat{\lambda}_1\left(B^B_{d,k}\right)$ and $\hat{\alpha}\left(B^B_{d,k}\right)$. Fixing these hyperparameters, select $\hat{\lambda}_2\left(B^B_{d,k}\right)$ and $\hat{\tau}\left(B^B_{d,k}\right)$ from a grid by minimizing the average BIC across the adapter time bars contained in $B^B_{d,k}$. This process is only carried out in Global Retrain mode. In the Local Adapt mode, transfer the learned hyperparameters from the aligned backbone time bar of the previous day. Regime shifts are flagged when the selected covariate set or coefficient significance profile changes persistently across adapter bars.

In the Global Retrain mode, the learned Hyperparameters are used as described in Stage 2 below to finalize the model subspace selection process. In the Local Adapt mode, the learned hyperparameters are transferred to help assess the model's structure stability.

Stage 2: Flexible adaLASSO-OLS Screening. Theorem 3 in [38] hints that the Flexible adaLASSO possesses the oracle property which establishes that

$$\sqrt{J}\left[\hat{\tilde{\mathbf{b}}}_{\mathrm{FaL},S}\left(B^A_{d,k,\ell}\right) - \tilde{\mathbf{b}}_S\right] \xrightarrow[J\to\infty]{\mathcal{L}} \sqrt{J}\left[\hat{\tilde{\mathbf{b}}}_{OLS,S}\left(B^A_{d,k,\ell}\right) - \tilde{\mathbf{b}}_S\right],$$

where $\hat{\tilde{\mathbf{b}}}_{\mathrm{FaL},S}\left(B^A_{d,k,\ell}\right)$ is the subset of non-zero coefficients in $\hat{\tilde{\mathbf{b}}}_{\mathrm{FaL,S}}\left(B^A_{d,k,\ell}\right)$[9] and $\hat{\tilde{\mathbf{b}}}_{OLS,S}\left(B^A_{d,k,\ell}\right)$ is the S-restricted ordinary least squares estimator of the model in Equation (6) computed on the covariates selected by the flexible adaLASSO. Under mild conditions on the covariate process and residuals (see Proposition 8.3. in [24]), it can be shown that

$$\sqrt{J}\left[\hat{\tilde{\mathbf{b}}}_{OLS,S}\left(B^A_{d,k,\ell}\right) - \tilde{\mathbf{b}}_S\right] \xrightarrow[J\to\infty]{\mathcal{L}} N(\mathbf{0}, V_S),$$

where $\mathbf{I}$ is the identity matrix of size the cardinality of the set of selected variables S,

$$V_S\left(B^A_{d,k,\ell}\right) = \mathbf{Q}_S^{-1}\left(B^A_{d,k,\ell}\right)\mathbf{\Omega}_S\left(B^A_{d,k,\ell}\right)\mathbf{Q}_S^{-1}\left(B^A_{d,k,\ell}\right),$$

and

$$\begin{aligned}
\mathbf{Q}_S\left(B^A_{d,k,\ell}\right) &= \lim_{J\to\infty}\frac{1}{J}\tilde{\mathbf{X}}_S^T(B^A_{d,k,\ell})\,\tilde{\mathbf{X}}_S(B^A_{d,k,\ell}) \\
\mathbf{\Omega}_S\left(B^A_{d,k,\ell}\right) &= \lim_{J\to\infty}\frac{1}{J}\sum_{j=1}^{J}\mathbb{E}\left\{\tilde{\mathbf{x}}_{S,d,k,\ell,j}\tilde{\mathbf{x}}_{S,d,k,\ell,j}^T\varepsilon^2_{d,k,\ell,j}\right\}.
\end{aligned}$$

Here, the subscript S is used to denote that the base expression is restricted to the variables in S. A consistent estimator of $V_S\left(B^A_{d,k,\ell}\right)$, called the heteroskedastic-consistent covariance matrix, is given by

$$\begin{aligned}
\hat{V}_{\mathrm{HC},S}\left(B^A_{d,k,\ell}\right) = J\cdot&\left(\tilde{\mathbf{X}}_S^T(B^A_{d,k,\ell})\,\tilde{\mathbf{X}}_S(B^A_{d,k,\ell})\right)^{-1} \\
&\times\left(\sum_{j=1}^{J}\tilde{\mathbf{x}}_{S,d,k,\ell,j}\,\tilde{\mathbf{x}}_{S,d,k,\ell,j}^T\,\hat{\varepsilon}^2_{d,k,\ell,j}\right) \\
&\times\left(\tilde{\mathbf{X}}_S^T(B^A_{d,k,\ell})\,\tilde{\mathbf{X}}_S(B^A_{d,k,\ell})\right)^{-1}.
\end{aligned}$$

[9] In this particular context, $x\in\mathbb{R}$ is non-zero if $|x|>\varepsilon$ for some small $\varepsilon>0$

In Global Retrain mode, the adaLASSO–OLS screening is performed by conducting heteroskedasticity-consistent t–tests on each adapter bar. For covariate index ν, the rejection criterion is

$$t_{\nu,d,k,\ell} = \sqrt{J}\,\frac{\left|\,[\hat{\tilde{\mathbf{b}}}_{\mathrm{OLS},S(B^A_{d,k,\ell})}(B^A_{d,k,\ell})]_\nu\right|}{\left[\hat{V}_{\mathrm{HC},\,S(B^A_{d,k,\ell})}(B^A_{d,k,\ell})\right]^{1/2}_{\nu\nu}} > z_{1-\alpha/2},$$

where $z_{1-\alpha/2}$ denotes the $1-\frac{\alpha}{2}$ quantile of the standard normal distribution. The empirical rejection frequency is then computed across the ℓ adapter bars, and covariates whose frequency exceeds a user-specified threshold are retained. The selected predictor set at the end of backbone bar $B^B_{d,k}$ is denoted $S(B^B_{d,k})$ and passed to the adapter in Local Adapt mode. A practical rule of thumb for the screening threshold is given by assuming that the variable-specific t-tests are i.i.d. Bernoulli variables with success probability α and then taking a high quantile of the Binomial distribution as ruled by the normal approximation. That is, select the ν-th variable whenever

$$\left|\left\{\ell : t_{\nu,d,k,\ell} > z_{1-\alpha/2}\right\}\right| > \alpha\, L \;+\; z_{1-\gamma}\,\sqrt{L\,\alpha\,(1-\alpha)},$$

where $\gamma > 0$ is a small constant. This screening procedure yields a set of covariates $S\left(B^B_{d,k,\ell,j}\right)$ to be used by the adapter module in the next trading session aligned backbone time bar in the Local Adapt mode.

8 Forecasting

In Local Adapt mode, a rolling-origin update scheme is employed [39], whereby forecasts advance one sampling bar at a time. At each sampling bar, the estimated CNN–LSTM weights $\mathbf{W}$, Hybrid ARMAX–GARCH–DeepLOB parameters $\boldsymbol{\varphi}$, and covariate subset S are used to produce a one-step-ahead micro-return forecast. Once an adapter bar (comprising several sampling bars) completes, the adapter's parameters are updated via the recursive combining rules of Section 7.2, and the process continues into the next bar. For simplicity, the first adapter time bar is used to start the recursions of the Hybrid ARMAX-GARCH model forecasting equations. Therefore no forecasts are produced for the first adapter time bar.

Figure 3 illustrates the rolling-origin update procedure within a single backbone bar divided into three adapter bars, each containing four sampling bars. Time progresses from left to right and from top row to bottom row. In the figure, dark blue cells indicate sampling bars for which data have been fully assimilated into the adapter's parameter estimation. Light blue cells denote sampling bars for which observations have been recorded and used to generate the upcoming (orange) forecast but whose data have not yet entered the parameter update. Orange cell mark the sampling time bars for which the one-step-ahead forecast is being produced. White cells correspond to sampling bars that have not yet received any data and for which no forecast is produced at the time.

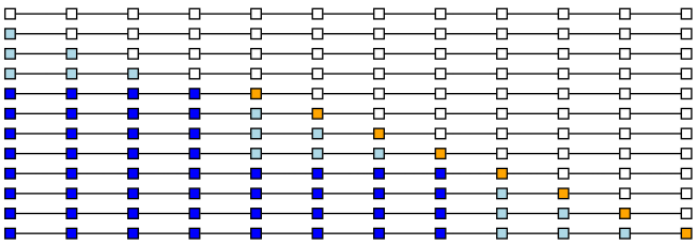


Figure 3: Rolling-origin update algorithm illustration on one backbone time bar.

The forecasts for the mean and volatility are produced according to a Hybrid ARMAX-GARCH-DeepLOB model. Let $d_{S,W}(B^S_{d,k,\ell,j})$ denote the part of the conditional mean which only depends on the constant term and exogenous variables. That is,

$$d(B^S_{d,k,\ell,j}) = c + \mathbf{b}_{\boldsymbol{\omega}}^T\,\mathbf{x}_{j-1} + \mathbf{b}_{\mathbf{X}}^T\,\phi_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right).$$

The optimal forecast for the mean and the squared volatility are given by the conditional expectation and variance respectively:

$$\mathbb{E}\left\{r^w(B^S_{d,k,\ell,j})\middle|\mathcal{F}_{d,k,\ell,j-1}\right\} = d\left(B^S_{d,k,\ell,j}\right) + \sum_{\nu=1}^{p}\phi_\nu\left(r^w(B^S_{d,k,\ell,j-\nu}) - d\left(B^S_{d,k,\ell,j-\nu}\right)\right) + \sum_{\nu=1}^{q}\theta_\nu\varepsilon_{d,k,\ell,j-\nu},$$

$$\mathrm{Var}\left(r^w\left(B^S_{d,k,\ell,j}\right)\middle|\mathcal{F}_{d,k,\ell,j-1}\right) = \omega + \sum_{\nu=1}^{Q}\alpha_\nu\varepsilon^2_{d,k,\ell,j-\nu} + \sum_{\nu=1}^{P}\beta_\nu\sigma^2_{d,k,\ell,j-\nu}.$$

Then to produce the forecast values for $r^w(B^S_{d,k,\ell,j})$, the parameter vector $\boldsymbol{\varphi}$ is replaced by its estimate $\hat{\boldsymbol{\varphi}}_{S\left(B^B_{d-1,k}\right)}\left(B^A_{d,k,1:\ell-1}\right)$ and the residuals $\varepsilon_{d,k,\ell,j}$ are estimated with $\tilde{\varepsilon}_{d,k,\ell,j}\left(\hat{\boldsymbol{\varphi}}_{S\left(B^B_{d-1,k}\right)}\left(B^A_{d,k,1:\ell-1}\right)\right)$ (see Section 7.1) using the data in the previous adapter time bar as a burn-in to help guarantee convergence of the estimated residuals.

9 Performance Measures

The selection of performance metrics in forecasting is a topic of contentious debate though empirical evidence suggests that most reasonable measures tend to rank systems similarly [16]. To evaluate the predictive accuracy of the proposed framework, this study employs the Relative Mean Absolute Error (RelMAE), computed for each adapter time bar. This metric quantifies the performance of the model relative to a benchmark by normalizing the Mean Absolute Error (MAE) of the model against that of a baseline. For a given adapter time bar $B^A_{d,k,\ell}$, it is defined as:

$$\mathrm{RelMAE}\left(B^A_{d,k,\ell}\right) = \frac{\sum_{j=1}^{J}\left|\tilde{\varepsilon}_{d,k,\ell,j}\left(\hat{\boldsymbol{\varphi}}\left(B^A_{d,k,1:\ell-1}\right)\right)\right|}{\sum_{j=1}^{J}\left|\hat{\varepsilon}^b_{d,k,\ell,j}\right|} = \frac{\mathrm{MAE}\left(B^A_{d,k,\ell}\right)}{\mathrm{MAE}_b\left(B^A_{d,k,\ell}\right)},$$

where $\tilde{\varepsilon}_{d,k,\ell,j}$ represents the residuals of the Hybrid ARMAX-GARCH-DeepLOB model, and $\hat{\varepsilon}^b_{d,k,\ell,j}$ represents the residuals of the chosen baseline.

Two distinct baselines are employed to assess different aspects of model performance:

1. **Naive Baseline ($\hat{\varepsilon}^{b,1}$):** To assess the value added by the modeling framework over a trivial predictor, the average micro-return from the corresponding backbone bar of the previous trading session is used as the naive benchmark. The residuals for this baseline are given by:

$$\hat{\varepsilon}^{b,1}_{d,k,\ell,j} = r^w\big(B^S_{d,k,\ell,j}\big) \;-\; \bar{r}^w\big(B^B_{d-1,k}\big),$$

$$\bar{r}^w\big(B^B_{d-1,k}\big) = \frac{1}{JL}\sum_{\ell=1}^{L}\sum_{j=1}^{J} r^w\big(B^S_{d-1,k,\ell,j}\big).$$

2. **Neural Baseline ($\hat{\varepsilon}^{b,2}$):** To quantify the adapter module ability to adapt the neural network imputation, the raw imputation generated by the CNN-LSTM backbone is used as a baseline for complementary analysis. Its residuals are defined as:

$$\hat{\varepsilon}^{b,2}_{d,k,\ell,j} = r^w\big(B^S_{d,k,\ell,j}\big) \;-\; \mathrm{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{j-1}-99:i_{j-1}}\right).$$

10 Data

10.1 Data Collection

Choosing the sampling frequency remains an open design choice [17]. The workflow in Figure 2 relies on two sampling scales: An event-driven stream providing order-flow imbalances (OFIs) and the micro-price; and a physical-time series containing all features listed in Table 3.

For the physical-time series,the sampling time bar width is set to $\delta = 100$ units of time (uot) in synthetic experiments and to $\delta = 2$ seconds in market-data experiments (see Section 10.2). Table 3 summarizes the measurements produced by the market-replay algorithm plus the VIX index measurements. To lighten notation, unless stated otherwise, the subscript j denotes the feature evaluated on sampling bar $B^S_{d,k,\ell,j}$. For stock features—i.e., covariates measured at a single time point—the observation is timestamped at the right endpoint of the corresponding sampling bar.

Table 3: List of extracted microstructural measures. In this table, $\star \in \{a, b\}$ denotes the side of the LOB, $h \in \{0, 1, \ldots, H\}$ denotes the number of ticks away from best bid or ask values, and $t \in \{1, \ldots, kN\}$ corresponds to the sequence in the time partition $\mathcal{P}^{(U)}_\delta = \{\delta j\}_{t=1}^{kN}$.

Feature Name	**Notation**	**Definition**	**References**
l-th best bid (ask)	$p^\star_{l,j}$	l-th best price on side $\star$	[7]
Available volume on side $\star$ and depth l	$v^\star_{l,j}$	Volume sitting at the l-th best price on side $\star$.	[7]
Spread	s_j	$p^b_{0,j} - p^a_{0,j}$.	[25, 19]
Micro-price	p^w_j	$\frac{v^a_{0,j}p^b_{0,j} + v^b_{0,j}p^a_{0,j}}{v^a_{0,j}+v^b_{0,j}}$.	[21]
Actual LOB depth	$\Xi^\star_j$	$\left\lvert p^\star_{L,j} - p^\star_{0,j} \right\rvert$.	[7]
Imbalance in trading volume	ρ_j	$\frac{v^b_{0,j}-v^a_{0,j}}{v^b_{0,j}+v^a_{0,j}}$.	[10]
Bid order flow	$OF^b_{l,j}$	$\begin{cases} v^b_{l,j} & p^b_{l,j} > p^b_{l,j-1}, \\ v^b_{l,j} - v^b_{l,j-1} & p^b_{l,j} = p^b_{l,j-1}, \\ -v^b_{l,j-1} & p^b_{l,j} < p^b_{l,j-1}. \end{cases}$	[14, 31]
Ask order flow	$OF^a_{l,j}$	$\begin{cases} v^a_{l,j} & p^a_{l,j} < p^a_{l,j-1}, \\ v^a_{l,j} - v^a_{l,j-1} & p^a_{l,j} = p^a_{l,j-1}, \\ -v^a_{l,j-1} & p^a_{l,j} > p^a_{l,j-1}. \end{cases}$	[14, 31]
Order flow imbalance	$OFI_{l,j}$	$\sum_{i \in I_j} OF^{b,\mathcal{P}^{(E)}}_{l,i} - OF^{a,\mathcal{P}^{(E)}}_{l,i}$.	[14, 31]
Roll's measure	R_j	$2\sqrt{\left\lvert \hat{cov}(\Delta \mathbf{p}_j, \Delta \mathbf{p}_{j-1}) \right\rvert}$.	[51, 19]
Roll's impact measure	$\tilde{R}_j$	$\frac{R_j}{p_{i^*_j} V_{i^*_j}}$.	[51, 19]
Kyle's λ	λ_j	$\frac{p_{i^*_j} - p_{i^*_j - W}}{\sum_{k=1}^W b_{i^*_j - W + k} V_{i^*_j - W + k}}$.	[35, 19]
Modified Amihud's measure	λ^A_j	$\frac{1}{W}\sum_{k=1}^W \frac{\left\lvert r_{i^*_j - W + k} \right\rvert}{p_{i^*_j - W + k} V_{i^*_j - W + k}}$.	[2, 19]
Volume-synchronized probability of informed trading	$VPIN_j$	$\frac{1}{W}\sum_{k=1}^W \frac{\left\lvert V^S_{i^*_j - W + k} - V^B_{i^*_j - W + k} \right\rvert}{V_{i^*_j - W + k}}$.	[18, 19]
CBOE volatility index	$VIX^{(\tau)}_j$	VIX Index, $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$.	[19]
Traded volume	TV_j	Volume executed on $B^S_{d,k,\ell,j}$. Including hidden orders.	[36]
Number of trades	$N_{T,j}$	Count of trades on $B^S_{d,k,\ell,j}$. Including executions against hidden orders.	[36]
Volume added inside the spread	$G^\star_{A,j}$	Volume added inside the spread on side $\star$ on $B^S_{d,k,\ell,j}$.	[36]
Number of add orders inside the spread	$N^\star_{G,A,j}$	Count of add orders inside the spread on side $\star$ on $B^S_{d,k,\ell,j}$.	[36]

Table 3 (continued)

Feature Name	Notation	Definition	References
Volume added h ticks away from best bid (ask)	$A^{\star}_{h,j}$	Volume added at h ticks below (above) the best bid (ask) on $B^{S}_{d,k,\ell,j}$.	[36]
Number of add orders h ticks away from best bid (ask)	$N^{\star}_{A,h,j}$	Count of add orders added h ticks below (above) the best bid (ask) on $B^{S}_{d,k,\ell,j}$	[36]
Volume cancelled h ticks away from best bid (ask)	$C^{\star}_{h,j}$	Volume cancelled h ticks below (above) the best bid (ask) on $B^{S}_{d,k,\ell,j}$	[36]
Number of cancel orders h ticks away from best bid (ask)	$N^{\star}_{C,h,j}$	Count of cancel orders h ticks below (above) the best bid (ask) on $B^{S}_{d,k,\ell,j}$	[36]

Some features in Table 3 require additional clarification as different sampling schemes interact. To compute the order flow imbalance l ticks away from touch, compute the OFs under the all-events partition $\mathcal{P}^{(E)}$ and denote these as $OF^{\star,\mathcal{P}^{(E)}}_{l,i}$, $\star \in \{a, b\}$. Then compute the OFI l ticks away from best bid and ask as

$$OFI^{\mathcal{P}^{(U)}_{\delta}}_{l,j} = \sum_{i \in I_j} OF^{b,\mathcal{P}^{(E)}}_{l,i} - OF^{a,\mathcal{P}^{(E)}}_{l,i},$$

where $I_j = \{i : \tau^{(E)}_i \in B^{S}_{d,k,\ell,j}\}$.

Other market microstructure measures are computed on a sequence of trade prices such as the Roll's measure, the Roll's impact measure, Kyle's λ, the modified Amihud's measure and the VPIN. The Roll's measure is defined as

$$R_j = 2\sqrt{\left|\hat{cov}(\Delta \mathbf{p}_j, \Delta \mathbf{p}_{j-1})\right|},$$

where $\Delta \mathbf{p}_j$ represents the last W price differences of trades that occurred before the end of $B^{S}_{d,k,\ell,j}$. Specifically, consider the random partition associated with the trade sampling scheme $\mathcal{P}^{(T)} = \{\tau^{(T)}_i\}^{N^{(T)}}_{i=1}$ and the corresponding trade price sequence $\{p_i\}^{N^{(T)}-1}_{i=1}$. Let $i^*_j = \max\{i : \tau^{(T)}_i \leq \delta j\}$. For a lookback window W, define

$$\Delta \mathbf{p}_j = (p_{i^*_j - W+1} - p_{i^*_j - W}, p_{i^*_j - W+2} - p_{i^*_j - W+1}, \ldots, p_{i^*_j} - p_{i^*_j - 1}).$$

Here, $\hat{cov}$ denotes the sample covariance.

The Roll's impact measure, given by

$$\tilde{R}_j = \frac{R_j}{p_{i^*_j} V_{i^*_j}},$$

is computed by dividing the most recent measurement of the Roll's measure by the product of the latest trade price and trade volume. Here, $\{V_i\}^{N^{(T)}-1}_{i=1}$ denotes the sequence of trade volumes.

Kyle's λ, defined as

$$\lambda_j = \frac{p_{i^*_j} - p_{i^*_j - W}}{\sum^{W}_{k=1} b_{i^*_j - W+k} V_{i^*_j - W+k}},$$

is the ratio of the difference in trade prices over the last W time steps and the net traded volumes. Here, $b_i = \text{sign}(p_i - p_{i-1})$ denotes the sign of the price change.

The modified Amihud's measure is defined as

$$\lambda_j^A = \frac{1}{W}\sum_{k=1}^{W}\frac{\left|r_{i_j^*-W+k}\right|}{p_{i_j^*-W+k}V_{i_j^*-W+k}},$$

where $r_i = \frac{p_i - p_{i-1}}{p_{i-1}}$ is the trade price return.

The volume-synchronized probability of informed trading (VPIN) is defined as

$$VPIN_j = \frac{1}{W}\sum_{k=1}^{W}\frac{\left|V^S_{i_j^*-W+k} - V^B_{i_j^*-W+k}\right|}{V_{i_j^*-W+k}},$$

where $V_i^B = V_i\Phi\left(\frac{p_i - p_{i-1}}{\sigma_{\Delta \mathbf{p}_j}}\right)$, $V_i^S = V_i - V_i^B$, and $\sigma_{\Delta \mathbf{p}_j}$ corresponds to the standard deviation of $\Delta \mathbf{p}_j$.

In summary, two datasets are derived from each trading session, each serving a distinct purpose. The first is an event-driven dataset containing event-driven order flow data and micro-return data, which supplies training inputs to the CNN-LSTM model. The second is a physical-time-driven dataset which provides additional covariates for the Hybrid ARMAX-GARCH-DeepLOB model.

10.2 Dataset

Following [7], stocks with larger tick sizes are prioritized, as mid-price returns can be imputed more accurately for such instruments. In this context, larger ticks translate to smaller time-weighted average spreads. To identify the optimal instrument, the time-weighted average spreads of the stable NASDAQ 100 constituents[10] were computed for the trading sessions on July 1st, 2022, June 30th, 2023, and July 1st, 2024. These rankings were subsequently averaged, identifying INTC as the instrument with the smallest time-weighted average spread. Consequently, MBO data for INTC was extracted for the months of May and June 2024 using the Databento API. It is noted that while INTC is used here as a representative, highly liquid NASDAQ asset, the results should be interpreted as illustrative of the methodology's capabilities rather than universal across all asset classes.

Additionally, VIX index data was retrieved via the Polygon API[11]. A market replay algorithm, which consists of taking the MBO data and reconstructing the LOB, was applied to record event-driven order flow features and generate the physical time-driven endogenous features[12].

This process resulted in two distinct datasets per trading session: an event-driven order flow dataset used for the backbone module global update, and a physical time-driven dataset of endogenous features. To ensure data quality, exclusion criteria were applied to avoid contamination from special market events. Specifically, days containing the following were excluded:

- Corporate actions: Ex-dividend dates, stock splits or reverse splits, share buybacks and rights issues, spin-offs, mergers, or acquisitions
- Earnings and company-specific news: Quarterly earnings announcements and guidance revisions, joint ventures, large partnerships or asset sales, regulatory actions or class-action lawsuits, product launches or major technological disclosures
- Derivative market events: Monthly options expirations (OPEX), quarterly triple witching weeks (stock index futures, index options, and single-stock options expiry), index futures rollovers (e.g., NASDAQ-100, S&P 500), index and ETF quarterly rebalancing
- Scheduled announcements: U.S. Employment Situation (non-farm payrolls), Consumer Price Index (CPI), Producer Price Index (PPI), PCE releases, FOMC interest rate decisions and press conferences, other major macroeconomic indicators (e.g., GDP, JOLTS)
- Exchange halts: Limit-Up/Limit-Down (LULD) volatility pauses, market-wide circuit breakers, exchange-initiated suspensions (technical or regulatory).

Section F of the supplementary material reports the specific excluded dates and the corresponding criteria satisfied. A total of **26 trading sessions** met the inclusion criteria and were retained for the analysis:

[10] Stable NASDAQ 100 constituents are defined here as those stocks that remained in the index continuously between July 1st, 2022, and June 30th, 2024.

[11] `https://polygon.io` (Accessed on 09-18-2025)

[12] With the exception of the VIX indices, which were merged subsequently.

- May: 2, 7, 9, 10, 13, 14, 16, 20, 21, 22, 23, 24, 28, 29, 30, and 31
- June: 3, 5, 6, 10, 11, 24, 25, 26, 27, and 28

11 Results

11.1 Experimental Setup and Reproducibility

All computational experiments were conducted on a remote machine equipped with an Intel Core i9–9940X CPU (14 cores, 28 threads, 3.30 GHz base clock), 125 GiB of RAM, and four NVIDIA GeForce RTX 2080 Ti GPUs (each with 11 GiB VRAM). Primary storage consisted of a 1.8 TB NVMe SSD, supplemented by a 3.5 TB HDD for data storage. The machine ran Ubuntu 20.04.6 LTS (Kernel 5.4.0-200-generic). The experiments were coded in Python 3.9.20, with R-based computations accessed through `rpy2` 3.5.16. The market replay process was implemented via a customized version of the MeatPy library[13].

Neural networks were developed using PyTorch 2.5.0 (CUDA 11.8). Stationarity tests relied on the R packages `urca` 1.3-4 and `fUnitRoot` 3042.79. LASSO regression was performed through the `glmnet` 4.1-8 R package. The `rugarch` 1.5-1 R package was employed for implementing ARMA(X)-GARCH models. Further dependencies are listed in the `environment.yml` located at the `ttstl_repo`[14] root directory.

To facilitate reproducibility and future research, the experimental artifacts—including the synthetic LOB dataset, pretrained CNN-LSTM model weights, and execution logs—are permanently archived on Zenodo (`https://doi.org/10.5281/zenodo.18304963`). The accompanying codebase is open-sourced across three GitHub repositories. The core market reconstruction engine is hosted at `https://github.com/judmejiabe/MeatPy.git`, a fork of the original `meatpy` library adapted to process Market-By-Order (MBO) data from Databento. Project-specific data manipulation pipelines, which utilize this engine to generate the feature sets described in Section 10.1, are available at `https://github.com/judmejiabe/evolving-microstructural-toolkit-data.git`. Finally, the implementation of the TTSTL framework, including the deep learning backbone, the adapter module, and all experimental scripts required to replicate the results in Sections 11.2 and 11.3, is provided at `https://github.com/judmejiabe/ttstl_repo.git`.

11.2 Synthetic Study

A simple Markovian SOFS model is introduced for synthetic data generation following the framework established in [13]. Consider a valid state of the LOB, denoted by $X \in \mathcal{L}$. Buy (sell) limit orders of size z and price k arrive at the matching engine according to independent Poisson processes with intensities $\lambda_+(k, X, z)$ ($\lambda_-(k, X, z)$). Similarly, order cancellations of size z and price k on the bid (ask) side occur according to independent Poisson processes with intensities $C_+(k, X, z)$ ($C_-(k, X, z)$).

Assuming that both sides of the LOB are non-empty, i.e., $X^+, X^- \neq 0 \in \mathbb{N}^d$, the order arrival intensities for $z = 1$ are given by:

$$\lambda_+(k, X, 1) = \begin{cases} 0.01 & \text{if } k = b_0(X) - 1, \\ 0.1 & \text{if } k = b_0(X), \\ 0.01 & \text{if } k = b_0(X) + 1, 2 \leq a_0(X) - b_0(X) < 3, \\ 0.2 & \text{if } k = b_0(X) + 1, 3 \leq a_0(X) - b_0(X), \\ 0.01 \exp\{\rho(X)\} & \text{if } k = a_0(X), \\ 0 & \text{otherwise,} \end{cases},$$

and

[13] `https://github.com/judmejiabe/MeatPy`

[14] `https://github.com/judmejiabe/ttstl_repo.git`

$$\lambda_-(k,X,1) = \begin{cases} 0.01 & \text{if } k = a_0(X)+1, \\ 0.1 & \text{if } k = a_0(X), \\ 0.01 & \text{if } k = a_0(X)-1, 2 \le a_0(X) - b_0(X) < 3, \\ 0.2 & \text{if } k = a_0(X)-1, 3 \le a_0(X) - b_0(X), \\ 0.01\exp\left\{-\rho(X)\right\} & \text{if } k = b_0(X), \\ 0 & \text{otherwise.} \end{cases}.$$

Here, $\rho(X) = \frac{v_0^b(X) - v_0^a(X)}{v_0^b(X) + v_0^a(X)}$ denotes the imbalance in trading volume of the state of the LOB represented through X. The order cancellation intensities are given by

$$C_+(k,X,1) = \begin{cases} 0.01 & \text{if } k = b_0(X)-1, \text{ and } V_1^b(X) \ge 1 \\ 0.1 & \text{if } k = b_0(X), \text{ and } V_0^b(X) \ge 1 \\ 0 & \text{otherwise,} \end{cases},$$

and

$$C_-(k,X,1) = \begin{cases} 0.01 & \text{if } k = a_0(X)-1, \text{ and } V_1^a(X) \ge 1 \\ 0.1 & \text{if } k = a_0(X), \text{ and } V_0^a(X) \ge 1 \\ 0 & \text{otherwise.} \end{cases}.$$

For all $z > 1$, the intensities are zero: $\lambda_+(k,X,z) = \lambda_-(k,X,z) = C_+(k,X,z) = C_-(k,X,z) = 0$.

The dynamics of this SOFS can be interpreted as follows: Orders arrive at and around the best quotes, as well as one tick beyond them. At-touch orders arrive with intensities ten times higher than both more aggressive and more passive orders. As the spread widens beyond three ticks, the LOB replenishes quickly. Marketable orders arrive at intensities similar to aggressive orders but increase with the imbalance, thereby pushing prices in the imbalance direction. Passive orders are canceled at the same rates as arrivals.

If the LOB bid side becomes empty ($X^+ = 0$), it is quickly replenished one tick below the best ask:

$$\lambda_+(k,X,z) = \begin{cases} 1 & \text{if } k = a_0(X)-1, \text{ and } z = 10, \\ 0 & \text{otherwise,} \end{cases}.$$

Similarly, if the ask side becomes empty ($X^- = 0$):

$$\lambda_-(k,X,z) = \begin{cases} 1 & \text{if } k = b_0(X)+1, \text{ and } z = 10 \\ 0 & \text{otherwise.} \end{cases}.$$

Note that between any two consecutive order arrival times τ_k and τ_{k+1}, the LOB remains unchanged. Hence, $\lambda_\star(k,X_t,z)$ and $C_\star(k,X_t,z)$ are constant over each interval $[\tau_k, \tau_{k+1})$ for $\star \in \{+,-\}$. Given the LOB state at time τ_k, the waiting time until the next message is exponentially distributed with a rate equal to the sum of all possible intensities. This property justifies the simulation algorithm described below.

Algorithm 1 generates simulated MBO data from which the LOB can be reconstructed for $t \in [0,T]$. To ensure computational feasibility, a maximum order size $z^{\max}$ is imposed.

Algorithm 1 Limit Order Book Simulation Algorithm

Require: T, X_0, λ_+, λ_-, C_+, C_-, d, $z^{\max}$
Ensure: Simulated Market-by-Order data
$t \leftarrow 0$
$i \leftarrow 0$
$X \leftarrow X_0$
while $t < T$ **do**
Compute $\lambda_\pm(k, X, z)$ and $C_\pm(k, X, z)$ for $k \in \{0, \ldots, d\}$ and $z \in \{1, \ldots, z^{\max}\}$
$\gamma \leftarrow \sum_{\Gamma \in \{\lambda, C\}} \sum_{\star \in \{+,-\}} \sum_{k=0}^{d} \sum_{z=1}^{z^{\max}} \Gamma_\star(k, X, z)$
Compute $\mathbf{p} = \big(\Gamma_\star(k, X, z)/\gamma\big)_{\Gamma \in \{\lambda, C\}, \star \in \{+,-\}, k \in \{0,\ldots,d\}, z \in \{1,\ldots,z^{\max}\}}$
Generate $\Delta t \sim \text{Exp}(\gamma)$
Generate $(\star, k, z) \sim \text{Categorical}(\mathbf{p})$
Send the order $(k, z, t + \Delta t, i, \star, \Gamma)$ to the matching engine.
The matching engine clears the order if necessary, updates the mbo feed and computes the new limit order book which is assigned to X
$t \leftarrow t + \Delta t$
$i \leftarrow i + 1$
end while

The simulations are set to start on a deep and balanced initial LOB state, $p_l^b(0) = 499 - l$, $p_l^a(0) = 501 + l$, and $v_l^\star(0) = 10$ for $l = 0, 1, \ldots, 400$ and $\star \in \{a, b\}$. Two simulated trading sessions of length T = 10,000,000 units of time (uot) are generated. The backbone, adapter, and sampling time bar lengths are set as 5,000,000, 100,000, and 100 uot. The data from one simulated trading session is used to fit TTSTL in global update mode and local adapt is applied to the other thus mimicking overnight recalibration. The LOB is reconstructed from the generated MBO data to construct an event-driven dataset of order flows and a physical-time driven dataset with the features presented in Table 3 with $H = 1$ and leaving the VIX index covariate out.

Twenty-eight different configurations of the TTSTL algorithm are implemented. A base configuration is set featuring an ARMAX(1,1)-GARCH(1,1)-DeepLOB with skewed-t distribution for the errors –the specifics of the base configuration are shown in Table S7 in Section G of the Supplementary material– and four different update rules interacting with seven covariate configurations are considered. The update rules considered are: Myopic (`M`), Simple Average (`SA`), Power Law with $\alpha = 3.0$ (`PL3`), and Exponential Smoothing with $\alpha = 0.1$ (`ES010`). The covariate configurations are: Full covariate set (`01`), parsimonious covariate set (i.e. considering the CNN-LSTM architecture ouput instead of the generative embedding additional to the microstructural features) (`02`), only microstructural covariates (i.e. $\mathbf{b}_{\mathbf{X}}^T = \mathbf{0}$, `03`), only neural covariates (i.e. $\mathbf{b}_{\boldsymbol{\omega}}^T = \mathbf{0}$, `04`), only CNN-LSTM output as a covariate bypassing the interface module (`05`), no covariates (`06`), and only imbalance in trading volume as a covariate bypassing the interface module (`07`). All of the possible configurations are tested across two simulated trading sessions: a Global Update is applied in session 1, and a Local Adaptation is applied in session 2. The configurations are ranked according to the RelMAE metric, computed for each adapter time bar within the backbone time bars of session 2 (Local Adapt mode). These backbone-specific rankings are then aggregated into an average ranking, from which the models are re-ranked. These are shown in Table 4. Figure S8 in Section G of the supplementary material displays a comparison of the RelMAE across all combinations of update rules and covariate configurations.

Several observations follow from these results. Models that incorporate a non-myopic update rules exhibit better performance accuracy. This finding is consistent with a heuristic interpretation in terms of variance reduction within the bias–variance decomposition [26]. The combined estimators seem to reduce variance without adding excessive bias to the forecast. The update rule and covariate configuration factors seem to have an interaction effect when explaining the forecasting accuracy. Also, observe that the average RelMAE in the second backbone bar tends to be smaller than the first backbone bar indicating a better TTSTL performance under more stationary environments.

The covariate configuration `05`, which bypasses the interface and forces the CNN-LSTM output into the regression structure, yielded identical results to the parsimonious covariate configuration `02`. These two configurations tied for the most accurate configuration across all Update Rules in both backbone bars. These observations showcase the CNN-LSTM ability to accurately transform LOB features into imputations which correlate with the target variable at least in the simulation framework presented. The equivalence between covariate configurations `02` and `05` showcases the variable filtering-accuracy of the

Table 4: RelMAE-based overall rankings and average RelMAE values for each TTSTL configuration across the two simulated backbone bars in Local Adapt mode. UR denotes the update rule, CC the covariate configuration, O/A Rank the overall ranking within each backbone bar, and Avg. RelMAE the corresponding average forecasting error.

UR	CC	O/A Rank BB1	Avg. RelMAE BB1	O/A Rank BB2	Avg. RelMAE BB2
M	01	19.0	96.21%	16.0	95.52%
M	02	10.5	94.99%	7.5	94.10%
M	03	18.0	95.75%	23.5	96.97%
M	04	20.0	96.26%	15.0	95.40%
M	05	10.5	94.99%	7.5	94.10%
M	06	28.0	97.21%	23.5	96.97%
M	07	24.0	96.85%	20.0	96.63%
SA	01	5.0	94.91%	14.0	94.84%
SA	02	1.5	94.59%	5.5	93.77%
SA	03	9.0	95.14%	25.5	96.49%
SA	04	8.0	95.00%	11.0	94.57%
SA	05	1.5	94.59%	5.5	93.77%
SA	06	25.0	96.77%	25.5	96.49%
SA	07	21.0	96.38%	19.0	96.17%
PL3	01	13.0	95.12%	13.0	94.89%
PL3	02	3.5	94.68%	3.5	93.83%
PL3	03	15.0	95.31%	27.5	96.64%
PL3	04	14.0	95.21%	10.0	94.63%
PL3	05	3.5	94.68%	3.5	93.83%
PL3	06	27.0	96.92%	27.5	96.64%
PL3	07	23.0	96.51%	18.0	96.28%
ES010	01	16.0	95.16%	12.0	94.75%
ES010	02	6.5	94.83%	1.5	93.73%
ES010	03	12.0	95.19%	21.5	96.47%
ES010	04	17.0	95.28%	9.0	94.45%
ES010	05	6.5	94.83%	1.5	93.73%
ES010	06	26.0	96.80%	21.5	96.47%
ES010	07	22.0	96.41%	17.0	96.10%

interface module operating the Flexible adaLASSO algorithm.

Configurations 01 and 04 yield the next best accuracy. Determining which one of these two is best is not straightforward as the full configuration outperforms the only generative features configuration in the first backbone bar but the order flips in the second backbone bar. Observe that the maximum difference in the average RelMAE is 0.1% in the first backbone bar and 0.3% in the second one. This signals that the expressivity of the generative features is at least as powerful as that of the manual features. The performance difference between these two configurations and the parsimonious configuration 02 is rather small. In fact, covariate configuration 01 differs from 02 in terms of average RelMAE by at most 0.44% in backbone bar 1 and 1.08% in backbone bar 2. Similarly, covariate configuration 04 differs from 02 by at most 0.52% and 0.81%. This illustrates the adapter module's ability to dynamically transform generative features into accurate forecasts. The question remains to whether the adapter module transforms these generative covariates into micro-return forecasts more accurately than the last layer of the CNN-LSTM produces imputations for the same quantity. Figure S9, which plots the RelMAE with the raw output of the CNN-LSTM instead of a backbone micro-return mean provides insights into this question. Indeed, covariate configurations 01 and 04 with non-myopic update rules outperform the raw CNN-LSTM imputations. Indicating the adequateness of the selection of the ARMAX-GARCH-DeepLOB for the adapter module.

Circumscribing the analysis to non-myopic update rules, covariate configurations featuring generative features (i.e. configurations 01, 02, 04, and 05) outperformed configurations which lack of this component. This result however should be interpreted carefully as the models with non generative covariates

represent less computational burden[15] and provide more interpretable insights into the LOB dynamics.

The features selected by the interface module in covariate configuration `03` (see the corridor plot in Figure S10, the relative contributions in Figure S11, and the parameter time series in Figure S12 all in in Section G of the supplementary material) in the locally adapted backbone bar 1 provide additional evidence that the flexible adaLASSO algorithm is a suitable statistical filter. In this backbone bar, the interface module selects the covariates ask order flow at touch, $OF^a_{0,j}$, bid order flow at touch $OF^b_{0,j}$, order flow imbalance at touch $OFI_{0,j}$, and imbalance in trading volume ρ_j.

A deductive explanation for this selection can be found in the dynamics of the Data Generating Process. In fact, a positive order flow in the bid (ask) side means either a larger queue at the best bid (ask) price, say case A, or a best bid (ask) price increase (decrease), case B. Case A events mean a positive (negative) impact in the continuous time-imbalance in trading volume which by design is set to increase the number of marketable buy (sell) orders thereby reducing the size of the best ask (bid) price queue and hence creating positive (negative) pressure in the micro-price. This is consistent with the sign of the estimated coefficients by the adapter module. In case B, a positive (negative) $OF^b_{0,j}$ ($OF^a_{0,j}$) coming as a consequence of a buy (sell) order inside the spread is not necessarily positively (negatively) correlated with a positive micro-return as the newly populated bid (sell) side queue can be quickly exhausted by a marketable sell (buy) order. Case A is far more likely than B by the DGP dynamics which explains the sign of the regression estimate.

The order flow imbalance aggregates differences in the event-driven order flow which contains more nuanced information than the arithmetic difference of physical-time driven order flows as it aggregates imbalances inside the sampling time bars. The order flow imbalance at touch $OFI_{0,j}$ increases as a consequence of case A and B events in OFS. Observe that a case A event and a case B event have the same impact in the $OFI_{0,j}$ feature but that case A events are significantly more likely than case B events justifying its positive sign. A thought experiment might reveal an interesting corrective effect carried out by the $OFI_{0,j}$. Imagine a balanced LOB snapshot with available liquidity $v^b_{0,j} = v^a_{0,j} = 10$ asset units and a spread of two ticks; imagine also the case where nine cancellations in the best bid price and one add order one tick above the best bid price took place in a given sampling time bar in that strict order. The resulting LOB will have a resulting configuration with $v^b_{0,j+1} = v^b_{1,j+1} = 1$ and $v^a_{0,j+1} = 10$ which most likely will cause a negative return in the following sampling time bar as given by the DGP. In this case $OF^b_{0,j+1} = 1$, which incorrectly adds a positive signal to the forecasting equations but $OFI_{0,j+1} = -9$ corrects such signal potentially moving it into negative territory.

The adapter module estimates a negative regression coefficient for the imbalance in trading volume feature, ρ_j. Notice that a positive continuous time imbalance in trading volume induces more marketable buy orders than sell ones which in principle reduces the size of the best ask queue thereby generating a positive return which is opposite to the estimated by the adapter module and smaller in magnitude compared to the regression coefficients of the other covariates. Several plausible explanations to this seemingly counterintuitive result arise: Perhaps counterintuitively, a snapshot of the imbalance in trading volume at the beginning of a sampling time bar is not a good summary of the imbalance in trading volume during the time bar as it is independent to queue scaling. That is, the imbalance in trading volume of say, $\rho_j = \frac{1}{2}$ does not distinguish between the LOB configurations with $v^b_{0,j} = 2$ and $v^a_{0,j} = 1$ from $v^b_{0,j} = 20$ and $v^a_{0,j} = 10$. The former case leads to a less persistent positive imbalance in trading volume than the latter. (ii) Another possible explanation for the flipped sign is due to a specific non-linear effect whereby a small queue is in front in terms of the price priority of a significantly larger queue. If the small queue is exhausted, the sign of the imbalance in trading volume can flip inside the time bar and generate pressure in the opposite direction. In any case, this apparent sign reversal likely arises from queue-size heterogeneity and non-linear interactions, suggesting TTSTL could benefit from incorporating non-linear features. The performance difference between covariate configurations `01`, `02` and `03` suggests that the CNN-LSTM architecture can learn non-linear interactions which improve forecasting accuracy. Therefore, a trade-off between interpretability and latency versus accuracy should be weighted before deploying TTSTL into a live application.

11.3 Study on INTC MBO Data

The TTSTL framework was applied to the INTC MBO dataset to assess its performance in a realistic high-frequency trading environment. For every trading date, the first and last 15 minutes of the session were excluded to focus the analysis on continuous double auction dynamics. Three 2-hour backbone bars were constructed for the remaining trading hours, each subdivided into 12 ten-minute adapter bars. The

[15] and therefore less latency

sampling period was set to two seconds. Guided by the synthetic study results (Section 11.2), the Exponential Smoothing combining rule was selected for the adapter updates, utilizing covariate configurations `01` through `06`. The CNN-LSTM component of the Backbone module was initialized using May MBO data, trained via the ADAM optimizer as detailed in Section 4.1. The adapter module employed an ARMAX(0,1)-EGARCH(1,1)-DeepLOB model with Negative Inverse Gaussian innovations to capture high-frequency leverage effects and heavy-tailed residual distributions. Table S8 in the supplementary material details the specific base configuration.

Table S9 (see Section G of the Supplement) presents the date-pair specific rankings and average RelMAE metrics across adapter time bars. Figure S13 displays the sequential RelMAE boxplots across date pairs in Local Adapt mode for the three backbone bars. The horizontal dashed line at 1.00 represents the baseline performance of the naive model (arithmetic average).

Several key patterns emerge from the results. Green rectangles in Figure S13 highlight instances where covariate configurations `05` (raw CNN-LSTM output) or `01` (full configuration) outperformed the no-covariate baseline (`06`). Conversely, red rectangles indicate periods where configuration `05` markedly underperformed. Black arrows mark the backbone bars where the CNN-LSTM weights were updated during the preceding Global Retrain step. Notably, in 22 out of the 30 constructed backbone bars, the interface module did not select any additional covariates beyond the base set. Consequently, configurations `01` through `04` yielded performance identical to configuration `06`. Configuration `06` consistently outperformed others in terms of RelMAE in 22 backbone bars, while configuration `05` was the top performer in 7 of the remaining 8 bars. Configuration `05` exhibited outstanding underperformance in six specific instances: 06-05 and 06-10 (Backbone Bar 1); 06-24, 06-25, and 06-27 (Backbone Bar 2); and 06-24 (Backbone Bar 3). This result, where the parsimonious hybrid model performs comparably—or better—than more complex DL-augmented versions, provides evidence of diminishing returns to complexity in the high-frequency domain. It further offers empirical support for statistical regularization, suggesting that the interface module's ability to filter noise is critical for robust performance.

Forecasting accuracy displayed intraday variations; with higher (less accurate) RelMAE scores observed in Backbone Bar 2 compared to Backbone Bar 3, and Backbone Bar 1 typically showing the lowest (most accurate) relative errors. This trend likely reflects the stabilizing effect of the midday trading lull, where the baseline average model becomes more effective compared to the more volatile open and close periods.

Analysis of atypical RelMAE behavior reveals a strong correlation between performance degradation and backbone updates. As seen in Figure S13 in Section G of the Supplementary Material, outstanding underperformance in configuration `05` frequently coincided with dates where the CNN-LSTM weights had been updated during the preceding global adaptation phase. This suggests that the model may have overfit to the specific dynamics of the previous day, failing to generalize to the new regime. This instability casts some doubt on the robustness of the standalone CNN-LSTM architecture for daily retraining without improved regularization.

To investigate potential exogenous drivers of this instability, aggregate statistics (mean, standard deviation, skewness, kurtosis) for the micro-return r_j^w, CNN-LSTM output $\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_j-99:i_j}\right)$, and order flow imbalance $OFI_{0,j}$ were examined (see Figures S14–S19 in the Supplement). No obvious distributional shifts in these covariates alone explain the calibration issues. However, the VIX series (Figures S20–S23 in the Supplement) reveals a significant anomaly: the VIX curve inverted on June 12th in anticipation of the CPI release and FOMC decision. This "double header" event seemed to have introduced heightened macro-uncertainty, known to drain liquidity and alter LOB dynamics, potentially contributing to the observed instability around these dates.

News analysis provides further context for the atypical sessions. The anomalies on June 5th followed the announcement of the Intel-Apollo joint venture, a positive catalyst. Similarly, the volatility on June 24th and 25th coincided with a broader Nvidia-led semiconductor selloff and rebound. These event-driven shifts underscore the CNN-LSTM architecture instability and the adapter's module ability to adapt under globally non-stationary but locally stationary regimes. This is likely due to the more efficient data usage and parsimonious structure.

The utility of the hybrid approach is evident when comparing the TTSTL forecasts against the raw neural network output. First, a key distinction between the baselines and the proposed framework lies in the nature of their outputs. While pure deep learning models typically produce deterministic point imputations, the Hybrid ARMAX-GARCH-DeepLOB model yields conditional predictive distributions. This capability allows for the construction of formal confidence intervals and the modeling of time-varying volatility, thereby bridging the gap between black-box signal extraction and rigorous econometric infer-

ence. Figure S24 (Supplement) demonstrates that the forecasting accuracy of the TTSTL forecasts is consistently lower than the imputation accuracy of the raw CNN-LSTM imputations across all covariate configurations by a significant margin. This confirms that the adapter module successfully refines the noisy, potentially biased neural network imputations into more accurate and statistically grounded forecasts.

Finally, it is acknowledged that the empirical analysis is limited to a single highly liquid asset and a fixed time window; robustness across assets and market regimes remains an important direction for future work.

12 Conclusions

In this work, the Two-Time-Scale Transfer Learning (TTSTL) framework was introduced. TTSTL can be interpreted as a modular statistical learning architecture separating representation learning from inference. By integrating deep learning-based feature extraction with a time series regression adapter, the framework addresses the dual challenges of non-stationarity and computational complexity inherent in high-frequency financial data.

The empirical results on simulated and INTC Market-by-Order data demonstrate that the TTSTL framework effectively refines generative imputations from the CNN-LSTM architecture into statistical forecasts. The adapter module consistently improved upon the raw neural network outputs, correcting for biases and local regime shifts that pure deep learning models often fail to capture. Furthermore, the interface module, utilizing the flexible adaLASSO, proved to be a capable stochastic filter, dynamically selecting relevant covariates and enhancing the parsimony of the forecasting model.

The standalone CNN-LSTM architecture exhibited instability following daily retraining, wherein overfitting to prior-day dynamics led to performance degradation in subsequent sessions. This finding casts doubt on the practicality of frequent, unconstrained retraining of complex deep learning models for LOB prediction, suggesting that simpler, more robust specifications or stronger regularization may be required for live deployment.

Finally, the results offer support for the soft version of the Efficient Market Hypothesis (EMH) in high-frequency settings. While the hybrid model demonstrated predictive skill relative to naive baselines, the marginal gains were often small and transient, particularly during periods of high information flow or macro-uncertainty. This suggests that external information should complement the target asset's MBO data to achieve sustainable low-latency alpha.

Author Contributions

- JDMB: Conceptualization, Data Curation, Formal Analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – Original Draft
- GWP: Conceptualization, Formal Analysis, Investigation, Methodology, Validation, Supervision, Writing – Original Draft, Writing – Review & Editing
- RH: Supervision, Resources (computational infrastructure), Writing – Review & Editing

Data and Code Availability

To facilitate reproducibility and future research, the experimental artifacts—including execution logs, output data, pretrained model weights, and the synthetic Market-by-Order dataset generated for this study—have been permanently archived and are openly available on Zenodo at `https://doi.org/10.5281/zenodo.18304963`. The complete codebase required to process this data and execute the experiments is open-sourced and accessible across three GitHub repositories: the core market reconstruction engine at `https://github.com/judmejiabe/MeatPy.git`, the data generation and manipulation pipelines at `https://github.com/judmejiabe/evolving-microstructural-toolkit-data.git`, and the TTSTL framework implementation at `https://github.com/judmejiabe/ttstl_repo.git`.

AI Usage Statement

During the preparation of this project, the authors utilized AI-assisted technologies and large language models to assist with language polishing, structural refinement, LaTeX formatting, and software development for the accompanying codebase. The authors comprehensively reviewed, edited, and validated all generated text and code, and take full responsibility for the final content of the publication and the provided software artifacts.

Declaration of Interest Statement

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Anindya Banerjee, Juan J Dolado, John W Galbraith, and David Hendry. *Co-integration, error correction, and the econometric analysis of non-stationary data*. Oxford university press, 1993.

[2] Yashar H Barardehi, Dan Bernhardt, and Ryan J Davies. Trade-Time Measures of Liquidity. *The Review of Financial Studies*, 32(1):126–179, 03 2018.

[3] Rina Foygel Barber, Emmanuel J. Candes, Aaditya Ramdas, and Ryan J. Tibshirani. Conformal prediction beyond exchangeability, 2023.

[4] Jean-Philippe Bouchaud, Marc Mézard, and Marc Potters. Statistical properties of stock order books: empirical results and models. *Quantitative Finance*, 2(4):251–256, August 2002.

[5] Farid Boussama. Ergodicité, mélange et estimation dans les modeles garch. *These de doctorat, Université Paris*, 7, 1998.

[6] Leo Breiman. Statistical modeling: The two cultures (with comments and a rejoinder by the author). *Statistical science*, 16(3):199–231, 2001.

[7] Antonio Briola, Silvia Bartolucci, and Tomaso Aste. Deep limit order book forecasting, 2024.

[8] Marine Carrasco and Xiaohong Chen. Mixing and moment properties of various garch and stochastic volatility models. *Econometric Theory*, 18(1):17–39, 2002.

[9] Álvaro Cartea, Samuel N Cohen, Rob Graumans, Saad Labyad, Leandro Sánchez-Betancourt, and Leon van Veldhuijzen. Statistical predictions of trading strategies in electronic markets. *Available at SSRN 4442770*, 2023.

[10] Alvaro Cartea, Ryan Donnelly, and Sebastian Jaimungal. Enhancing trading strategies with order book signals. *Applied Mathematical Finance*, 25(1):1–35, 2018.

[11] Tolga Cenesizoglu, Georges Dionne, and Xiaozhou Zhou. Asymmetric effects of the limit order book on price dynamics. *Journal of Empirical Finance*, 65:77–98, 2022.

[12] Rama Cont, Mihai Cucuringu, and Chao Zhang. Cross-impact of order flow imbalance in equity markets. *Quantitative Finance*, 23(10):1373–1393, 2023.

[13] Rama Cont, Pierre Degond, and Lifan Xuan. A mathematical framework for modelling order book dynamics, 2023.

[14] Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The Price Impact of Order Book Events. *Journal of Financial Econometrics*, 12(1):47–88, 06 2013.

[15] Marcelo G Cruz, Gareth W Peters, and Pavel V Shevchenko. *Fundamental aspects of operational risk and insurance analytics: A handbook of operational risk*. John Wiley & Sons, 2015.

[16] Fotios Petropoulos Diamantis Koutsandreas, Evangelos Spiliotis and Vassilios Assimakopoulos. On the selection of forecasting accuracy measures. *Journal of the Operational Research Society*, 73(5):937–954, 2022.

[17] Chiranjit Dutta, Kara Karpman, Sumanta Basu, and Nalini Ravishanker. Review of statistical approaches for modeling high-frequency trading data. *Sankhya B*, 85(Suppl 1):1–48, 2023.

[18] David Easley, Marcos Lopez de Prado, and Maureen O'Hara. Discerning information from trade data. *Journal of Financial Economics*, 120(2):269–285, 2016.

[19] David Easley, Marcos López de Prado, Maureen O'Hara, and Zhibai Zhang. Microstructure in the machine age. *The Review of Financial Studies*, 34(7):3316–3363, 2021.

[20] Christian Francq and Jean-Michel Zakoian. *GARCH models: structure, statistical inference and financial applications*. John Wiley & Sons, 2019.

[21] Jim Gatheral and Roel CA Oomen. Zero-intelligence realized variance estimation. *Finance and Stochastics*, 14(2):249–283, 2010.

[22] Martin D. Gould, Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison. Limit order books, 2013.

[23] Clive WJ Granger and Paul Newbold. Spurious regressions in econometrics. *Journal of econometrics*, 2(2):111–120, 1974.

[24] James Douglas Hamilton. *Time Series Analysis*. Princeton University Press, Princeton, NJ, 1994.

[25] Joel Hasbrouck. Measuring the information content of stock trades. *The Journal of Finance*, 46(1):179–207, 1991.

[26] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. The elements of statistical learning: data mining, inference, and prediction, 2017.

[27] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.

[28] Konark Jain, Nick Firoozye, Jonathan Kochems, and Philip Treleaven. Limit order book simulations: A review, 2024.

[29] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[30] Robert Kissell. Chapter 2 - market microstructure. In Robert Kissell, editor, *The Science of Algorithmic Trading and Portfolio Management*, pages 47–85. Academic Press, San Diego, 2014.

[31] Petter N. Kolm, Jeremy Turiel, and Nicholas Westray. Deep order flow imbalance: Extracting alpha at multiple horizons from the limit order book. *Mathematical Finance*, 33(4):1044–1081, 2023.

[32] Petter N. Kolm and Nicholas Westray. Improving deep learning of alpha term structures from the order book. *SSRN*, 2024. Available at SSRN: `https://ssrn.com/abstract=4770476`.

[33] Andreas Koukorinis, Gareth W Peters, and Guido Germano. Memory, persistence, and the price process in interest rate futures contracts: A statistical investigation at the mesoscale using kernel two-sample testing. *Available at SSRN 4959155*, 2026.

[34] Denis Kwiatkowski, Peter CB Phillips, Peter Schmidt, and Yongcheol Shin. Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root? *Journal of econometrics*, 54(1-3):159–178, 1992.

[35] Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985.

[36] Jeremy Large. Measuring the resiliency of an electronic limit order book. *Journal of Financial Markets*, 10(1):1–25, 2007.

[37] David Madigan and Greg Ridgeway. Bayesian data analysis for data mining. *Handbook of Data Mining*, pages 103–132, 2003.

[38] Marcelo C. Medeiros and Eduardo F. Mendes. l1-regularization of high-dimensional time-series models with non-gaussian and heteroskedastic errors. *Journal of Econometrics*, 191(1):255–271, 2016.

[39] Stefan Meisenbacher, Marian Turowski, Kaleb Phipps, Martin Rätz, Dirk Müller, Veit Hagenmeyer, and Ralf Mikut. Review of automated time series forecasting pipelines. *WIREs Data Mining and Knowledge Discovery*, 12(6):e1475, 2022.

[40] Juan Diego Mejia Becerra, Gareth W. Peters, and Ruimeng Hu. On the asymptotic properties of the conditional maximum likelihood estimator for time series regression models with conditional heteroskedasticity. Working paper, 2025.

[41] OMX NASDAQ. Nasdaq omx psx totalview-itch 5.0, 2014.

[42] Efstathios Panayi and Gareth Peters. Stochastic simulation framework for the limit order book using liquidity motivated agents, 2015.

[43] Christine A Parlour and Duane J Seppi. Limit order markets: A survey. *Handbook of financial intermediation and banking*, 5:63–95, 2008.

[44] Bernhard Pfaff, Eric Zivot, and Matthieu Stigler. *urca: Unit Root and Cointegration Tests for Time Series Data*. CRAN, May 27, 2024, 2024. R package version 1.3-4.

[45] Quang Pham, Chenghao Liu, and Steven C. H. Hoi. Continual Learning, Fast and Slow. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(1):134–149, January 2024.

[46] Peter CB Phillips. Understanding spurious regressions in econometrics. *Journal of econometrics*, 33(3):311–340, 1986.

[47] Matteo Prata, Giuseppe Masi, Leonardo Berti, Viviana Arrigoni, Andrea Coletta, Irene Cannistraci, Svitlana Vyetrenko, Paola Velardi, and Novella Bartolini. Lob-based deep learning models for stock price trend prediction: a benchmark study. *Artificial Intelligence Review*, 57(5):1–45, 2024.

[48] Philip Protter. *Stochastic integration and differential equations. A new approach*, volume 21. Springer Berlin, Heidelberg, 01 2005.

[49] Kylie-Anne Richards. *Modelling the dynamics of the limit order book in financial markets*. PhD thesis, UNSW Sydney, 2019.

[50] Kylie-Anne Richards, William TM Dunsmuir, and Gareth W Peters. Score test for marks in hawkes processes. *International Journal of Data Science and Analytics*, pages 1–16, 2024.

[51] Richard Roll. A simple implicit measure of the effective bid-ask spread in an efficient market. *The Journal of Finance*, 39(4):1127–1139, 1984.

[52] Said E Said and David A Dickey. Testing for unit roots in autoregressive-moving average models of unknown order. *Biometrika*, 71(3):599–607, 1984.

[53] Daniel Straumann and Thomas Mikosch. Quasi-maximum-likelihood estimation in conditionally heteroscedastic time series: A stochastic recurrence equations approach. *The Annals of Statistics*, 34(5), October 2006.

[54] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.

[55] Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Using deep learning to detect price change indications in financial markets. In *2017 25th European signal processing conference (EUSIPCO)*, pages 2511–2515. IEEE, 2017.

[56] Svitlana Vyetrenko, David Byrd, Nick Petosa, Mahmoud Mahfouz, Danial Dervovic, Manuela M. Veloso, and Tucker Hybinette Balch. Get real: realism metrics for robust limit order book market simulations. *Proceedings of the First ACM International Conference on AI in Finance*, 2019.

[57] Diethelm Wuertz, Tobias Setz, Yohan Chalabi, and Georgi N. Boshnakov. *fUnitRoots: Rmetrics - Modelling Trends and Unit Roots*, 2024. Accessed: 01-19-2025.

[58] Yihui Xie. Spurious regression: Simulation and theoretical analysis, 2006. (`https://db.yihui.org/docs/Spurious-Regression-Yihui-Xie-2006.pdf`) Accessed: 01-19-2025.

[59] Tianyu Zhang and Jing Lei. Online estimation with rolling validation: Adaptive nonparametric estimation with streaming data, 2025.

[60] Zihao Zhang, Bryan Lim, and Stefan Zohren. Deep learning for market by order data, 2021.

[61] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, June 2019.

[62] Hui Zou. The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, 101(476):1418–1429, 2006.

Supplementary Material

Contents

A Mathematical Representation of the Order Flow Sequence and Limit Order Book **2**

B Alternative Sampling Schemes **7**

C CNN–LSTM and DeepLOB Model Specifications **8**
C.1 Building Blocks . . . 8
C.1.1 Permutation . . . 8
C.1.2 Concatenation . . . 8
C.1.3 Activation Functions . . . 8
C.1.4 Batch Normalization . . . 9
C.1.5 Max-Pooling . . . 9
C.1.6 Zero-Padding . . . 9
C.1.7 Fully Connected Layer . . . 10
C.1.8 Convolutional Filters . . . 10
C.1.9 LSTM Layer . . . 10
C.2 Composite Structures . . . 11
C.2.1 The Composite Convolutional Layer . . . 11
C.2.2 The Inception Layer . . . 12
C.3 Neural Network Architectures . . . 12
C.3.1 The CNN-LSTM Architecture . . . 12
C.3.2 The DeepLOB Architecture . . . 13

D Stationarity Testing **15**

E Case Study: Model Subspace Selection **17**
E.1 Background . . . 17
E.2 Simulation Design . . . 19
E.3 Results and Discussion . . . 20
E.3.1 Multicollinearity Effect . . . 20
E.3.2 Effect of Initial Weights . . . 21
E.4 Conclusions and Recommendations . . . 21
E.5 Reproducibility . . . 22

F Events Matching Exclusion Criteria on the INTC MBO Dataset **23**
F.1 Corporate Actions . . . 23
F.2 Earnings and Company-Specific News . . . 23
F.3 Derivative Market Events . . . 24
F.4 Scheduled Announcements . . . 24
F.5 Exchange Halts . . . 24

G Complimentary Tables and Figures **25**

A Mathematical Representation of the Order Flow Sequence and Limit Order Book

This section is devoted to mathematically describing the OF sequence and the LOB drawing ideas from [13, 22, 42]. First observe that most LOBs operate with a minimum price variation, or tick size, denoted θ, and a maximum price, denoted D. For simplicity, and without losing generality, prices will be measured in ticks. Also let $d = \frac{D}{\theta} \in \mathbb{N}\setminus\{0\}$ be the maximum price measured in ticks and denote $\mathbb{N}_{0:d} = \{0, 1, \dots, d\}$ the set of possible tick-measured prices.

Definition 1. *Let $o = (p_o, z_o, t_o, i_o, s_o, k_o)$ denote a message submitted to the matching engine at time $t_o \in [0, T]$, with price $p_o \in \mathbb{N}_{0:d}$, size $z_o \in \mathbb{N}$, identifier $i_o \in \mathbb{N}$, side $s_o \in \{1, -1\}$ and type $k_o \in \{L, C, M\}$. $s_o = 1$ ($s_o = -1$) indicates that the message o is submitted in the bid (ask) side of the LOB. L, C and M identify limit, cancellations and market orders respectively.*

Definition 2. *Let $\mathbf{o} = \{o_i\}_{i=1}^{N-1}$, $N \in \mathbb{N}$, denote an ordered sequence of messages such that $t_{o_i} \leq t_{o_{i+1}}$. This sequence is known as the order flow sequence.*

To favor notation lightness, The subscript o_i will be dropped in favor of i in the elements of the message sequence $\mathbf{o}$ whenever no ambiguity is present (e.g. t_i instead of t_{o_i} will be used). For simplicity, identify buy (sell) market orders in the order sequence $\mathbf{o}$ as limit orders with the maximum (minimum) possible price. That is,

$$(p_o, z_o, t_o, i_o, 1, M) \equiv (d, z_o, t_o, i_o, 1, L)$$
$$(p_o, z_o, t_o, i_o, -1, M) \equiv (0, z_o, t_o, i_o, -1, L)$$

Definition 3. *Let $d \in \mathbb{N} \setminus \{0\}$ be the maximum price of an asset. The order aggregator, $\mathcal{X} : \mathbb{N} \times \mathbb{N} \times [0, T] \times \mathbb{N} \times \{-1, 1\} \times \{L, C\} \to \mathbb{Z}^d \times \mathbb{Z}^d$, groups limit orders and cancellations by price and adds its sizes*

$$\mathcal{X}(\mathbf{o}) = \left(\sum_{o \in \mathbf{o}} (I_L^+(o) - I_C^+(o)) z_o e_{p_o}, \sum_{o \in \mathbf{o}} (I_L^-(o) - I_C^-(o)) z_o e_{p_o} \right),$$

where the market orders in $\mathbf{o}$ are identified as limit orders, $e_i = (0, 0, \dots, 0, 1, 0, \dots, 0) \in \mathbb{Z}^d$ where 1 is the i-th position, $I_k^+(o)$ ($I_k^-(o)$) takes value 1 if $s_o = 1$ ($s_o = -1$) and $k_o = k$, $k \in \{L, C\}$.

Two pathological scenarios might arise with the established notation. First, a cancellation message might exceed the size of the initial order or target a nonexistent order, inadvertently transforming into a market order that could circumvent the established price-time priority rules. Second, a single order could deplete all available liquidity on the opposite side of the LOB, leading to a situation where the price becomes undefined. Although these scenarios are rare, occurring under highly specific conditions, we preclude them by setting forth two basic assumptions:

(A1) Every cancellation correspond to a message submitted earlier, with no cancellation exceeding the outstanding size

(A2) No buy (sell) market order exceeds the ask (bid) volume of the limit order book.

The set of possible values of $\mathbf{o}$ under the restrictions (A1) and (A2) is denoted as $\mathcal{O}$. To reconstruct the LOB, the modular framework described in [13] is employed. This framework decomposes the dynamics of the limit order book into two distinct elements: the incoming order flow and the market clearing procedure. The incoming order flow is conceptualized as a marked point process, while the market clearing procedure is a deterministic operator responsible for pairing buy and sell orders.

Denote $X = (X^+, X^-) \in E := \mathbb{N}^d \times \mathbb{N}^d$ a configuration of limit orders.[16] Observe that the set of valid configurations of LOBs is a proper subset of E. Some notation is necessary to determine this set. First, for $Z \in \mathbb{N}^d$, consider the measure

$$\nu_Z = \sum_{k=1}^{d} Z_k \delta_k,$$

where δ_k denotes the Dirac's delta measure at $k \in \{1, \dots, d\}$. Also, let

[16] i.e. $X = \mathcal{X}(\mathbf{x})$ for some $\mathbf{x} \in \mathcal{O}$.

- $\nu_X^{\pm} = \nu_{X^{\pm}}$
- $\tau^i(Z)$ be such that $\nu_{\tau^i(Z)} = \nu_Z 1_{[i,\infty)}$
- $\tau_i(Z)$ be such that $\nu_{\tau_i(Z)} = \nu_Z 1_{[0,i]}$
- $a(X) = \inf \operatorname{supp}(\nu_X^+)$, with $a(0) = 0$
- $b(X) = \sup \operatorname{supp}(\nu_{X^+}^-)$, with $b(0) = d$

Observe that $a(X)$ and $b(X)$ correspond to the best ask and bid of a limit order book snapshot respectively. The set of valid limit order book configurations is thus

$$\mathcal{L} := \{X = (X^+, X^-) \in E : a(X) > b(X)\}.$$

Definition 4. *Let $\mathbf{o} \in \mathcal{O}$. Define the mapping $J : [0,T] \times \mathcal{O} \to \mathbb{Z}^d \times \mathbb{Z}^d$ as follows*

$$J(t, \mathbf{o}) = \begin{cases} \mathcal{X}(\{o \in \mathbf{o} : t_o = t_i\}) & \text{if } t = t_i \ \exists i = 1, \dots, N, \\ 0 & \text{otherwise.} \end{cases}$$

Observe that J models changes in the LOB right before matching. The matching process is modeled in [13] through a deterministic order matching clearing operator $\mathcal{C} : E \to \mathcal{L}$ whose set of invariant points is $\mathcal{L}$. It can be shown that, under some conditions, $\mathcal{C}$ is uniquely characterized as follows:

$$\mathcal{C}(X)^+ = \begin{cases} \tau_{p_B(X)}(X^+) & \text{if } g_X(p_B(X)) \le -X^+_{p_B(X)}, \\ \tau_{p_B(X)-1}(X^+) - \min(0, g_X(p_B(X)))e_{p_B}(X) & \text{if } g_X(p_B(X)) > -X^+_{p_B(X)}. \end{cases}$$

$$\mathcal{C}(X)^- = \begin{cases} \tau^{p_A(X)}(X^+) & \text{if } g_X(p_A(X)) \ge X^-_{p_A(X)}, \\ \tau^{p_A(X)-1}(X^+) + \max(0, g_X(p_A(X)))e_{p_A}(X) & \text{if } g_X(p_A(X)) < X^-_{p_A(X)}, \end{cases}$$

where $X \in E$ is the set of limit orders. Let $B_X(k) = \nu^+[k, \infty)$, $S_X(k) = \nu_X^-[0,k]$, $g(k) = S_X(k) - B_X(k)$, and

$$p_A(X) = \inf\{k \in \{1, \dots, d\} : g_X(k) > 0\},$$
$$p_B(X) = \sup\{k \in \{1, \dots, d\} : g_X(k) < 0\}.$$

The reconstruction of an LOB given $\mathbf{o} \in \mathcal{O}$ is now straightforward:

Definition 5. *Given $\mathbf{o} \in \mathcal{O}$, the reconstructed LOB is a mapping $L : [0,T] \times \mathcal{O} \to \mathcal{L}$ such that*

$$L(t, \mathbf{o}) = \begin{cases} L(t_*, \mathbf{o}) & \text{if } t \ne t_i \ \forall i = 1, \dots, N, \\ \mathcal{C}\left(L(t_*, \mathbf{o}) + J(t_i, \mathbf{o})\right) & \text{if } t = t_i \ \exists i = 1, \dots, N. \end{cases}$$

Here, $t_* = \max\{t_j : t_j < t\}$.

Definition 6. *For a valid LOB configuration, $X \in \mathcal{L}$, let $b_h(X) = b(X) - h$ ($a_h(X) = a(X) + h$) be the price h ticks below (above) the best bid (ask). Also let $V_h^b(X) = \nu_X^+\{b_h(X)\}$ ($V_h^a(X) = \nu_X^-\{a_h(X)\}$) be the corresponding volume sitting h ticks below (above) the best ask. For convenience, denote $a_h(t) = a_h(X(t))$, $b_h(t) = b_h(X(t))$, $V_h^a(t) = V_h^a(X(t))$ and $V_h^b(t) = V_h^b(X(t))$.*

An alternative approach, known as the compressed representation, to describe the liquidity available in the LOB is in terms of its levels. The l-th bid (ask) price level is defined as the l-th highest (lowest) price in the LOB that has positive volume available. Formally,

Definition 7. *For a valid LOB configuration, $X \in \mathcal{L}$, the l-th level bid (ask) price, is defined iteratively as $p_l^b(X) = \inf \operatorname{supp}\left(\tau_{p_{l-1}^b(X)-1}(X^+)\right)$ ($p_l^a(X) = \inf \operatorname{supp}\left(\tau^{p_{l-1}^a(X)+1}(X^-)\right)$) for $l \ge 1$ and $p_0^b(X) = b_0(X)$ ($p_0^a(X) = a_0(X)$). Similarly, denote the volume available at the l-th bid (ask) level as $v_l^b(X) = \nu_X^+\{p_l^b(X)\}$ ($v_l^a(X) = \nu_X^-\{p_l^a(X)\}$). To keep notation light, denote $p_l^a(t) = p_l^a(X(t))$, $p_l^b(t) = p_l^b(X(t))$, $v_l^a(t) = v_l^a(X(t))$ and $v_l^b(t) = v_l^b(X(t))$.*

Each representation offers distinct trade-offs. For instance, working with decompressed limit order book data may simplify the mathematical modeling of order flow, while compressed formats are more memory-efficient—an especially important advantage for illiquid assets. Moreover, compressed representations often provide a more suitable input to neural network models, potentially enhancing their predictive accuracy.

We have assumed a fixed message sequence $\mathbf{o} \in \mathcal{O}$ and reconstructed the limit order book for such sequence. To add stochasticity, let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered, complete and right-continuous probability space and consider the following

Definition 8. *Let τ be a $\mathbb{F}$-stopping time and π, ζ, ι, σ and κ; $\mathcal{F}_\tau$-adapted random variables with values in $\mathbb{N}_{0:d}$, $\mathbb{N}$, $\mathbb{N}$, $\{-1, 1\}$, and $\{L, C, M\}$ respectively. The corresponding stochastic message is denoted as $\omega = (\pi, \zeta, \tau, \iota, \sigma, \kappa)$.*

Analogously, for a sequence of non-explosive increasing sequence of $\mathbb{F}$-adapted stopping times $(\tau_i)_{i=1}^{\infty}$ and $\mathcal{F}_{\tau_i}$-measurable random variables π_i, ω_i, σ_i, ι_i and κ_i for which assumptions (A1) and (A2) hold, define the stochastic message sequence

$$\boldsymbol{\omega} = (\omega_i : \tau_i \leq T),$$

where $\omega_i = (\pi_i, \zeta_i, \tau_i, \iota_i, \sigma_i, \kappa_i)$.

The L function from definition 5 can be employed to specify a stochastic LOB.

Definition 9. *The LOB process is a stochastic process, denoted $(X(t))_{t\geq 0}$ and defined as*

$$X(t) = L(t, \boldsymbol{\omega}).$$

For the sake of clarity, the previous definitions are illustrated below. Consider the following order flow sequence $\mathbf{o} \in \mathcal{O}$ represented in the following table

Table S1: Example OF sequence

p_i	z_i	t_i	i_i	s_i	k_i
10	10	0.04	1	1	L
11	20	1.23	2	1	L
12	100	2.45	3	-1	L
12	50	3.14	4	-1	L
8	50	4.75	5	-1	L
9	150	5.63	6	1	L
13	30	6.14	7	1	L
13	45	7.85	8	-1	L
12	70	8.32	3	-1	C
12	50	9.78	4	-1	C

Definition 3 applied to the sequence $\mathbf{o}$ implies

$$\begin{aligned}\mathcal{X}(\mathbf{o}) = \big(&150e_9 + 10e_{10} + 20e_{11} + 30e_{13},\\ &50e_8 + 30e_{12} + 45e_{13}\big).\end{aligned}$$

This object is depicted in Figure S1.

Following the price-time priority rule, orders 1 through 4 are not marketable and are simply added to the LOB. The aggressive sell order 5 then arrives, clearing orders 1 and 2 entirely and leaving 20 units unexecuted on the ask side sitting at 8 ticks, with the bid side empty. When the buy order 6 arrives, it clears the remaining 20 units of order 5, leaving 130 units on the bid side sitting at 9 ticks. Order 7 next clears 30 units of order 3, leaving 120 units at 12 ticks on the ask side. Another 45 units are then added at price 13 on the ask side. Finally, messages 9 and 10 cancel the remaining volumes of orders 3 and 4. Thus leaving 130 units sitting on the bid side at 9 ticks and 45 units sitting at 13 ticks on the ask side. As a side note, observe that $\mathcal{C}\left(\mathcal{X}(\mathbf{o})\right) = (150e_9 + 10e_{10}, 30e_{12} + 45e_{13})$ which is different from the resulting



Figure S1: Aggregated example orders.

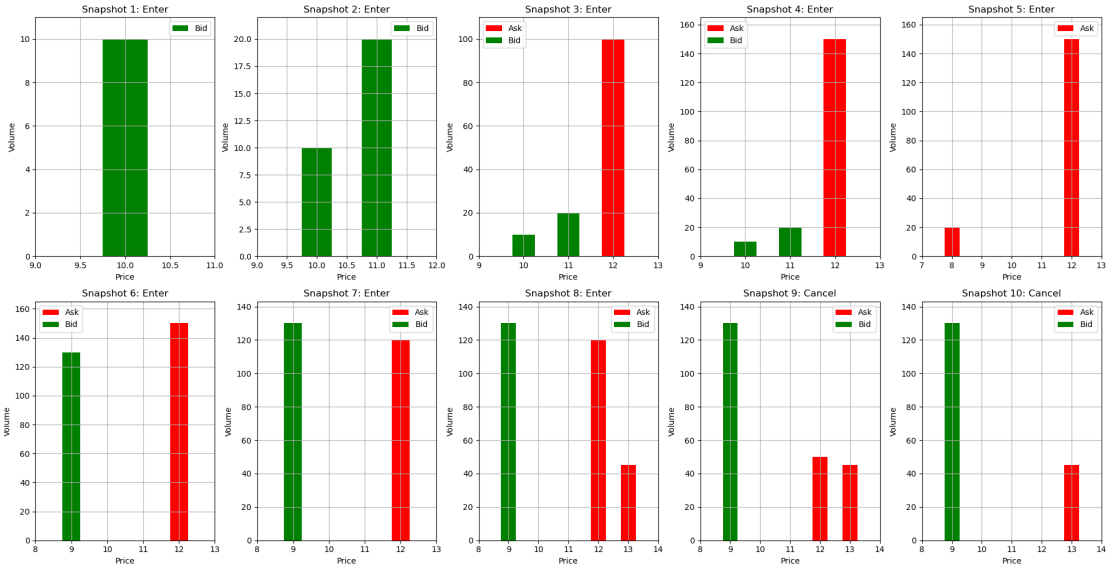


Figure S2: Sequence of LOB snapshots corresponding to Table S1.

LOB of the example $L(t = 10, \mathbf{o}) = (130e_9, 45e_{13})$. This reflects the fact that continuously clearing the LOB (as in trading hours) can yield different results to discrete clearing (as in opening/closing auctions).

Figure S2 illustrates the corresponding sequence of LOB snapshots resulting from these events.

A useful distinction between the OF sequence and MBO data lies in how MBO data can be constructed from the OF sequence without requiring explicit order matching. One way to achieve this is by labeling addition messages as `A` and cancellation messages as `C`. Whenever an outstanding order executes, a series of messages—`T` (trade), `F` (filled), and `C` (cancellation)—can be inserted at the same timestamp to indicate the trade initiator, the filled order, and the final cancellation needed for LOB reconstruction. Under this scheme, the LOB can be fully reconstructed using only `A` and `C` messages, eliminating the need for a separate clearing process. Table S2 illustrates this procedure.

Table S2: Example MBO data.

timestamp	**order_id**	**side**	**volume**	**price**	**message_type**
0.04	1	B	10	10	A
1.23	2	B	20	11	A
2.45	3	A	100	12	A
3.14	4	A	50	12	A
4.75	5	A	20	11	T
4.75	2	B	20	11	F
4.75	2	B	20	11	C
4.75	5	A	10	10	T
4.75	1	B	10	10	F
4.75	1	B	10	10	C
4.75	5	A	20	8	A
5.63	6	B	20	8	T
5.63	5	A	20	8	F
5.63	5	A	20	8	C
5.63	6	B	130	9	A
6.14	7	B	30	12	T
6.14	3	A	30	12	F
6.14	3	A	30	12	C
7.85	8	A	45	13	A
8.32	3	A	70	12	C
9.78	4	A	50	12	C

B Alternative Sampling Schemes

For completeness, we define the *volume bars* partition $\mathcal{P}^{(V)}$ via the stopping times

$$\tau_i^{(V)} \;=\; \inf\Big\{\, t > \tau_{i-1}^{(V)} : \sum_{k:\,\kappa_k = M} \zeta_k > i \cdot \texttt{Vol} \Big\}, \quad \tau_0^{(V)} = 0,$$

where $i, \texttt{Vol} \in \mathbb{Z}^+$. Each interval $\Delta\tau_i^{(V)} \;=\; \tau_i^{(V)} - \tau_{i-1}^{(V)}$ corresponds to the time required for $\texttt{Vol}$ units to trade. Furthermore, given any partition $\mathcal{P}$, we define the counting process

$$N^{\mathcal{P}}(t) \;=\; \Big|\big\{\tau \in \mathcal{P} :\, 0 < \tau \le t\big\}\Big|,$$

such that $N^{\mathcal{P}}(t_1, t_2) = N^{\mathcal{P}}(t_2) - N^{\mathcal{P}}(t_1)$ counts the number of partition points in $(t_1, t_2]$.

C CNN–LSTM and DeepLOB Model Specifications

This section formalizes the CNN–LSTM and DeepLOB architectures [31, 61]. Section C.1 introduces the fundamental tensor operations, which are assembled into the composite layers described in Section C.2. These composite layers are then connected via convolutional and inception modules to yield the full CNN–LSTM and DeepLOB architectures, as detailed in Section C.3.

C.1 Building Blocks

This section describes the fundamental tensor operations that compose the CNN-LSTM and DeepLOB architectures.

C.1.1 Permutation

For simplicity, consider a tensor $T \in \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$ as a multidimensional array with entries denoted by $T_{i_1 i_2 \cdots i_d}$. Denote the data batches as four-dimensional tensors in $\mathbb{R}^{N \times C \times H \times W}$, where N denotes the batch size, C the number of channels, H the temporal length, and W the spatial width.

Tensor permutation serves as a higher-dimensional analogue to the matrix transpose. Define the permutation between the k-th and l-th dimensions as

$$\begin{aligned} \mathrm{Perm}^{k,l} : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_k \times \cdots \times N_l \times \cdots \times N_d} &\to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_l \times \cdots \times N_k \times \cdots \times N_d} \\ T &\mapsto \mathrm{Perm}^{k,l}(T) \end{aligned}$$

where $\left[\mathrm{Perm}^{k,l}(T)\right]_{i_1 i_2 \cdots i_k \cdots i_l \cdots i_d} = T_{i_1 i_2 \cdots i_l \cdots i_k \cdots i_d}$.

C.1.2 Concatenation

Tensor concatenation along the k-th dimension is defined as

$$\begin{aligned} \mathrm{Concat}^k : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_k \times \cdots \times N_d} \times \mathbb{R}^{N_1 \times N_2 \times \cdots \times N'_k \times \cdots \times N_d} &\to \mathbb{R}^{N_1 \times N_2 \times \cdots \times (N_k + N'_k) \times \cdots \times N_d} \\ (T, S) &\mapsto \mathrm{Concat}^k(T, S) \end{aligned}$$

where,

$$\left[\mathrm{Concat}^k(T,S)\right]_{i_1 i_2 \cdots i_k \cdots i_d} = \begin{cases} T_{i_1 i_2 \cdots i_k \cdots i_d} & 1 \le i_k \le N_k, \\ S_{i_1 i_2 \cdots i_k \cdots i_d} & N_k + 1 \le i_k \le N_k + N'_k. \end{cases}$$

C.1.3 Activation Functions

Two activation functions are necessary: The Sigmoid and the Leaky RelU. For a scalar $x \in \mathbb{R}$ these two activation functions are defined as

$$\begin{aligned} \mathrm{Sigmoid}(x) &= \frac{1}{1 + \exp(-x)}, \\ \mathrm{LeakyReLU}_\alpha(x) &= \begin{cases} x & x \ge 0, \\ \alpha x & x < 0. \end{cases} \end{aligned}$$

For a tensor $T \in \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$, define $\mathrm{Sigmoid} : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d} \to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$ and $\mathrm{LeakyReLU}_\alpha : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d} \to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$

$$\begin{aligned} \left[\mathrm{Sigmoid}(T)\right]_{i_1 i_2 \cdots i_d} &= \mathrm{Sigmoid}(T_{i_1 i_2 \cdots i_d}), \\ \left[\mathrm{LeakyReLU}_\alpha(T)\right]_{i_1 i_2 \cdots i_d} &= \mathrm{LeakyReLU}_\alpha(T_{i_1 i_2 \cdots i_d}). \end{aligned}$$

C.1.4 Batch Normalization

In the CNN-LSTM model, Spatial Batch Normalization, PyTorch's `BatchNorm2d`[17], originally proposed in [27], is applied to each convolutional feature map. For a tensor of size N and feature maps of shape (C, H, W), `BatchNorm2d` computes per-channel statistics over all $N \times H \times W$ activations, then learns one pair of affine parameters (γ_c, β_c). This normalization is supposed to alleviate the internal covariate shift problem by fixing each channel's mean to zero and variance to one. This stabilization has been empirically shown to allow for higher learning rates and faster convergence, and can indirectly alleviate issues like vanishing or exploding gradients.

More precisely, let $T \in \mathbb{R}^{N\times C\times H\times W}$ denote an activation tensor of size N each with C channels of spatial size $H \times W$. Denote by T_{icjk} the element for sample i, channel c, and spatial position (j, k). The Spatial Normalization operation is an application $BN^{\phi,\varepsilon} : \mathbb{R}^{N\times C\times H\times W} \to \mathbb{R}^{N\times C\times H\times W}$ defined as

$$\big[\mathrm{BN}^{\phi,\varepsilon}(T)\big]_{icjk} = \gamma_c \frac{T_{icjk} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta_c.$$

Here, γ_c and β_c are trainable parameters, μ_c and σ_c^2 are channel means and variances respectively, ϕ is a momentum hyperparameter and, ε is a constant used for numerical stability. The computations of μ_c and σ_c^2 differ for training and testing. During training, these quantities are computed in a batch-wise fashion as

$$\mu_c = \mu_c^{(B)} = \frac{1}{N\,H\,W}\sum_{i=1}^{N}\sum_{j=1}^{H}\sum_{k=1}^{W} T_{icjk}, \quad \sigma_c^2 = \sigma_c^{2\,(B)} = \frac{1}{N\,H\,W}\sum_{i=1}^{N}\sum_{j=1}^{H}\sum_{k=1}^{W}\big(T_{icjk} - \mu_c^{(B)}\big)^2.$$

During testing (or inference) the batch mean, these quantities are computed in the likes of a exponential moving average with momentum ϕ. That is

$$\mu_c \leftarrow (1-\phi)\,\mu_c + \phi\,\mu_c^{(B)}, \qquad \sigma_c^2 \leftarrow (1-\phi)\,\sigma_c^2 + \phi\,\sigma_c^{2\,(B)}.$$

The interested reader is invited to consult [27] for details on the gradients of this transformation and some numerical experiments.

C.1.5 Max-Pooling

Max-pooling is a downsampling operation that accentuates the strongest activations within local neighborhoods. Given a 4D tensor $T \in \mathbb{R}^{N\times C\times H\times W}$, a max-pool layer with window size $n_h \times n_w$ and stride (s_h, s_w) produces an output tensor

$$\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T) \in \mathbb{R}^{N\times C\times H'\times W'},$$

where

$$H' = \left\lfloor \frac{H - n_h}{s_h} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - n_w}{s_w} \right\rfloor + 1.$$

Its entries are given by

$$\big[\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T)\big]_{ickl} = \max\{T_{ick'l'} : k' \in \{(k-1)s_h + i' : i' = 1, 2, \ldots, n_h\}, \\ l' \in \{(l-1)s_w + j' : j' = 1, 2, \ldots, n_w\}\},$$

for $i = 1, \ldots, N$, $c = 1, \ldots, C$, $k = 1, \ldots, H'$, $l = 1, \ldots, W'$. In other words, each output cell $\Big[\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T)\Big]_{ickl}$ is the maximum value within the $n_h \times n_w$ patch of T starting at spatial location $((k-1)\,s_h, (l-1)\,s_w)$ in channel c of sample i.

C.1.6 Zero-Padding

Convolution and pooling layers typically reduce spatial dimensions. However, certain architectures require the output tensor to have the same height and width as the input. The Zero-padding operation

[17] `https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html` (Accessed on 04-29-2025)

achieves this by surrounding the input with zeros along its spatial axes. Formally, for $T \in \mathbb{R}^{N\times C\times H\times W}$, the padded tensor

$$\mathrm{ZP}_{p_h,p_w}(T) \in \mathbb{R}^{N\times C\times(H+2p_h)\times(W+2p_w)},$$

has entries

$$\big[\mathrm{ZP}_{p_h,p_w}(T)\big]_{ickl} = \begin{cases} 0, & k \le p_h \quad \text{or} \quad k > H + p_h, \\ 0, & l \le p_w \quad \text{or} \quad l > W + p_w, , \\ T_{ic(k-p_h)(l-p_w)}, & \text{otherwise}, \end{cases}$$

for $i = 1,\dots,N$, $c = 1,\dots,C$, $k = 1,\dots,H+2p_h$, and $l = 1,\dots,W+2p_w$. This adds p_h rows of zeros above and below, and p_w columns of zeros to the left and right of each feature map.

C.1.7 Fully Connected Layer

A fully connected layer performs an affine transformation of its input. Concretely, given an input vector $a \in \mathbb{R}^D$, a weights matrix $W^{(\mathrm{FC})} \in \mathbb{R}^{n_h\times D}$, and a bias vector $b \in \mathbb{R}^{n_h}$, the layer output is

$$\mathrm{FC}_{n_h}(a) \;=\; W^{(\mathrm{FC})}\,a \;+\; b \;\in\; \mathbb{R}^{n_h}.$$

When applied to a batch of N inputs, arranged as $A \in \mathbb{R}^{N\times D}$, the same transformation is applied independently to each row:

$$\big[\mathrm{FC}_{n_h}(A)\big]_{i\cdot} \;=\; \mathrm{FC}_{n_h}\big(A_{i\cdot}\big), \quad i = 1,\dots,N.$$

Note that $\mathrm{FC}_{n_h}(A) \in \mathbb{R}^{N\times n_h}$

C.1.8 Convolutional Filters

Convolutional filters detect local patterns—such as edges or textures—by sliding small learnable kernels over the input. Concretely, let $T \in \mathbb{R}^{N\times C\times H\times W}$ and $W^{(\mathrm{Conv})} \in \mathbb{R}^{C'\times C\times k_h\times k_w}$ denote, respectively, a batch of N feature-map tensors with C channels and spatial size $H \times W$, and a set of C' convolutional kernels of size $k_h \times k_w$. With stride (s_h, s_w) and bias vector $b \in \mathbb{R}^{C'}$, the convolution operation produces the output

$$\big[\mathrm{Conv}^{C',k_h,k_w}_{s_h,s_w}(T)\big]_{ic'kl} = \big[T * W^{(\mathrm{Conv})}\big]_{ic'kl} + b_{c'},$$

where

$$\big[T * W^{(\mathrm{Conv})}\big]_{ic'kl} = \sum_{c=1}^{C}\sum_{k'=1}^{k_h}\sum_{l'=1}^{k_w} T_{ic((k-1)s_h+k')((l-1)s_w+l')} W^{(\mathrm{Conv})}_{c'ck'l'}$$

for $i = 1,\dots,N$, $c' = 1,\dots,C'$, $k = 1,\dots,H'$, and $l = 1,\dots,W'$. Note that $\mathrm{Conv}^{C',k_h,k_w}_{s_h,s_w}(T) \in \mathbb{R}^{N\times C'\times H'\times W'}$ where

$$H' \;=\; \left\lfloor \frac{H-k_h}{s_h} \right\rfloor + 1, \quad W' \;=\; \left\lfloor \frac{W-k_w}{s_w} \right\rfloor + 1.$$

If the stride is $(1,1)$, the subscript may be omitted for notational simplicity.

C.1.9 LSTM Layer

A long–short term memory (LSTM) layer processes sequential inputs while maintaining internal cell and hidden states. Define the LSTM operator H_{n_h} for input dimension D and hidden dimension n_h as follows. For $x \in \mathbb{R}^D, \quad a, c \in \mathbb{R}^{n_h}$, set

$$H_{n_h}(x, a, c) = (a', c').$$

Where

$$\begin{aligned}
\Gamma_f &= \mathrm{Sigmoid}\big(W^{(fx)}\,x + W^{(fa)}\,a + b^{(f)}\big),\\
\Gamma_u &= \mathrm{Sigmoid}\big(W^{(ux)}\,x + W^{(ua)}\,a + b^{(u)}\big),\\
\Gamma_o &= \mathrm{Sigmoid}\big(W^{(ox)}\,x + W^{(oa)}\,a + b^{(o)}\big),\\
\tilde{c} &= \tanh\big(W^{(cx)}\,x + W^{(ca)}\,a + b^{(c)}\big),\\
c' &= \Gamma_f \odot c + \Gamma_u \odot \tilde{c},\\
a' &= \Gamma_o \odot \tanh(c').
\end{aligned}$$

Here, for each gate symbol $\star \in \{f, u, o, c\}$,

$$W^{(\star x)} \in \mathbb{R}^{n_h \times D}, \quad W^{(\star a)} \in \mathbb{R}^{n_h \times n_h}, \quad b^{(\star)} \in \mathbb{R}^{n_h},$$

are trainable parameters, and $\odot$ denotes element-wise product. Given a sequence $T \in \mathbb{R}^{H \times D}$ with rows $x^{\langle i \rangle}$ for $i = 1, \ldots, H$, initialize

$$a^{\langle 0 \rangle} = 0, \quad c^{\langle 0 \rangle} = 0,$$

and recursively compute for $i = 1, \ldots, H$,

$$(a^{\langle i \rangle}, c^{\langle i \rangle}) = H_{n_h}\big(x^{\langle i \rangle}, a^{\langle i-1 \rangle}, c^{\langle i-1 \rangle}\big).$$

Define the sequence-to-vector transformation

$$\begin{aligned}
\mathrm{LSTM}_{n_h} : \mathbb{R}^{H \times D} &\to \mathbb{R}^{n_h},\\
T &\mapsto \mathrm{LSTM}_{n_h}(T) = a^{\langle H \rangle}.
\end{aligned}$$

For batched input $T \in \mathbb{R}^{N \times H \times D}$, the LSTM acts independently on each batch slice:

$$\big[\mathrm{LSTM}_{n_h}(T)\big]_{i\cdot} = \mathrm{LSTM}_{n_h}(T_{i\cdot\cdot}), \quad i = 1, \ldots, N,$$

so that

$$\mathrm{LSTM}_{n_h} : \mathbb{R}^{N \times H \times D} \to \mathbb{R}^{N \times n_h}.$$

C.2 Composite Structures

C.2.1 The Composite Convolutional Layer

The composite convolutional layer applies three blocks of

$$\text{Convolution} \;\to\; \text{LeakyReLU} \;\to\; \text{BatchNorm},$$

each with its own kernel size and stride. For some input in $\mathbb{R}^{N \times C \times H \times W}$ the composite convolutional layer applies a convolution filter of size $(1, k_w)$ and stride $(1, s_w)$. Observe this filter smooths the features in the space dimension. After applying a Leaky ReLU activation function and Batch Normalization, a time convolution filter of size $(4, 1)$ and stride $(1, 1)$ is applied. Then a Leaky ReLU and Batch Normalization are applied again. This last process is applied twice (see the composite convolution layer in Figure S3 for a clear representation). Define the outer and inner layers to keep notation compact

$$\begin{aligned}
\mathrm{Outer}_{s_w}^{k_w}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}_{1,s_w}^{32,1,k_w}(T),\\
\mathrm{Inner}^{(1)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{32,4,1}(T),\\
\mathrm{Inner}^{(2)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{32,4,1}(T).
\end{aligned}$$

Thus, the composite convolution layer with initial size $(1, k_w)$ and stride $(1, s_w)$ combines one outer layer and two inner layers as follows

$$\mathrm{CConv}_{s_w}^{k_w}(T) = \mathrm{Inner}^{(2)} \circ \mathrm{Inner}^{(1)} \circ \mathrm{Outer}_{s_w}^{k_w}(T).$$

Unless stated otherwise, no operations within the architecture share trainable parameters.

C.2.2 The Inception Layer

The inception layer is used to reduce the amount of computation needed to apply convolution filters in the deeper layers of a neural network [54]. It involves applying a set of $(1,1)$ sized convolutional filters and max-pooling layers with padding are combined with an activation function and Batch Normalization. Then convolution operations of different size are applied and concatenated. Let us define the following layers

$$\begin{aligned}
\mathrm{IL}^{(1)}(T) &= \mathrm{BN}^{\phi,\varepsilon,(2)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,3,1} \circ ZP^{1,0} \circ \\
&\qquad \mathrm{BN}^{\phi,\varepsilon,(1)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,1,1}(T), \\
\mathrm{IL}^{(2)}(T) &= \mathrm{BN}^{\phi,\varepsilon,(2)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,5,1} \circ ZP^{2,0} \circ \\
&\qquad \mathrm{BN}^{\phi,\varepsilon,(1)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,1,1}(T), \\
\mathrm{IL}^{(3)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,3,1} \circ MP^{3,1}_{1,1} \circ ZP^{1,0}(T).
\end{aligned}$$

Once again, the superscripts in parentheses emphasize that the layers with similar notation do not share trainable parameters. The inception layer is the concatenation, along the second dimension, of these three tensors.

$$\mathrm{IL}(T) = \mathrm{Concat}^2(\mathrm{IL}^{(1)}(T), \mathrm{Concat}^2(\mathrm{IL}^{(2)}(T), \mathrm{IL}^{(3)}(T))).$$

C.3 Neural Network Architectures

C.3.1 The CNN-LSTM Architecture

The CNN-LSTM architecture of Kolm et al. [31], which is depicted in Figure S3, is now formalized. As argued in Section 4.1, there are multiple possibilities for the neural network output. The multihorizon approach explained this section is considered but the other possibilities can be constructed by modifying the outer layer. Let K be the output dimension. The CNN-LSTM neural network transformation is a map $\mathrm{CNNLSTM} : \mathbb{R}^{N\times 100\times 20} \to \mathbb{R}^{N\times K}$, where N is the training batch size, defined as

$$\mathrm{CNNLSTM}(\mathbf{X}) = \mathrm{FC}_K \circ \mathrm{LSTM}_{64} \circ \mathrm{Perm}^{3,2} \circ \mathrm{IL} \circ \mathrm{CConv}_1^{10} \circ \mathrm{CConv}_2^2(\mathbf{X}).$$

Here, $\mathbf{X}$ is a 3D tensor of whitened order flow matrices $\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}$. That is,

$$\mathbf{X} = \left[\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_1-99:i_1}, \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_2-99:i_2}, \ldots, \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_N-99:i_N}\right].$$

Table S3 presents the intermediate dimensions of the layers with its respective trainable parameters assuming one horizon ($K = 1$) and a batch of size 32 ($N = 32$). The resulting model has $133,249$ parameters.

Table S3: CNN-LSTM architecture components.

Layer	Notation	Input shape	Output shape	Trainable Parameters
Conv. Layer	CConv_2^2	$(32,1,100,20)$	$(32,32,94,10)$	8544
Conv. Layer	CConv_1^{10}	$(32,32,94,10)$	$(32,32,88,1)$	18720
Inception Layer	IL	$(32,32,88,1)$	$(32,194,88,1)$	39872
Permutation	$\mathrm{Perm}^{3,2}$	$(32,192,88,1)$	$(32,88,192,1)$	0
LSTM	LSTM_{64}	$(32,82,192)$	$(32,64)$	66048
Fully Connected	FC_1	$(32,64)$	$(32,1)$	65

During inference mode, the CNN-LSTM architecture will produce an output for each order flow matrix. Such output can be denoted $\mathrm{CNNLSTM}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right)$. Moreover, the FC_K layer can be bypassed in order to obtain the penultimate layer representation which is effectively a feature map $\phi_{\mathbf{W}} : \mathbb{R}^{100\times 20} \to \mathbb{R}^{64}$ defined as

$$\phi_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right) = \mathrm{LSTM}_{64} \circ \mathrm{Perm}^{3,2} \circ \mathrm{IL} \circ \mathrm{CConv}^{10}_{1} \circ \mathrm{CConv}^{2}_{2}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right).$$

The subscript $\mathbf{W}$ is convenient to highlight the dependency on the parameters of the CNN-LSTM architecture which are updated iteratively.

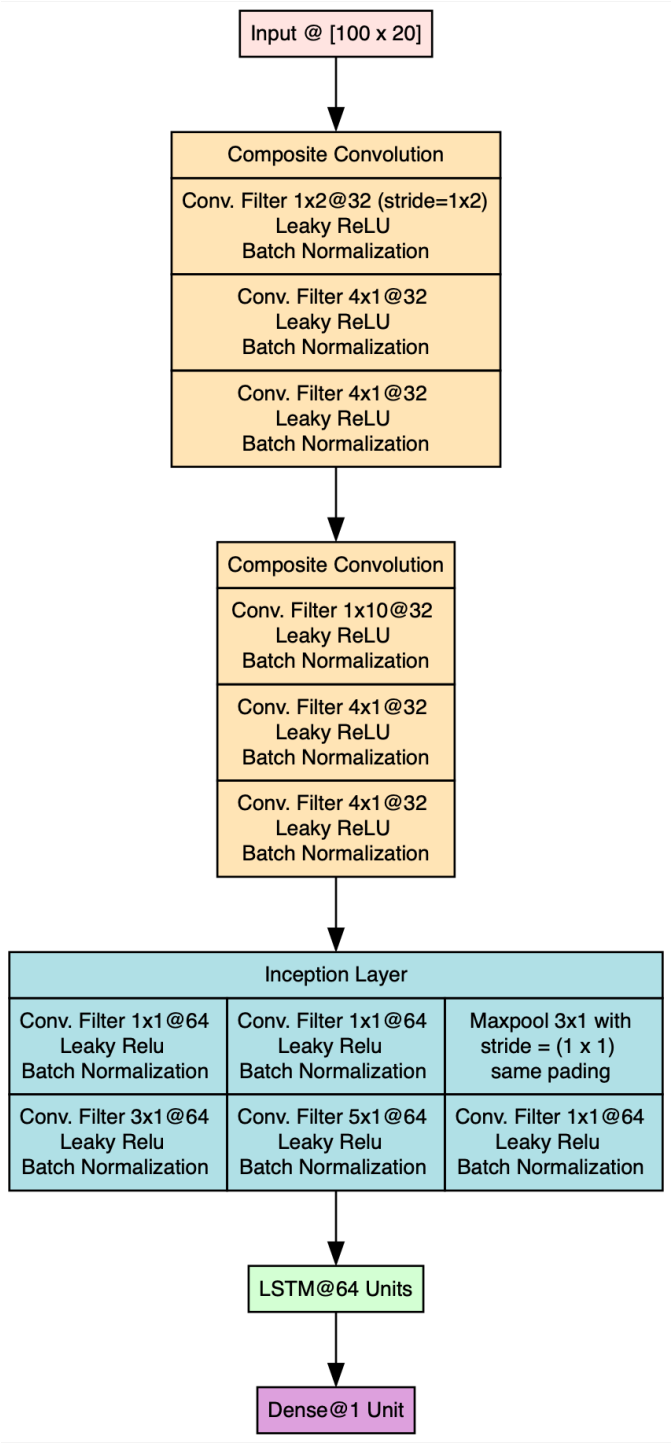


Figure S3: Illustration of the CNN-LSTM architecture.

C.3.2 The DeepLOB Architecture

Similarly, the DeepLOB architecture by Zhang et al. [61] adds an extra composite convolutional layer at the top of the CNN-LSTM architecture and takes raw prices and volumes as inputs. Formally DeepLOB :

$\mathbb{R}^{N\times 100\times 40} \to \mathbb{R}^{N\times K}$ with

$$\text{DeepLOB}(\mathbf{X}) = \text{FC}_K \circ \text{LSTM}_{64} \circ \text{Perm}^{3,2} \circ \text{IL} \circ \text{CConv}_1^{10} \circ \text{CConv}_2^{2,(2)} \circ \text{CConv}_2^{2,(1)}(\mathbf{X}).$$

Here, $\mathbf{X}$ is a 3D tensor of whitened limit order book price-volume information as defined in Section 3.1:

$$\mathbf{X} = \left[\tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_1-99:i_1}, \tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_2-99:i_2}, \ldots, \tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_N-99:i_N}\right].$$

Table S4 illustrates the components of this network assuming a scalar target variable ($K = 1$) and a batch of size 32 ($N = 32$). The resulting model comprises $143,777$ parameters.

Table S4: DeepLOB architecture components.

Layer	Notation	Input shape	Output shape	Trainable Parameters
Conv. Layer	$\text{CConv}_2^{2,(1)}$	$(32, 1, 100, 40)$	$(32, 32, 94, 20)$	8544
Conv. Layer	$\text{CConv}_2^{2,(2)}$	$(32, 32, 94, 20)$	$(32, 32, 88, 10)$	10528
Conv. Layer	CConv_1^{10}	$(32, 32, 88, 10)$	$(32, 32, 82, 1)$	18720
Inception Layer	IL	$(32, 32, 82, 1)$	$(32, 194, 82, 1)$	39872
Permutation	$\text{Perm}^{3,2}$	$(32, 194, 82, 1)$	$(32, 82, 194, 1)$	0
LSTM	LSTM_{64}	$(32, 82, 194)$	$(32, 64)$	66048
Fully Connected	FC_1	$(32, 64)$	$(32, 1)$	65

D Stationarity Testing

It is well known in forecasting theory that exogenous variables must be stationary to avoid spurious regressions [58]. Three main tests are adopted:

(A) Augmented Dickey–Fuller (ADF) Test. For any variable y (see Section 10.1), the ADF test with local trend considers the following autoregressive model:

$$y_j = c + \big(1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p\big) y_j + \varepsilon_j,$$

which can be equivalently written as

$$\Delta y_j = \alpha + \gamma y_{j-1} + \sum_{k=1}^{p-1} \zeta_k \, \Delta y_{j-k} + \varepsilon_j, \tag{S1}$$

where $\rho = \sum_{\ell=1}^{p} \phi_\ell$, $\zeta_k = -\sum_{\ell=k+1}^{p} \phi_\ell$, and B is the backshift operator. Note that $\gamma \le 0$, and $\gamma = 0$ corresponds to $\rho = 1$, indicating a unit root in $\phi(z) = 1 - \phi_1 z - \cdots - \phi_p z^p$. The ADF test compares

$$H_0^{\mathrm{ADF}} : \gamma = 0 \quad \text{versus} \quad H_1^{\mathrm{ADF}} : \gamma < 0.$$

Implementation details follow [1] and are available in the `fUnitRoots` package [57]. The test statistic is

$$Z_t^{\mathrm{ADF}} = \frac{\hat{\gamma}}{\mathrm{s.e.}(\hat{\gamma})},$$

where $\hat{\gamma}$ is estimated by ordinary least squares (OLS) on (S1). Because Z_t^{ADF} follows a distribution derived from stochastic integrals rather than a Student's t-distribution, the critical values differ accordingly [1].

A well-known drawback of the ADF test is its sensitivity to the choice of lag length p. In practice, this requires manual input or automated procedures. For simplicity, the current setup fixes $p = 10$ for all variables. Results from the ADF test are then compared with those of the PP test, which removes the lag-length requirement at the cost of reduced power.

(B) Phillips–Perron (PP) Test. The PP test uses a baseline model:

$$y_j = \rho \, y_{j-1} + u_j, \quad u_j = \sum_{k=0}^{\infty} \psi_k \, \varepsilon_{j-k} = \psi(B) \, \varepsilon_j,$$

where ε_j is an iid sequence of random variables with mean zero and variance σ^2. Assuming $\sum_{k=0}^{\infty} k \, \psi_k < \infty$, define $\gamma_k = \mathbb{E}[u_j \, u_{j-k}] = \sigma^2 \sum_{s=0}^{\infty} \psi_s \psi_{s+k}$ and $\lambda = \sigma \sum_{s=0}^{\infty} \psi_s = \sigma \, \psi(1)$. The test contrasts

$$H_0^{\mathrm{PP}} : \rho = 1 \quad \text{versus} \quad H_1^{\mathrm{PP}} : |\rho| < 1.$$

The statistic is

$$Z_\tau^{\mathrm{PP}} = \Big(\frac{\hat{\gamma}_0}{\hat{\lambda}^2}\Big)^{1/2} \frac{\hat{\rho} - 1}{\mathrm{s.e.}(\hat{\rho})} \; - \; \frac{1}{2}\Big(\hat{\lambda}^2 - \hat{\gamma}_0\Big)\Big(\frac{\hat{\lambda}^2}{\sum_{j=1}^{T} (y_j - \bar{y})^2}\Big)^{1/2},$$

where

$$\hat{\gamma}_k = \frac{1}{T} \sum_{j=k+1}^{T} \hat{u}_j \, \hat{u}_{j-k},$$

$$\hat{\lambda}^2 = \hat{\gamma}_0 + 2 \sum_{k=1}^{q} \Big(1 - \frac{k}{q+1}\Big) \hat{\gamma}_k,$$

$$q = \Big\lfloor 12 \Big(\frac{T}{100}\Big)^{1/4} \Big\rfloor.$$

Since Z_τ^{PP} depends on functionals of Brownian motion rather than standard distributions, critical values must be derived from the appropriate asymptotic theory. This test is implemented via the `urca` package [44].

(C) KPSS Test. For completeness, the Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test [34] is included in a confirmatory role. Unlike the ADF and PP tests, the KPSS null hypothesis assumes stationarity. The model is

$$y_j = r_j + u_j, \quad r_j = r_{j-1} + e_j,$$

where e_j is an iid sequence with zero mean and variance σ_e^2, and u_j satisfies properties similar to those in the PP framework. That is $u_j = \psi(B)\varepsilon_j$, for some $\varepsilon \overset{iid}{\sim} (0, \sigma^2)$, and $\sum_{k=0}^{\infty} k\psi_k < \infty$. The hypotheses are

$$H_0^{KPSS} : \sigma_e^2 = 0 \quad \text{versus} \quad H_1^{KPSS} : \sigma_e^2 > 0.$$

The test statistic is

$$\hat{\eta}_{\mu}^{\text{KPSS}} = \sum_{k=1}^{T} \frac{S_k^2}{\hat{\lambda}^2},$$

where $S_k = \sum_{j=1}^{k} \hat{u}_j$ represents the partial sum of residuals ($\hat{u}_j = y_j - \bar{y}$), and $\hat{\lambda}^2$ follows the same form as in the PP test. As in the previous tests, critical values derive from nonstandard limiting distributions. The `urca` package used for computing the rejection region of the this test.

E Case Study: Model Subspace Selection

This Monte Carlo study compares adaptive LASSO (adaLASSO) [62, 38] with OLS-Bonferroni selection for exact support recovery and examines how quickly the failure probability decays to zero. The study is organized as follows. Section E.1 reviews the Medeiros & Mendes framework describing the DGP and assumptions presented in [38]. Section E.2 describes the particular DGP used for the simulations and the dynamics of design space growth. Section E.3 analyzes the simulated data to quantify the effect of multicollinearity and initial weights selection. Section E.4 draws conclusions and recommendations and Section E.5 provides precise instructions for the replication of the present study.

E.1 Background

This section discusses the key regularity conditions and results in [38] and provides some intuitive interpretation to these ideas.

First, consider the data generating process (DGP)

$$y_t = \alpha_0 + \boldsymbol{\theta}^T \mathbf{x}_t + u_t. \tag{S2}$$

Here, $\{\mathbf{x}_t\}_{t\in\mathbb{Z}}$ is an n_T-dimensional covariate process, which possibly contains lags of the endogenous process, and $\{u_t\}_{t\in\mathbb{Z}}$ is a martingale difference sequence. Denote by T the sample size of observations of the process $\{y_t\}_{t\in\mathbb{Z}}$, n_T the total number of covariates, and s_T the number of relevant covariates. For simplicity assume that the first s_T elements of $\mathbf{x}_t$ correspond to the relevant variables, which can effectively be denoted as $\mathbf{x}_t = [\mathbf{x}_t(1), \mathbf{x}_t(2)]$. Also write $\boldsymbol{\theta}^T = [\boldsymbol{\theta}^T(1), \boldsymbol{\theta}^T(2)]$ and let $\boldsymbol{\theta}_0$ denote the true parameter which effectively implies that $\boldsymbol{\theta}_0^T = [\boldsymbol{\theta}_0^T(1), \mathbf{0}^T]$. Assuming, without loss of generality, that $\alpha_0 = 0$, The adaLASSO technique consists in minimizing the objective function given by

$$f(\boldsymbol{\theta}) = \left\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\right\|_2^2 \; + \; \lambda_T \sum_k w_k \left|\theta_k\right|.$$

Here, $w_k = \left|\theta_{I,k}\right|^{-\tau}$ are computed from $\tau > 0$ and $\boldsymbol{\theta}_I$ an initial parameter estimate. The initial parameter estimate could be given, among others, by OLS, LASSO or elastic-net. The choice of the algorithm for the initial parameters is left out of the scope of this simulation study. Medeiros and Mendes group their assumptions into four categories: Data Generating Process (DGP), Design Space (DESIGN), Initial Weights (WEIGHTS), and Regularization Parameter (REG). These assumptions are restated below for self-completeness.

(DGP1) Define $\mathbf{Z}_t = (\mathbf{x}_t^T,\, u_t)^T$. $\{\mathbf{Z}_t\}_{t\in\mathbb{Z}}$ is zero–mean and weakly stationary.

(DGP2) $\mathbb{E}\left\{u_t \mid \mathcal{F}_t\right\} = 0$, where $\mathcal{F}_t = \sigma(\mathbf{x}_t, \mathbf{Z}_{t-1}, \mathbf{Z}_{t-2}, \dots)$.

(DGP3)

$$\max_{1\le i\le n_T} \left|\tfrac{1}{T}\sum_{t=1}^{T}(x_{it}^2 - E[x_{it}^2])\right| \xrightarrow{\mathbb{P}} 0.$$

(DGP4) For some $m \ge 2$, there exists $c_m < \infty$ such that

$$\mathbb{E}\left|x_{it}\, u_t\right|^m \le c_m \quad \forall t \in \mathbb{Z}, i \in 1 : n_T.$$

(DESIGN1) θ_0 is an open subset $\boldsymbol{\Theta}_{n_T} \subseteq \mathbb{R}^{n_T}$ which contains $\mathbf{0} \in \mathbb{R}^{n_T}$.

(DESIGN2) There exists $\theta_{\min} > 0$ such that $\min_{i\le s_T} |\theta_{0,i}| > \theta_{\min}$.

(DESIGN3) Denote $\hat{\boldsymbol{\Omega}}_{11} := \frac{1}{T}\mathbf{X}^T(1)\mathbf{X}(1)$ and $\boldsymbol{\Omega}_{11} := \mathbb{E}\left\{\mathbf{x}_t(1)\mathbf{x}_t^T(1)\right\}$

(a) There exists a constant $\phi_{\min} \in (0,1)$ such that

$$\inf_{\|\boldsymbol{\alpha}\|=1} \boldsymbol{\alpha}^T \boldsymbol{\Omega}_{11} \boldsymbol{\alpha} > 2\phi_{\min}.$$

Note that this is equivalent to the smallest eigenvalue of $\boldsymbol{\Omega}_{11}$ being greater than $2\phi_{\min}$.

(b) $\|\boldsymbol{\Omega}_{11} - \hat{\boldsymbol{\Omega}}_{11}\|_\infty = \max_{ij} \left|\boldsymbol{\Omega}_{11} - \hat{\boldsymbol{\Omega}}_{11}\right|_{ij} \le \frac{\phi_{\min}}{s_T}$

(WEIGHTS1) The weights on inactive endogenous variables diverge fast enough: for some $\xi \in (0,1)$ and $c_w > 0$,

$$\min_{i \in s_T+1:n_T} T^{-\xi/2}\, w_i \;>\; c_w \sqrt{\frac{s_T}{\phi_{\min}}},$$

with probability converging to one

(WEIGHTS2) The weights on the active set are uniformly bounded:

$$\sum_{i=1}^{s_T} w_i^2 < s\, w_{\max,T}^2, \quad w_{\max,T} < T^{\xi/2},$$

with probability converging to one.

(REG) Let $\lambda_T > 0$ be the Adaptive-LASSO penalty parameter. It is required that

$$\frac{n_T^{1/m} T^{(1-\xi)/2}}{\lambda_T} \xrightarrow[T\to\infty]{} 0 \quad \text{and} \quad \frac{\lambda_T}{\sqrt{T}} \frac{s_T^{1/2}}{w_{\max,T}\, \phi_{\min}} \xrightarrow[T\to\infty]{} 0.$$

Further examination of these assumptions reveal interesting patterns. For analysis convenience, define the following dynamics

$$\begin{aligned} n_T &= \lceil c_\delta\, T^\delta \rceil, \\ s_T &= \lceil c_\gamma\, T^\gamma \rceil, \\ \lambda_T &= c_\alpha n_T^{1/m}\, T^{\frac{1-\xi}{2}+\frac{\alpha}{m}}, \\ w_{S,T}^2 &= c_{\beta_S}\, T^{\beta_S}, \quad w_{S^c,T}^2 = c_{\beta_{S^c}}\, T^{\beta_{S^c}}. \end{aligned}$$

Here, $w_{S,T}$ and $w_{S^c,T}$ model the growth order of the active and inactive weights respectively. Basic calculus shows that the assumptions (WEIGHTS1), (WEIGHTS2), and (REG) above can be rewritten as

$$\begin{aligned} \beta_{S^c} - \xi > \gamma, & \qquad \text{(WEIGHTS1)} \\ \beta_S < \xi, & \qquad \text{(WEIGHTS2)} \\ \alpha > 0, \quad \frac{2}{m}\big(\alpha + \delta\big) < \xi - \gamma - \beta_S. & \qquad \text{(REG)} \end{aligned}$$

These inequalities reveal interesting relationships under the light of the following results.

Theorem 3 (Theorem 1 of [38])**.** *Under Assumptions* DGP, DESIGN, WEIGHTS *and* REG*, suppose moreover that*

$$\theta_{\min} > \frac{\lambda_T\, s_T^{1/2}}{T^{1-\xi/2}\, \phi_{\min}}.$$

Then the adaptive LASSO estimator $\hat{\theta}$ satisfies

$$\mathbb{P}\big(\operatorname{sign}(\hat{\theta}) = \operatorname{sign}(\theta_0)\big) \xrightarrow[T\to\infty]{} 1.$$

In the theorem above, the sign function is defined as $\operatorname{sign}(x) = I_{\{x>0\}} - I_{\{x<0\}}$ which implies that $\operatorname{sign}(0) = 0$.

Theorem 4 (Theorem 2 of [38])**.** *Let $\hat{\boldsymbol{\theta}}_{ols}(1)$ denote the OLS estimator of the true active coefficients $\boldsymbol{\theta}_0(1)$. Under the assumptions of Theorem 3, it holds that for any s_T-vector $\boldsymbol{\alpha}$ with $\|\boldsymbol{\alpha}\|_2 = 1$,*

$$\sqrt{T}\, \boldsymbol{\alpha}^T \left(\hat{\boldsymbol{\theta}}(1) - \boldsymbol{\theta}_0(1)\right) = \sqrt{T}\, \boldsymbol{\alpha}^T \left(\hat{\boldsymbol{\theta}}_{ols}(1) - \boldsymbol{\theta}_0(1)\right) \;+\; o_p(1).$$

Theorem 2—often called the *oracle property*—goes one step further: not only does the adaptive LASSO select the correct support with probability tending to one, but its active-set estimates are asymptotically as efficient as if we knew the true model in advance. Moreover, the proof of Theorem 1 shows that, for some $c > 0$ and for a sufficiently large T,

$$\mathbb{P}\big(\operatorname{sign}(\hat{\theta}) \neq \operatorname{sign}(\theta_0)\big) \;\leq\; \frac{c}{T^{\alpha}}, \tag{S3}$$

so the probability of selection failure decays at rate $T^{-\alpha}$. In practice this means that larger α yields faster convergence—yet α itself is bounded by the REG assumption's *window*. Note also that heavier-tailed errors (smaller m) tighten this window, as do steeper growth in the number of covariates, denser true parameter vectors θ_0, or more aggressive penalization on the active set.

E.2 Simulation Design

Simulated data is generated from the DGP

$$y_t = \phi y_{t-1} \;+\; \sum_{j=1}^{n_T} \beta_j \, x_{jt} \;+\; u_t,$$

where,

$$\begin{cases} \mathbf{x}_t = \phi_X \, \mathbf{x}_{t-1} \;+\; \mathbf{L}\, \mathbf{v}_t, & v_{jt} \overset{iid}{\sim} t_{m+1}, \\ u_t = \sigma_t \eta_t & \eta_t \overset{iid}{\sim} t_{m+1}, \\ \sigma_t^2 = \omega + \alpha_\star u_{t-1}^2 + \beta_\star \sigma_{t-1}^2. \end{cases}.$$

Here, ϕ_X is a scalar and $\mathbf{L}$ is chosen to be a lower triangular matrix such that $\mathbf{R} := \mathbf{L}\mathbf{L}^{\mathbf{T}}$, where $\mathbf{R}$ is the desired correlation matrix. Take $\beta_j = 1$ if $j \leq s_T$ and zero otherwise. Take $\phi = 0.8$, $\phi_X = 0.5$, $\omega = 0.1$, $\alpha_\star = 0.1$, $\beta_\star = 0.8$, and $m = 8$. Construct $\mathbf{R}$ starting from the identity matrix and replace its entries $(1,2)$ and $(2,1)$ with ρ (i.e. $\mathbf{R}_{1,2} = \mathbf{R}_{2,1} = \rho$), where $\rho \in \{0, 0.25, 0.5, 0.75, 0.99\}$. The parameters governing the dynamics of the number of candidate variables and penalizing weights are given below.

Table S5: Simulation parameters.

c_α	α	c_{β_S}	β_S	$c_{\beta_{S^c}}$	β_{S^c}	c_γ	γ	c_δ	δ	ξ
1	0.25	1	0.125	1	1.5	2	0.25	1	0.5	0.75

Partitioning the interval $[64, 1024]$ into 15 roughly equally-spaced values of T produces the simulation grid shown in Table S6. Each (T, n_T, s_T) configuration is then replicated $N = 100$ times for each of the specified correlation levels. For each of the samples, adaLASSO is fit assuming equal weights for the active set ($w_{S,T}$) and weights equal to $w_{S^c,T}$ for the inactive set. This helps to control the dynamics of the penalizing weights which in practice emerge from τ and the initial estimate selection algorithm. For simplicity, the weight corresponding to the lagged value of y_t is left close to zero and only one lag is included in $\mathbf{X}$. OLS is then fit and the variance of $\hat{\tilde{\boldsymbol{\beta}}}$ is estimated via the homoskedastic formula (as a benchmark) and the heteroskedasticity-consistent estimator (HCE). Specifically

$$\hat{V}_{\text{hom}} = \hat{\sigma}^2 (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1}; \quad \hat{\sigma}^2 = \frac{1}{T - n_T - 1} \sum_{t=1}^{T} \hat{u}_t^2,$$

$$\hat{V}_{\text{het}} = (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1} \left(\sum_{t=1}^{T} \tilde{\mathbf{x}}_t \tilde{\mathbf{x}}_t^T \hat{u}_t^2 \right) (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1},$$

where in the above the *tilded* quantities denote the corresponding augmented quantity to account for the inclusion of one lag of the endogenous variable. Statistical significance is assessed throught the t test with Bonferroni correction. That is, a variable will be selected following each of the OLS approaches iff

$$\frac{\hat{\tilde{\beta}}_i}{V_{ii}^{1/2}} > t_{T-s_T-1,1-\frac{0.025}{(n_T+1)}}.$$

Following the execution of these three variable selection algorithms. The proportion of correctly selected models is calculated for each of the different $T\rho$-combinations.

Table S6: Simulation grid: sample size T and corresponding $n_T = \lceil c_\delta T^\delta \rceil$, $s_T = \lceil c_\gamma T^\gamma \rceil$.

T	n_T	s_T
64	8	6
133	12	7
202	15	8
270	17	9
339	19	9
407	21	9
476	22	10
544	24	10
613	25	10
682	27	11
750	28	11
819	29	11
887	30	11
956	31	12
1024	32	12

E.3 Results and Discussion

E.3.1 Multicollinearity Effect

Figure S4 (see Section G) displays the exact recovery rate (fraction of simulations where $\text{sign}(\hat{\boldsymbol{\theta}}) = \text{sign}(\boldsymbol{\theta})$) as a function of T, for different correlation levels ρ. Several clear patterns emerge. First, as expected, increasing the sample size improves support recovery for both methods in all scenarios – the curves generally rise toward 1.0 as T grows. However, the adaLASSO (red curves) achieves much higher recovery rates at moderate sample sizes than OLS (blue/green curves) in the presence of strong multicollinearity. For low correlation ($\rho = 0$ or 0.25), the gap between methods is small: OLS with Bonferroni performs nearly as well as adaLASSO when predictors are independent or only mildly correlated. In fact, for $\rho = 0$, both methods eventually reach 100% recovery with sufficiently large T (hundreds of observations), and their trajectories are quite close. This is because when predictors are uncorrelated, the OLS t-tests remain informative (each coefficient can be estimated independently with little variance inflation), so the Bonferroni penalty for multiple testing is the main hurdle – which large T can overcome by reducing standard error. AdaLASSO also works well in this regime, effectively similar to the ordinary LASSO which is known to be consistent under mutual independence of features.

When correlation increases to moderate levels ($\rho = 0.5$ shown in Figure S4, top-right panel, and $\rho = 0.75$ in bottom-left), the performance of OLS selection deteriorates dramatically relative to adaLASSO. For $\rho = 0.5$, the OLS method (blue/green lines) requires a much larger T to approach high recovery probability – there is a noticeable rightward shift of the OLS curve compared to the adaLASSO curve. The adaLASSO achieves near-perfect support recovery at $T = 339$ observations when $\rho = 0.5$, whereas OLS-Bonferroni needs roughly double that sample size to reach comparable accuracy. The gap widens further at $\rho = 0.75$: here the OLS approach almost never succeeds for small or moderate samples (recovery probability essentially 0 for $T < 300$). Even at $T = 1024$, OLS is only recovering the true model in about 50–60% of trials, whereas adaLASSO already exceeds 90% success by that point. the adaLASSO appears more robust to multicollinearity when T is large enough, eventually achieving near-certain recovery even at $\rho = 0.75$ in the simulations (red curve tending to 1.0 as T grows). OLS, however, remains very sensitive: even at $T = 1024$, the OLS method never reaches 100% recovery for $\rho = 0.75$.

The most extreme case is $\rho = 0.99$ (Figure S4, bottom-middle), where predictors are almost perfectly correlated. This scenario is akin to having duplicate or nearly-duplicate variables in the dataset –

a problem for OLS inference. Accordingly, the OLS selection method essentially fails entirely: the blue/green curves at $\rho = 0.99$ sit near zero across all sample sizes tested. The adaLASSO (red curve) also finds this scenario challenging (its recovery probability grows only slowly with T and does not reach 1 even at $T = 1024$), but it clearly outperforms OLS: by $T = 1024$ observations, adaLASSO achieves about 70% exact recovery, whereas OLS is essentially at 0%. This sensitivity to multicollinearity of the adaLASSO approach connects with the assumption DESIGN3 as $\phi_{\min}$ decreases with ρ. The simulation confirms that multicollinearity breaks down the performance of naive OLS variable selection, while adaLASSO is far more resilient in discovering the correct predictors (especially as T becomes large).

To connect with the theoretical result from [38] (see S3), the rate at which the failure probability decreases with T is examined. Figure S5 plots the Monte Carlo estimated failure rates on logarithmic scales. If the failure probability follows a power-law $T^{-\alpha}$, an approximately straight line with slope $-\alpha$ in the log–log representation would be observed. Indeed, for the adaLASSO (red) a roughly linear downward trend in all panels is seen - except when the curve saturates -, consistent with a polynomial decay in T. At every correlation level, the estimated failure rate (red) declines faster than the theoretical bound line (dashed), indicating a steeper power-law exponent. Thus, the adaLASSO simulation results are consistent with the theoretical guarantee of polynomial-decay consistency: even under dependence and heavy-tailed noise, it is observed that $\mathbb{P}\big(\text{sign}(\hat{\theta}) \neq \text{sign}(\theta_0)\big)$ trends down at least as fast as $cT^{-\alpha}$ for some $\alpha > 0$.

The OLS-based methods (blue/green lines in Figure S5) tell a similar story when the multicollinearity level is below or at 50%. However, the failure probability rate of decrease exhibits a slower convergence when the multicollinearity level is at or above 75%. This reflects the fact that OLS with Bonferroni can eventually consistently select the true model when predictors have low to moderate correlation. For higher correlations ($\rho \geq 0.75$), the blue/green points in Figure S5 are almost horizontal or very mildly declining on the log–log scale. This underscores the superior performance of adaLASSO under high multicollinearity regimes when the DGP, DESIGN, WEIGHTS, and REG conditions hold.

E.3.2 Effect of Initial Weights

It might be suggested that adaLASSO may enjoy an unfair edge under the simulation configuration presented, since the true parameter values are "encoded" in the evolution of the penalty weights w_i. To address this concern, a targeted sensitivity experiment was conducted: setting the correlation parameter $\rho = 0$ and raising the inactive-set weights to $\beta_{S^c} = 1.15$, while holding all other hyper-parameters fixed. The results (Figures S6 and S7) show that, under these conditions, adaptive-LASSO requires substantially larger sample sizes before its exact-recovery probability saturates. Crucially, however, once the sample size exceeds a threshold, the failure probability decays very rapidly, in line with the oracle-type behavior predicted by theory.

These findings make clear that careful tuning of the initial-weight construction is important when T is limited or the signal in the inactive set is relatively strong.

E.4 Conclusions and Recommendations

In this Monte Carlo case study the adaLASSO and Bonferroni-corrected t-tests were compared, focusing on exact support recovery and the empirical rate at which the failure probability decays to zero. The main findings are:

- **Exact recovery vs. multicollinearity.** When predictors are uncorrelated or only mildly correlated ($\rho \leq 0.25$), both methods eventually recover the true support with high probability as T grows. However, under moderate to strong multicollinearity ($\rho \geq 0.5$), OLS-Bonferroni requires much larger T and in extreme cases ($\rho = 0.99$) essentially fails, whereas adaLASSO still achieves a decent recovery rate for moderately large samples.

- **Polynomial decay of failure rate.** On a log–log scale, the adaLASSO's failure probability exhibits an approximately linear decline of slope $-\alpha$, consistent with the theoretical bound

$$\Pr\big(\text{sign}(\hat{\theta}) \neq \text{sign}(\theta_0)\big) \;\lesssim\; cT^{-\alpha}.$$

  In contrast, OLS-Bonferroni only attains an adaLASSO-comparable decay when ρ is small; its curve flattens out for higher correlations.

- **Sensitivity to initial-weight tuning.** By increasing the inactive-set exponent β_{S^c}, it was shown that adaLASSO may require larger T before its recovery probability "saturates." Yet once past a moderate threshold, its convergence is very rapid, demonstrating the oracle-property in practice. This underscores the importance of the selection of an initial weights selection method that discriminates inactive variables.

Practical recommendations. Based on these results, practitioners should:

- Employ adaLASSO in high-dimensional, time-series settings where multicollinearity is present, since it substantially outperforms OLS-based selection under dependence.
- Carefully tune the hyperparameters of the initial weights selection.
- Use OLS-Bonferroni as a simple baseline when predictors are mildly correlated.
- Validate selection performance before deploying the method in empirical applications.

Overall, the present study confirms that adaLASSO not only achieves model selection consistency and the oracle property under the Medeiros & Mendes framework, but also delivers robust performance in finite samples—especially when multicollinearity and heteroskedastic, heavy-tailed errors are present.

E.5 Reproducibility

All the simulation results presented here can be reproduced using the code available in the GitHub repository `judmejiabe/case-study-adaLASSO`. The repository contains detailed scripts for generating the data, running the adaptive LASSO and OLS selection procedures, and producing the figures.

F Events Matching Exclusion Criteria on the INTC MBO Dataset

This section lists the dates excluded from the analysis as a consequence of meeting any of the excluding criteria highlighted in Section 10.2. The exclusion criteria is reproduced below for the reader's convenience.

- Corporate actions: Ex-dividend dates, stock splits or reverse splits, share buybacks and rights issues, spin-offs, mergers, or acquisitions
- Earnings and company-specific news: Quarterly earnings announcements and guidance revisions, joint ventures, large partnerships or asset sales, regulatory actions or class-action lawsuits, product launches or major technological disclosures
- Derivative market events: Monthly options expirations (OPEX), quarterly triple witching weeks (stock index futures, index options, and single-stock options expiry), index futures rollovers (e.g., NASDAQ-100, S&P 500), index and ETF quarterly rebalancing
- Scheduled announcements: U.S. Employment Situation (non-farm payrolls), Consumer Price Index (CPI), Producer Price Index (PPI), PCE releases, FOMC interest rate decisions and press conferences, other major macroeconomic indicators (e.g., GDP, JOLTS)
- Exchange halts: Limit-Up/Limit-Down (LULD) volatility pauses, market-wide circuit breakers, exchange-initiated suspensions (technical or regulatory).

F.1 Corporate Actions

- May 6, 2024. Event Type: Ex-dividend date. Description: Ex-dividend date for quarterly cash dividend of $0.125 per share. Source: `https://stocklytics.com/stocks/intc/dividends`

F.2 Earnings and Company-Specific News

- April 25, 2024 (after market close, impacting May 1). Event Type: Earnings release. Description: Intel reported Q1 2024 earnings, the first quarter under its internal foundry reporting model; the Foundry segment revenue was $4.4 billion (down 10% year-over-year). Source: `https://www.intc.com/news-events/press-releases/detail/1635/intel-reports-first-quarter-2024-financial-results`
- May 8, 2024. Event Type: Regulatory action (export license revocation). Description: The U.S. Commerce Department revoked certain export licenses for shipments to Huawei, affecting Intel's ability to supply that customer. Source: `https://www.marketwatch.com/story/intels-stock-drops-after-u-s-revokes-licenses-for-exports-to-china-customer-f057da21`
- June 4, 2024. Event Type: Joint venture. Description: Intel and Apollo announced a definitive agreement under which Apollo acquired a 49% equity stake in a joint venture related to Intel's Fab 34 facility in Leixlip, Ireland (Intel retains operational control). Source: `https://newsroom.intel.com/corporate/co-invest-program-news-2024`
- June 17, 2024. Event Type: Class action lawsuit. Description: Bronstein, Gewirtz & Grossman announced a securities-fraud class action against Intel regarding statements about the internal foundry model and foundry losses (class period Jan 25–Apr 25, 2024). Source: `https://www.globenewswire.com/de/news-release/2024/05/13/2880132/0/en/INTC-INVESTOR-ALERT-Bronstein-Gewirtz-Grossman-LLC-Announces-that-Intel-Corporation-Investors-with-Substantial-Losses-Have-Opportunity-to-Lead-Class-Action-Lawsuit.html`
- June 21, 2024. Event Type: Class action lawsuit notice. Description: Levi & Korsinsky announced a class action filing and lead-plaintiff deadline of July 2, 2024, reiterating allegations concerning Intel's internal foundry disclosures. Source: `https://www.prnewswire.com/news-releases/class-action-filed-against-intel-corporation-intc---july-2-2024-deadline-to-join--contact-levi--korsinsky-302178578.html`

F.3 Derivative Market Events

- May 17, 2024. Event Type: Options expiration (OPEX). Description: Monthly U.S. equity and ETF options expiration. Source: https://www.macroption.com/options-expiration-calendar/
- June 17-21, 2024. Event Type: Triple witching week. Description: Simultaneous expiration of stock index futures, stock index options, and single-stock options, coinciding with June OPEX. Source: https://www.bankrate.com/investing/triple-witching/

F.4 Scheduled Announcements

- May 3, 2024. Event Type: U.S. Employment Situation (April NFP). Description: BLS reported non-farm payrolls at 175k (below consensus), released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/empsit_05032024.htm
- May 15, 2024. Event Type: CPI release (April). Description: Consumer Price Index data released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/cpi_05152024.htm
- June 4, 2024. Event Type: JOLTS (Job Openings). Description: The Bureau of Labor Statistics released the April Job Openings and Labor Turnover Survey at 10:00 a.m. ET. Source: https://www.bls.gov/news.release/archives/jolts_06042024.htm
- June 7, 2024. Event Type: U.S. Employment Situation (May NFP). Description: Non-farm payrolls release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/empsit_06072024.htm
- June 12, 2024. Event Type: CPI release (May). Description: Consumer Price Index release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/cpi_06122024.htm
- June 12, 2024. Event Type: FOMC decision. Description: Federal Reserve announced policy decision and released updated Summary of Economic Projections at 2:00p ET, followed by press conference. Source: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm
- June 13, 2024. Event Type: PPI release (May). Description: Producer Price Index release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/ppi_06132024.htm
- June 14, 2024. Event Type: Import/Export Prices (May). Description: BLS Import and Export Price Indexes released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/ximpim_06142024.htm

F.5 Exchange Halts

- None. No Nasdaq trading halts were recorded for INTC between May 1 and June 30, 2024. Source: Nasdaq Trader Trading Halts database https://www.nasdaqtrader.com/Trader.aspx?id=TradeHalts

G Complimentary Tables and Figures

Table S7: Base configuration used in synthetic data experiments.

Category	Parameter	Value
General	Ticker	SIM (Simulated LOB)
	Δ^B	$5'000{,}000.0$ uot
	Δ^A	100,000.0 uot
	δ	100.0 uot
	Seed	19991810
	Session start	0.0 uot
	Session finish	$10'000{,}000.0$ uot
	Target Variable	r^w
Backbone	Training Epochs	10
	Batch size	32
	Validation proportion	0.1
	Buffer proportion	0.05
	Windsorization quantile	0.005
	ADF rejection threshold	0.8
	KPSS verification threshold	0.2
	NA threshold	0.1
	Learning rate	0.0001
	Correlation threshold	0.95
Interface	α	$\{0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0\}$
	λ_1	logspace$(-4, 1, 25)$
	τ	$\{0.25, 0.5, 1.0, 1.5, 2.0\}$
	λ_2	logspace$(0, 1, 25)$
	Validation proportion	0.2
	Selection threshold	0.17 ($L = 50$, $\alpha = 0.00005$)
	Tolerance threshold	10^{-6}
	OLS significance level	0.05
	Surrogate model lags	5
Adapter	ARMA order	(1,1)
	GARCH order	(1,1)
	Error distribution	Skewed t

Table S8: Base configuration used in the experiments with market data.

Category	Parameter	Value
General	Ticker	INTC
	Δ^B	7,200.0 sec
	Δ^A	600.0 sec
	δ	2.0 sec
	Seed	One per date pair
	Session start	9:45 AM EST
	Session finish	3:45 PM EST
	Target Variable	r^w
Backbone	Training Epochs	10
	Batch size	32
	Validation proportion	0.1
	Buffer proportion	0.05
	Windsorization quantile	0.005
	ADF rejection threshold	0.8

Table S8 (continued)

Category	Parameter	Value
	KPSS verification threshold	0.2
	NA threshold	0.2
	Learning rate	0.0001
	Correlation threshold	0.95
	α	{0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0}
	λ_1	logspace$(-4, 1, 25)$
	τ	{0.25, 0.5, 1.0, 1.5, 2.0}
	λ_2	logspace$(0, 1, 25)$
Interface	Validation proportion	0.2
	Selection threshold	0.29 ($L = 12$, $\alpha = 0.00005$)
	Tolerance threshold	10^{-6}
	OLS significance level	0.05
	Surrogate model lags	5
	ARMA order	(0,1)
Adapter	**EGARCH** order	(1,1)
	Error distribution	Negative Inverse Gaussian
	Update Rule	Exponential Smoothing

Table S9: RelMAE-based overall rankings and average RelMAE values for each TTSTL configuration across the backbone bars in the INTC MBO dataset. DP denotes the date pair, UR the update rule, CC the covariate configuration, Rank BBX the overall ranking within backbone bar X, and RelMAE BBX the corresponding average forecasting error.

DP	UR	CC	Rank BB0	RelMAE BB0	Rank BB1	RelMAE BB1	Rank BB2	RelMAE BB2
1	ES010	01	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	02	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	03	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	04	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	05	6.0	100.02%	6.0	100.75%	6.0	99.20%
1	ES010	06	3.0	99.98%	3.0	100.63%	3.0	99.17%
2	ES010	01	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	02	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	03	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	04	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	05	6.0	100.43%	6.0	100.75%	1.0	100.32%
2	ES010	06	3.0	99.84%	3.0	100.56%	4.0	100.39%
3	ES010	01	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	02	3.0	99.85%	3.0	100.17%	5.5	100.59%
3	ES010	03	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	04	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	05	6.0	99.91%	6.0	100.32%	5.5	100.59%
3	ES010	06	3.0	99.85%	3.0	100.17%	2.5	100.53%
4	ES010	01	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	02	3.0	99.64%	1.5	101.04%	4.0	101.48%
4	ES010	03	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	04	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	05	6.0	100.45%	1.5	101.04%	1.0	101.38%
4	ES010	06	3.0	99.64%	4.5	102.53%	4.0	101.48%
5	ES010	01	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	02	3.0	99.93%	1.5	99.76%	4.0	99.77%
5	ES010	03	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	04	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	05	6.0	100.06%	1.5	99.76%	1.0	99.77%
5	ES010	06	3.0	99.93%	4.5	99.80%	4.0	99.77%

Continued on next page

Table S9 – continued from previous page

DP	UR	CC	Rank BB0	RelMAE BB1	Rank BB1	RelMAE BB2	Rank BB3	RelMAE BB2
6	ES010	01	4.0	100.40%	3.5	102.23%	2.5	100.14%
6	ES010	02	4.0	100.40%	5.5	131.02%	5.5	127.17%
6	ES010	03	4.0	100.40%	1.5	100.67%	2.5	100.14%
6	ES010	04	4.0	100.40%	3.5	102.23%	2.5	100.14%
6	ES010	05	1.0	100.36%	5.5	131.02%	5.5	127.17%
6	ES010	06	4.0	100.40%	1.5	100.67%	2.5	100.14%
7	ES010	01	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	02	3.0	99.99%	5.5	112.60%	3.0	99.26%
7	ES010	03	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	04	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	05	6.0	100.03%	5.5	112.60%	6.0	99.27%
7	ES010	06	3.0	99.99%	2.5	99.00%	3.0	99.26%
8	ES010	01	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	02	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	03	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	04	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	05	6.0	98.67%	1.0	100.88%	6.0	100.27%
8	ES010	06	3.0	98.15%	4.0	101.29%	3.0	100.14%
9	ES010	01	3.0	99.77%	2.5	100.34%	2.0	100.46%
9	ES010	02	3.0	99.77%	5.5	101.93%	5.0	100.56%
9	ES010	03	3.0	99.77%	2.5	100.34%	5.0	100.56%
9	ES010	04	3.0	99.77%	2.5	100.34%	1.0	100.46%
9	ES010	05	6.0	99.75%	5.5	101.93%	3.0	100.68%
9	ES010	06	3.0	99.77%	2.5	100.34%	5.0	100.56%
10	ES010	01	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	02	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	03	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	04	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	05	6.0	100.70%	6.0	100.59%	6.0	99.88%
10	ES010	06	3.0	100.33%	3.0	100.57%	3.0	99.83%

$$r_j^w \ \mathrm{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right) OFI_{0,j}$$

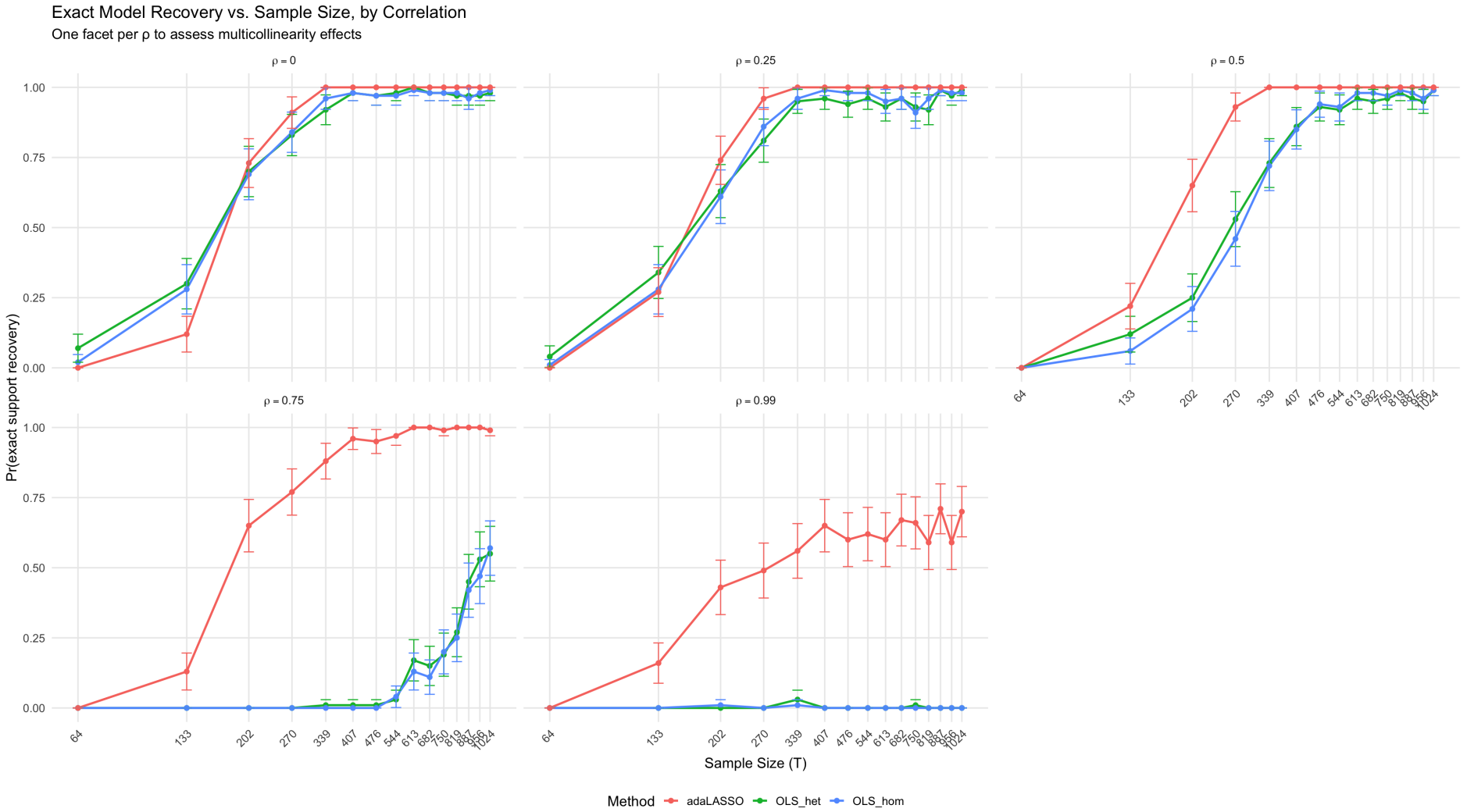


Figure S4: Probability of exact support recovery versus sample size (T) for adaptive LASSO (red) and OLS-Bonferroni (blue/green), under varying predictor correlation ρ. Each panel corresponds to a different correlation level among predictors (from $\rho = 0$ no correlation, up to $\rho = 0.99$ extreme collinearity). Error bars (± 1.96 SE) are shown for the Monte Carlo estimates.

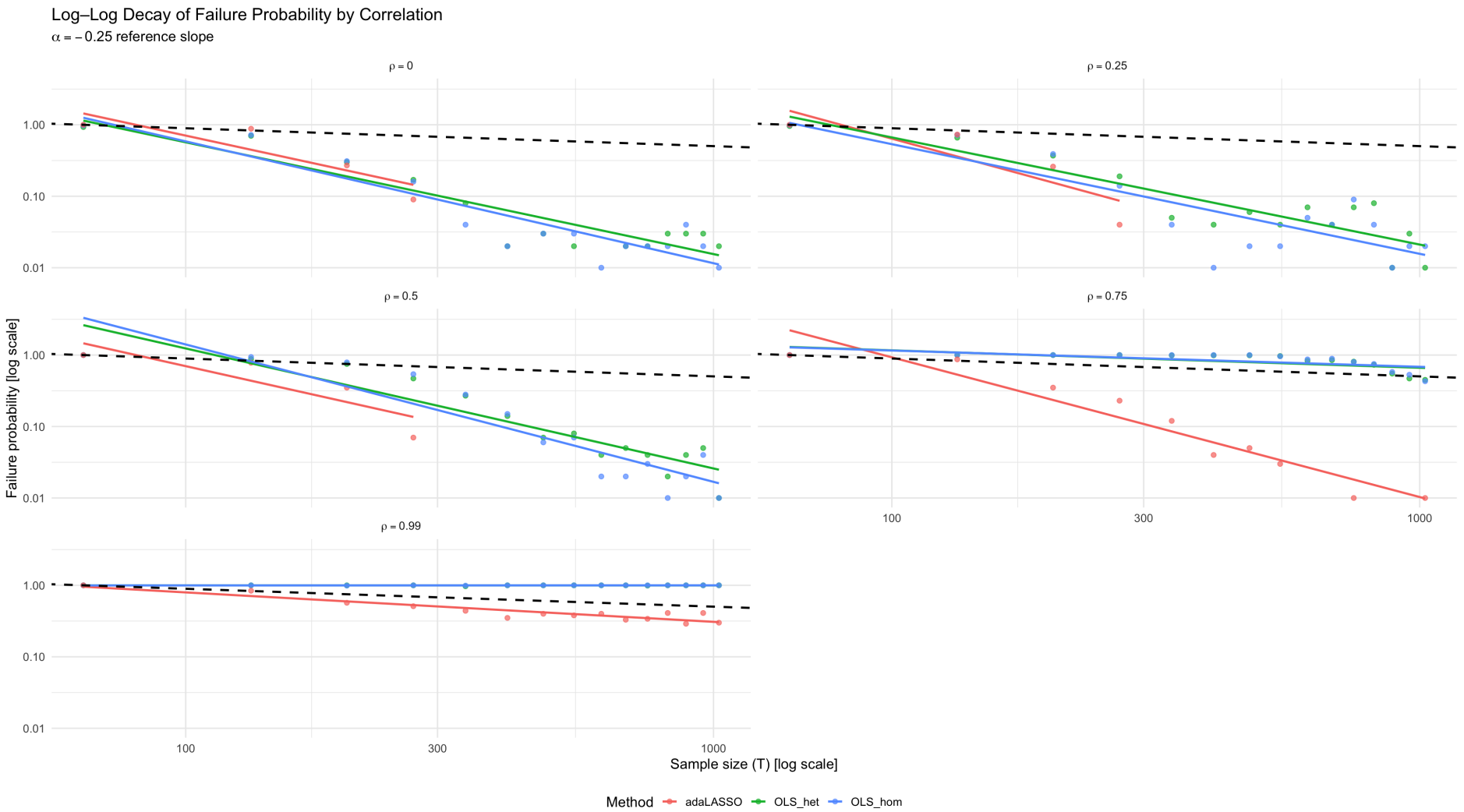


Figure S5: Log–log plot of support recovery failure probability ($\mathbb{P}(\text{sign}(\hat{\boldsymbol{\theta}}) \neq \text{sign}(\boldsymbol{\theta}))$) versus sample size T, for different correlation levels. Each panel shows the decay of failure probability as T increases on a log–log scale. The red points/line correspond to adaLASSO; blue/green to OLS (homoskedastic or robust OLS, which behave similarly). The black dashed line in each panel indicates a reference slope of -0.25 (a power-law $T^{-0.25}$) as a guide.

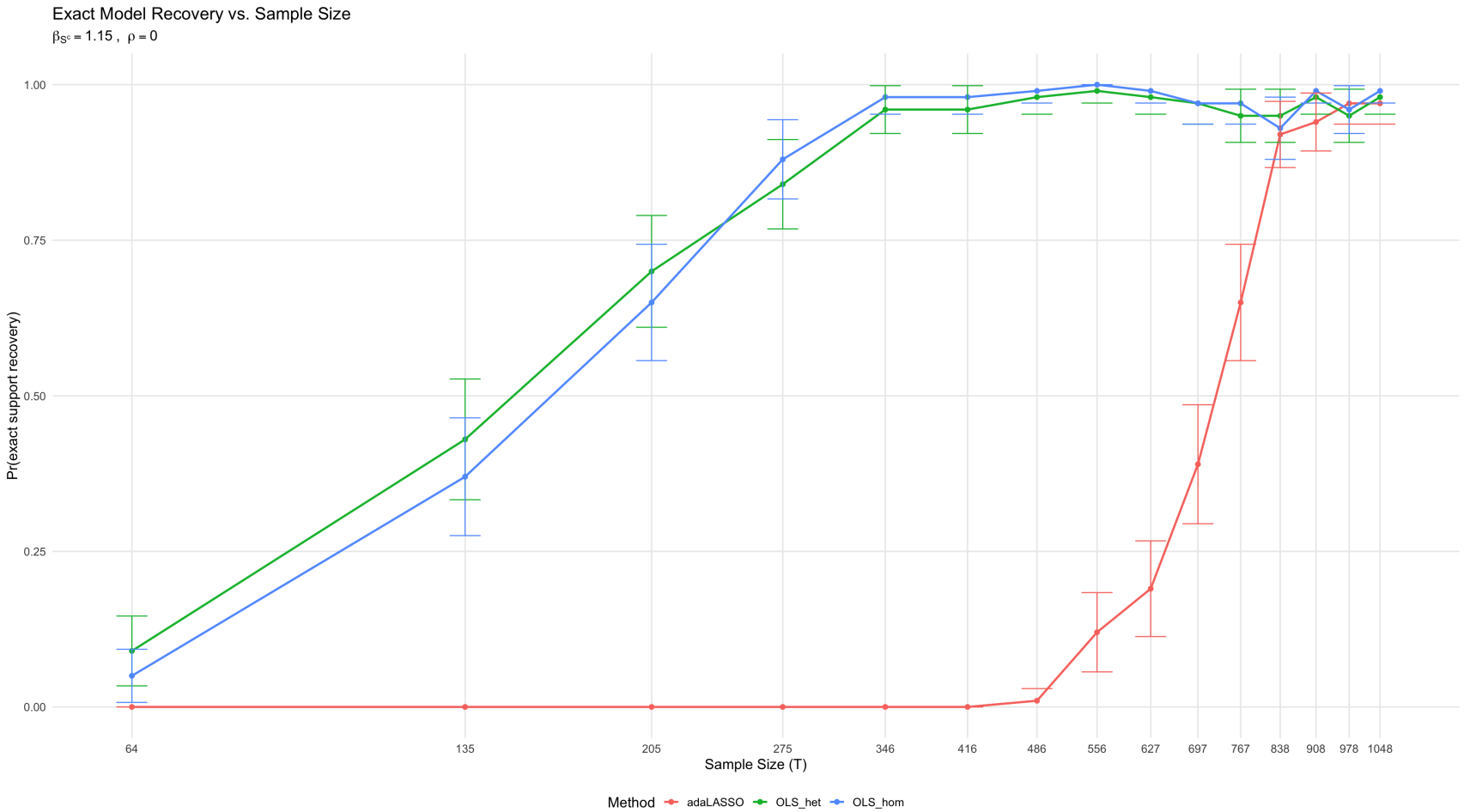


Figure S6: Probability of exact support recovery versus sample size (T) for adaptive LASSO (red) and OLS-Bonferroni (blue/green), under uncorrelated covariates and a comparatively slower discrimination rate $\beta_{S^c} = 1.15$.

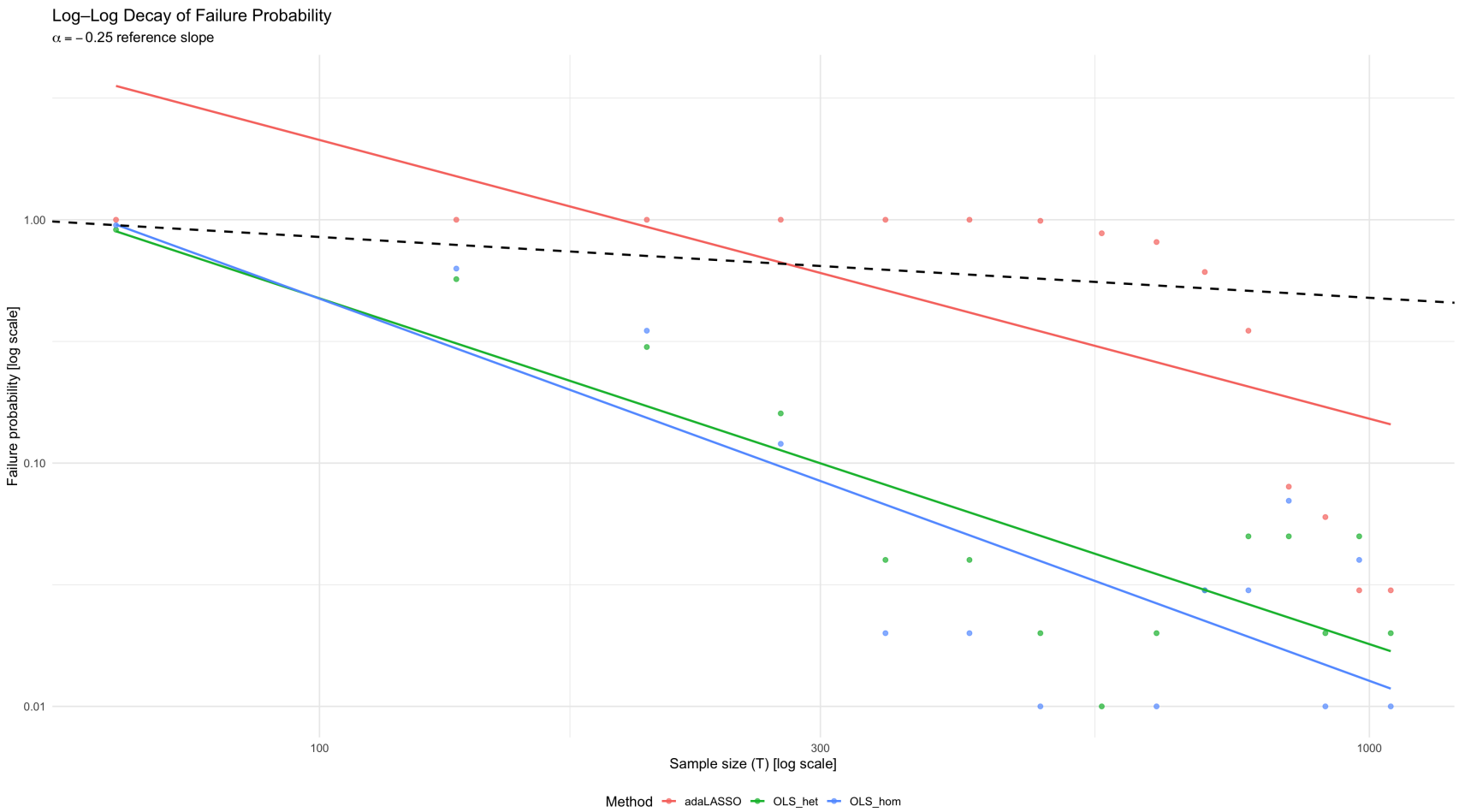


Figure S7: Log–log plot of support recovery failure probability ($\mathbb{P}(\text{sign}(\hat{\boldsymbol{\theta}}) \neq \text{sign}(\boldsymbol{\theta}))$) versus sample size T, assuming uncorrelated variables and a comparatively slower discrimination rate $\beta_{S^c} = 1.15$.

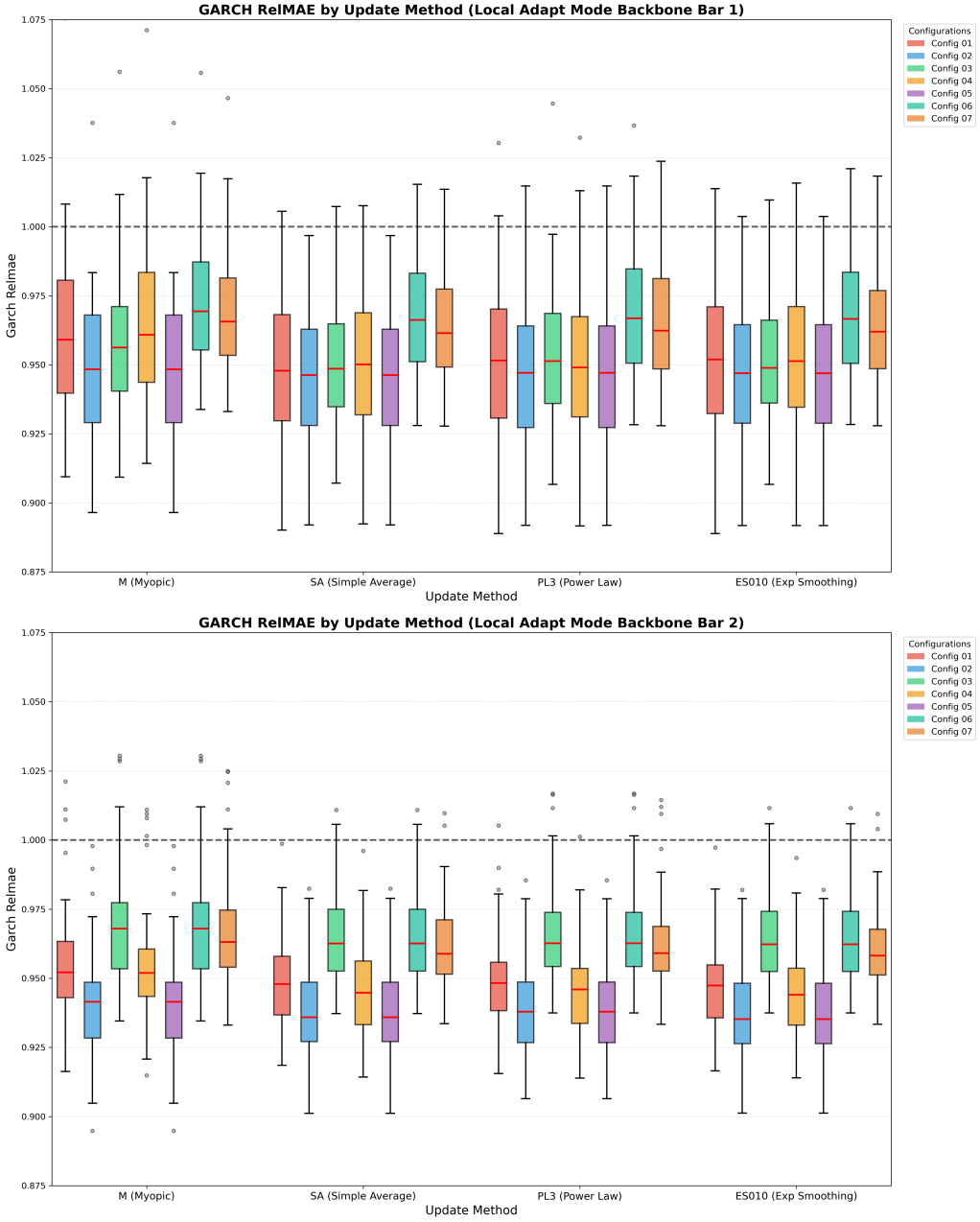


Figure S8: GARCH RelMAE comparison across update methods and configurations.

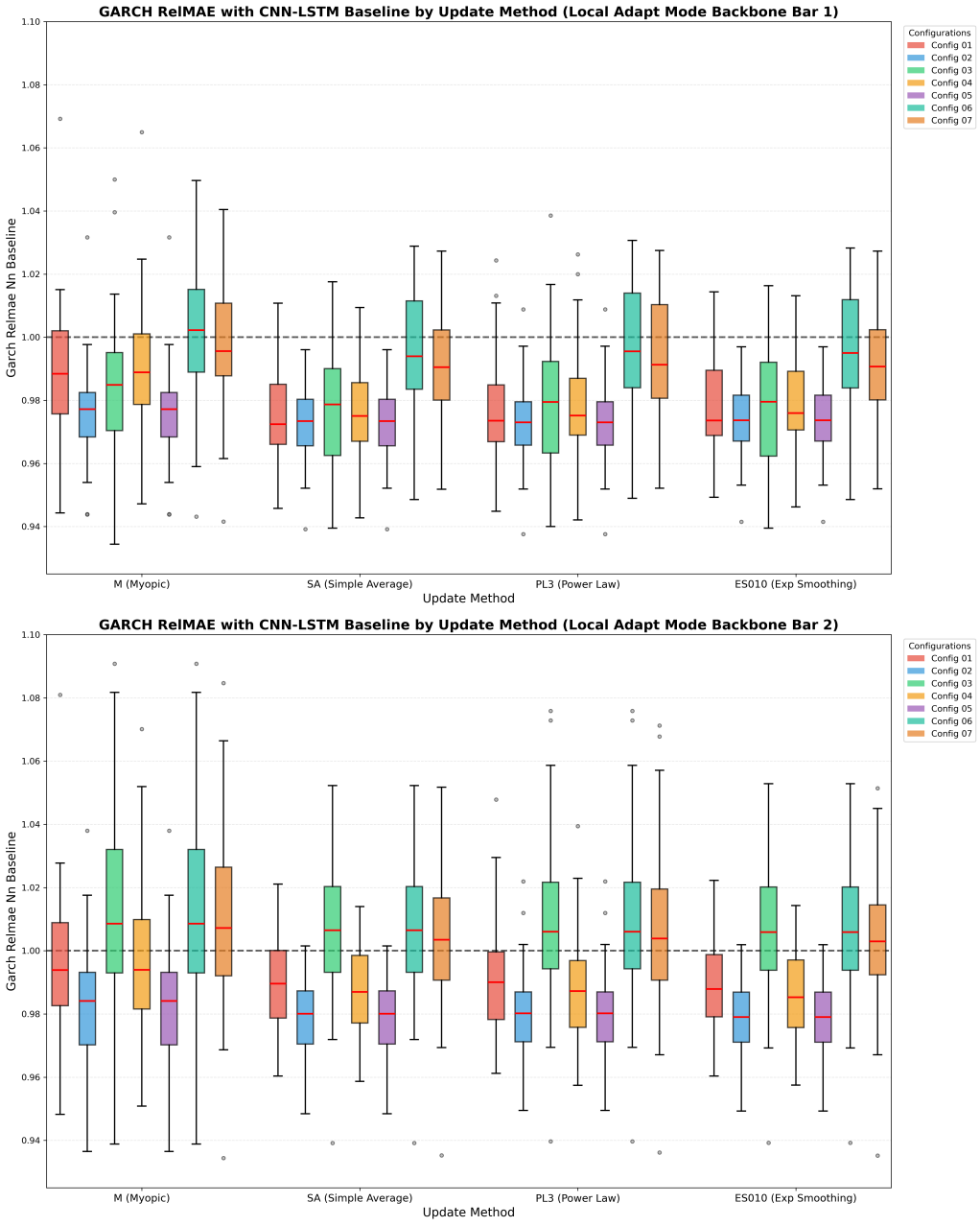


Figure S9: GARCH RelMAE comparison across update methods and configurations with CNN-LSTM baseline.

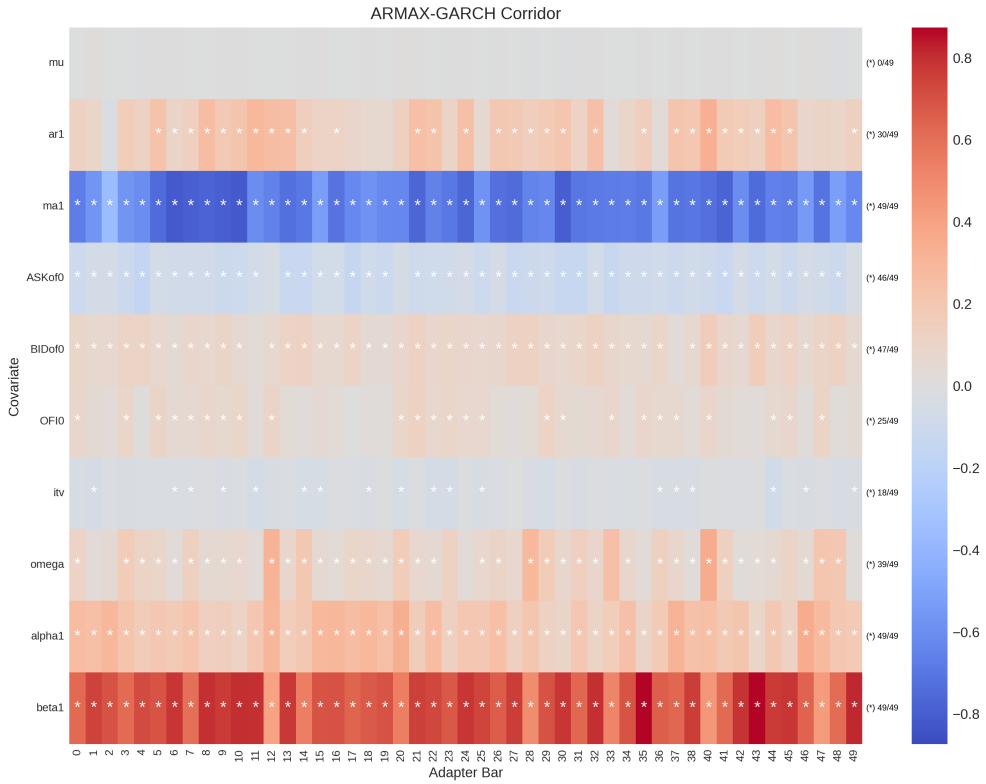


Figure S10: ARMAX–GARCH-DeepLOB corridor for configuration `SA_03`. Cells show bar-specific coefficient magnitudes by adapter bar; asterisks mark statistical significance.

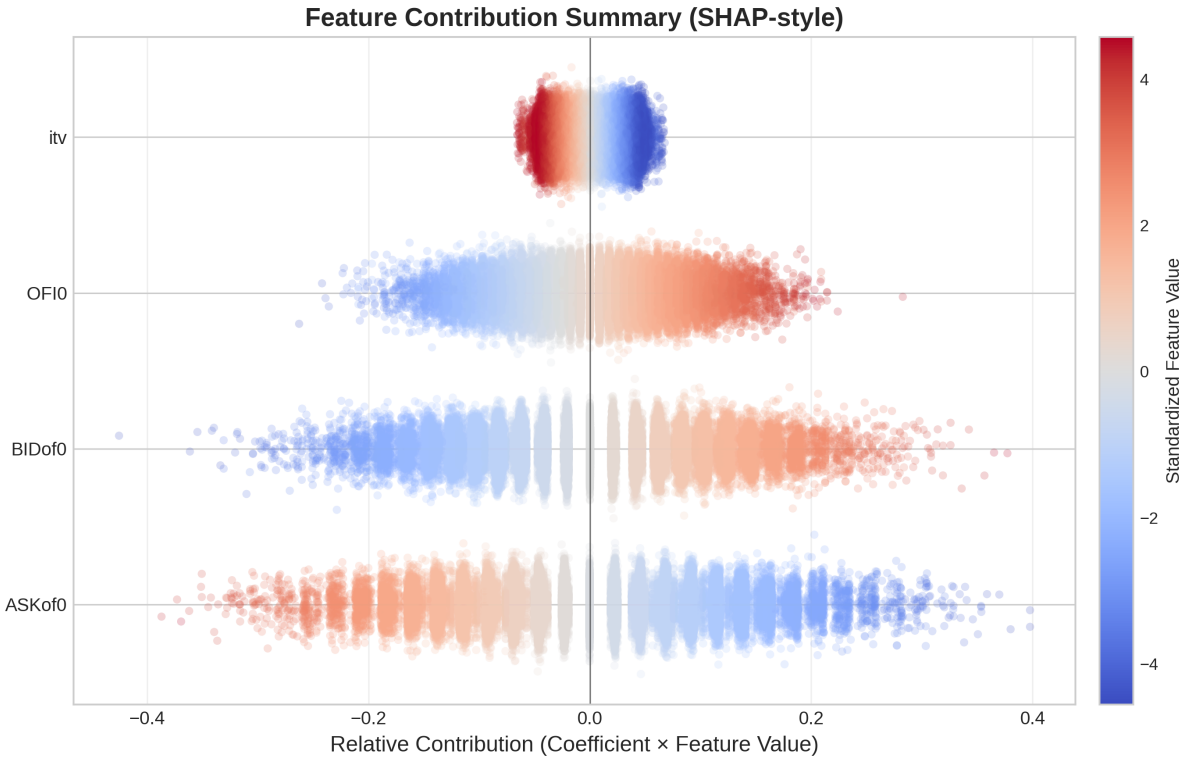


Figure S11: ARMAX–GARCH-DeepLOB SHAP scores `SA_03`. The figure plots the product of the coefficient and the feature value for each sample.



Figure S12: Evolution of parameter estimates for configuration `SA_03`. Blue lines denote combined estimates, while red dashed lines denote bar-specific estimates across adapter bars.

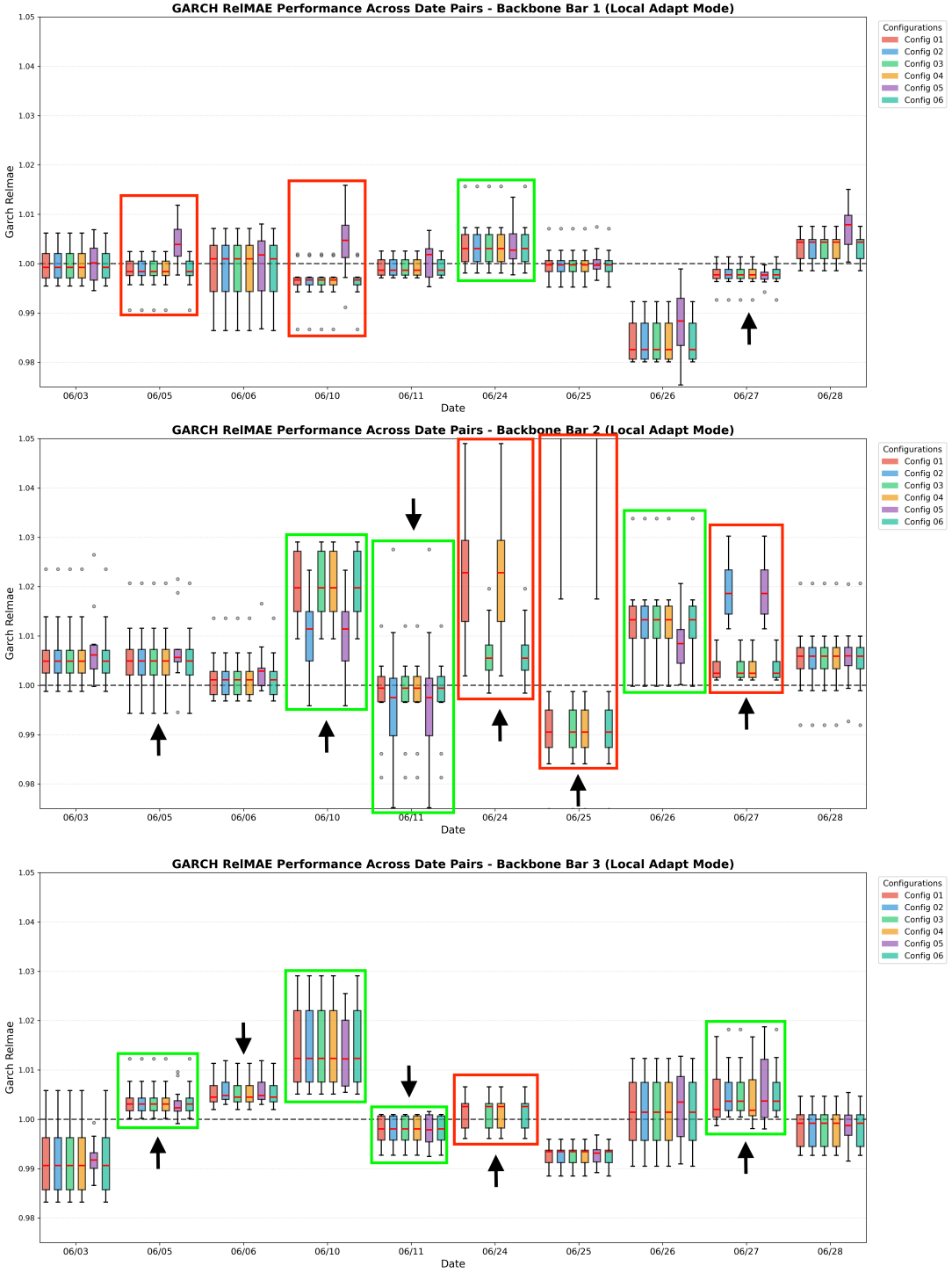


Figure S13: Sequential RelMAE boxplots for the INTC dataset across date pairs in Local Adapt mode for Backbone Bar 1 (top), Backbone Bar 2 (middle), and Backbone Bar 3 (bottom). The horizontal dashed line at 1.00 represents the naive model baseline. Green rectangles highlight instances where covariate configurations `05` (raw CNN-LSTM output) or `01` (full configuration) outperformed configuration `06` (no covariates). Red rectangles indicate date pairs where configuration `05` exhibited significant underperformance. Black arrows mark backbone bars where the CNN-LSTM weights were updated during the preceding Global Retrain step. Note that the boxplots corresponding to configurations `02` and `05` of Backbone Bars 2 and 3 on 06/24, and Backbone Bar 2 on 06/27, lie above the superior margin.

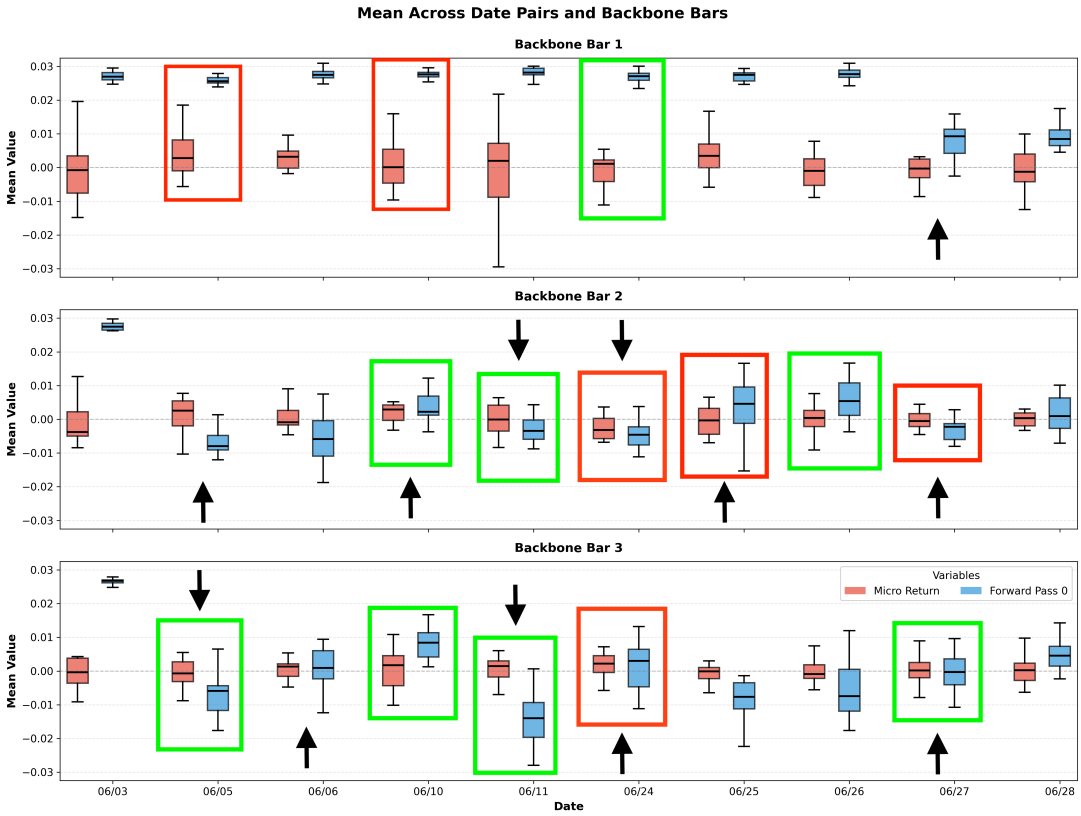


Figure S14: Distribution of the **mean** of the Micro-Return (r_j^w) and CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

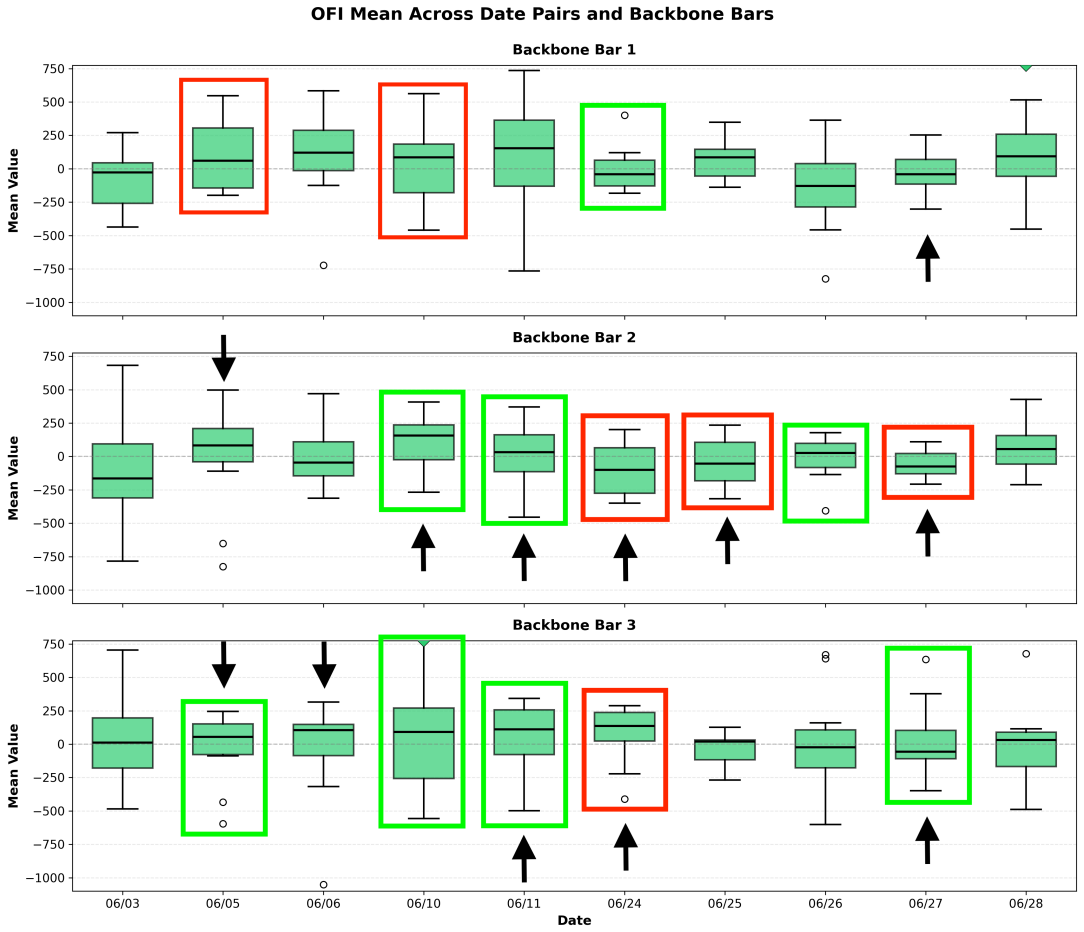


Figure S15: Distribution of the **mean** of the Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

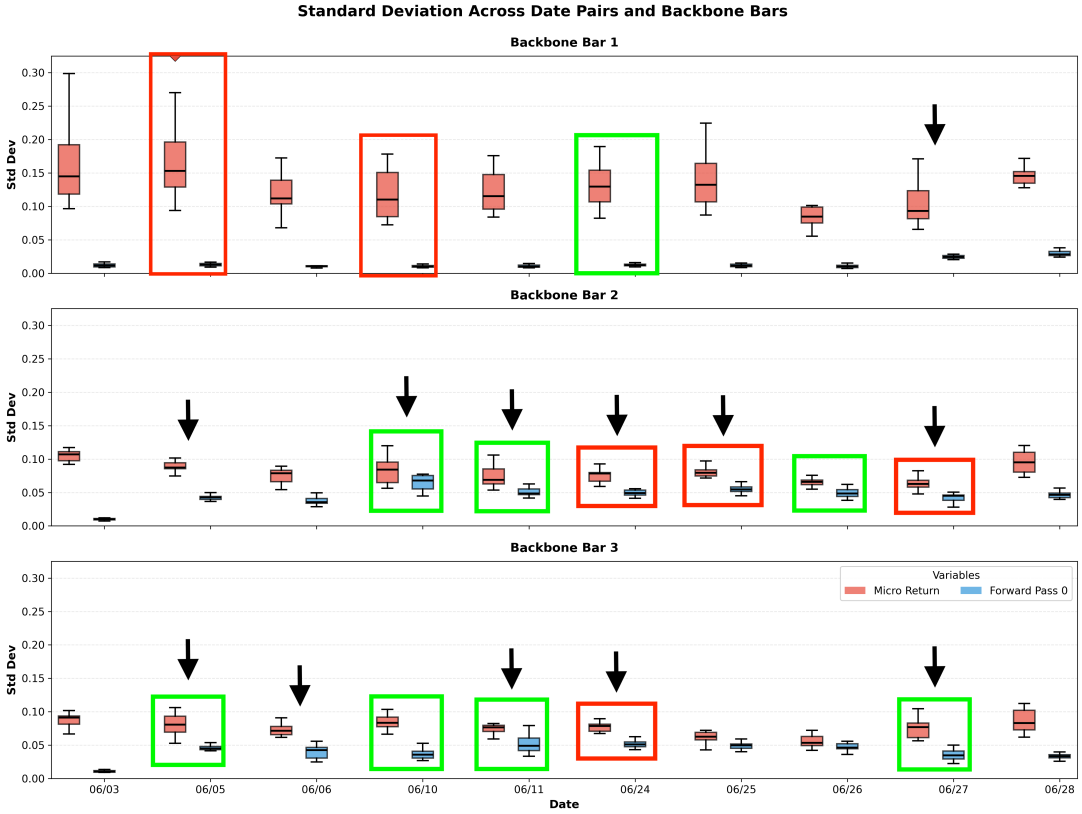


Figure S16: Distribution of the **standard deviation** of the Micro-Return (r_j^w) and CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

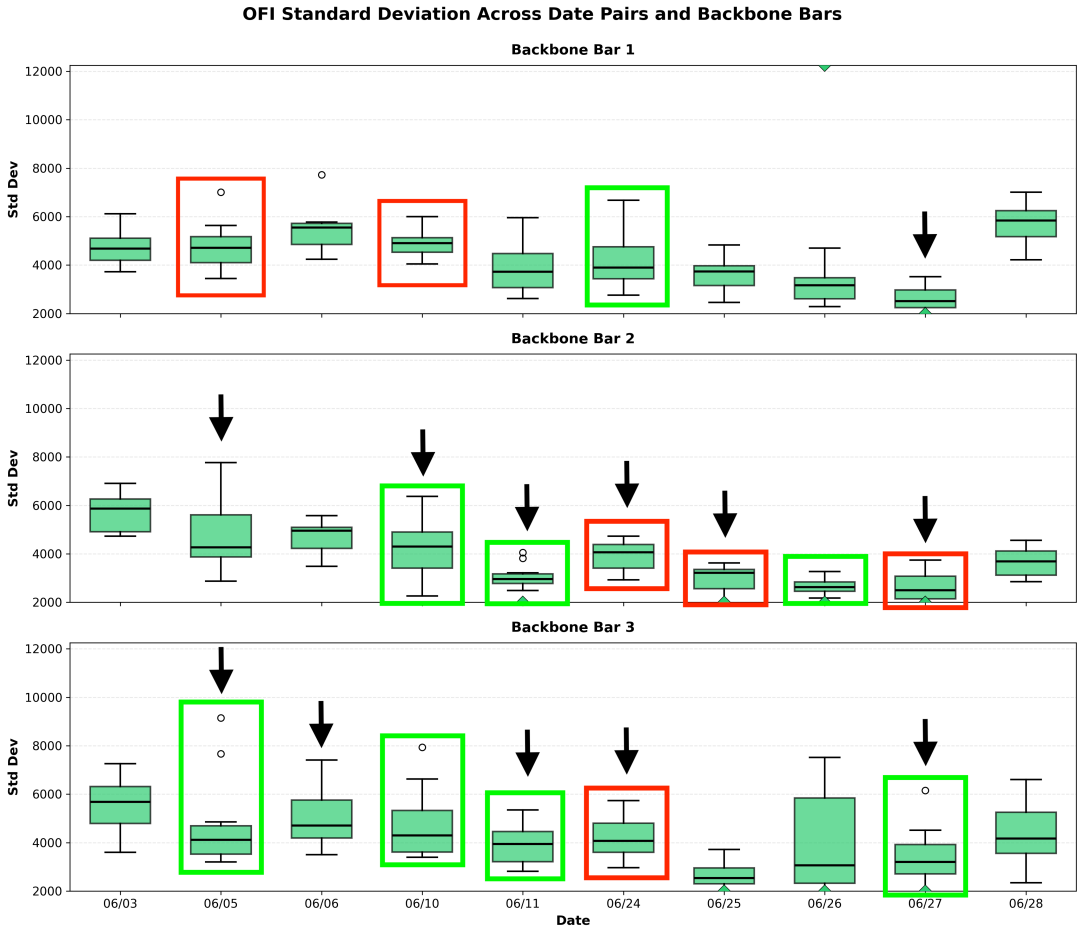


Figure S17: Distribution of the **standard deviation** of the Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

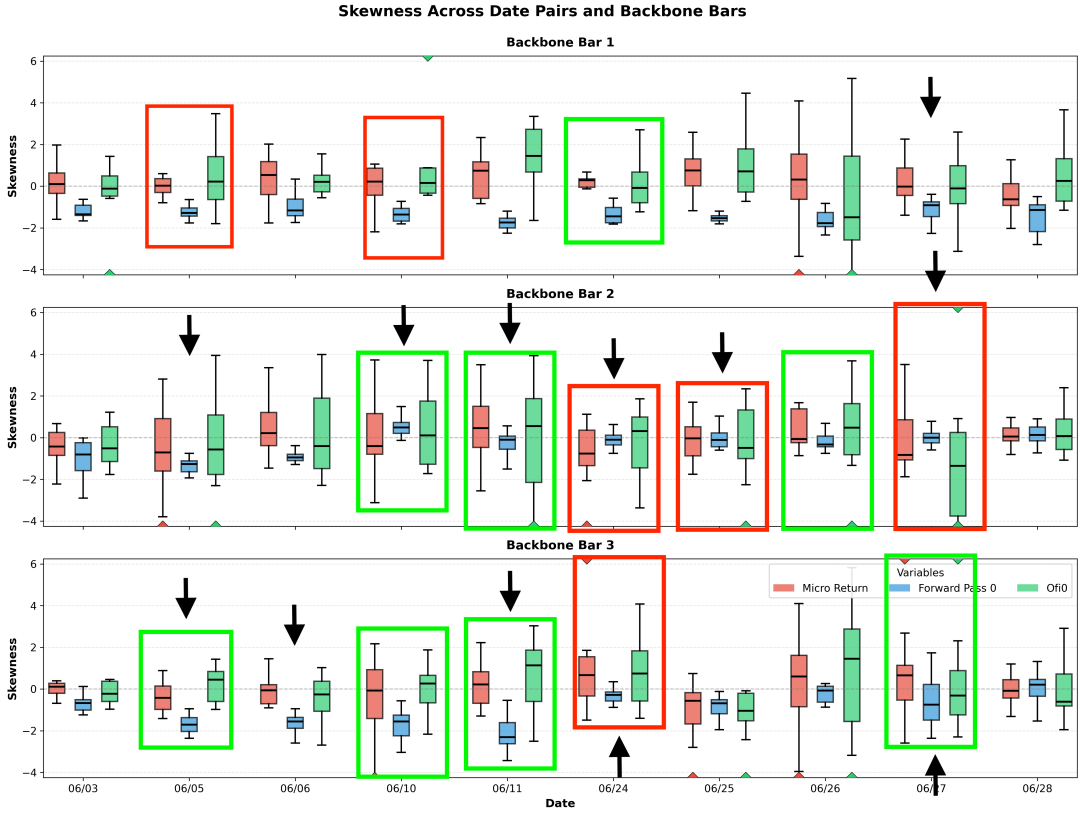


Figure S18: Distribution of the **skewness** of the Micro-Return (r_j^w), CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$), and Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

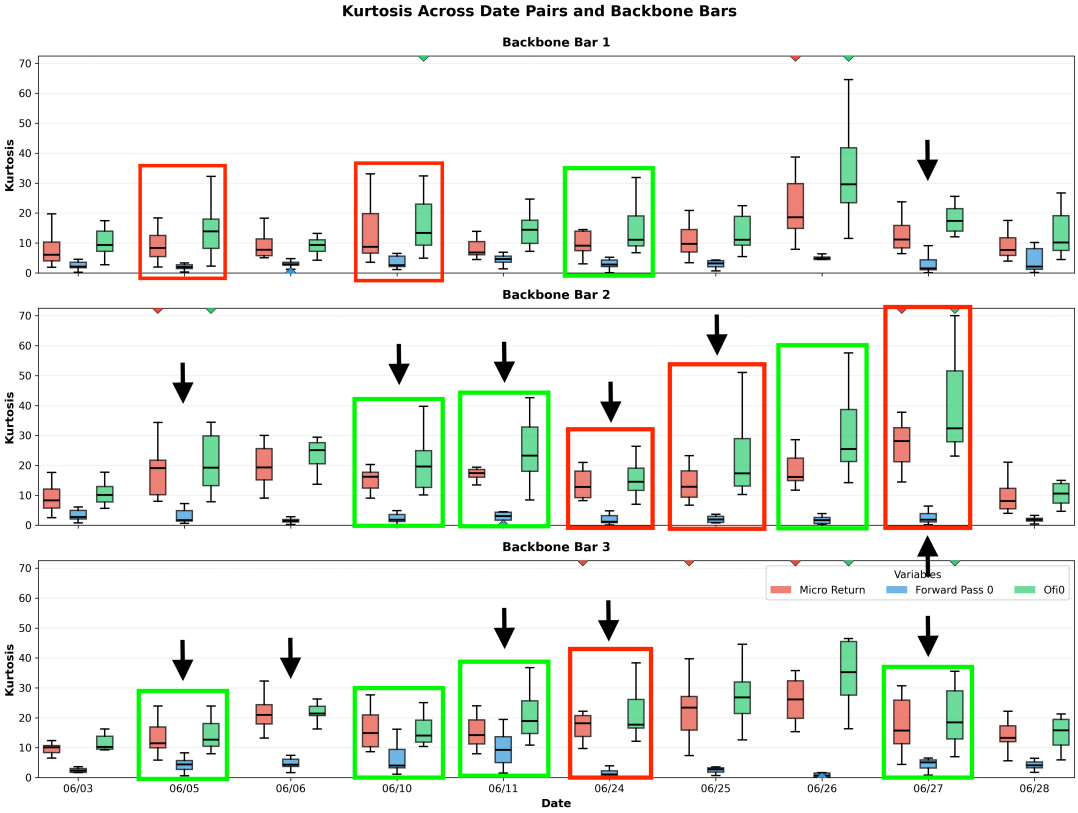


Figure S19: Distribution of the **kurtosis** of the Micro-Return (r_j^w), CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$), and Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

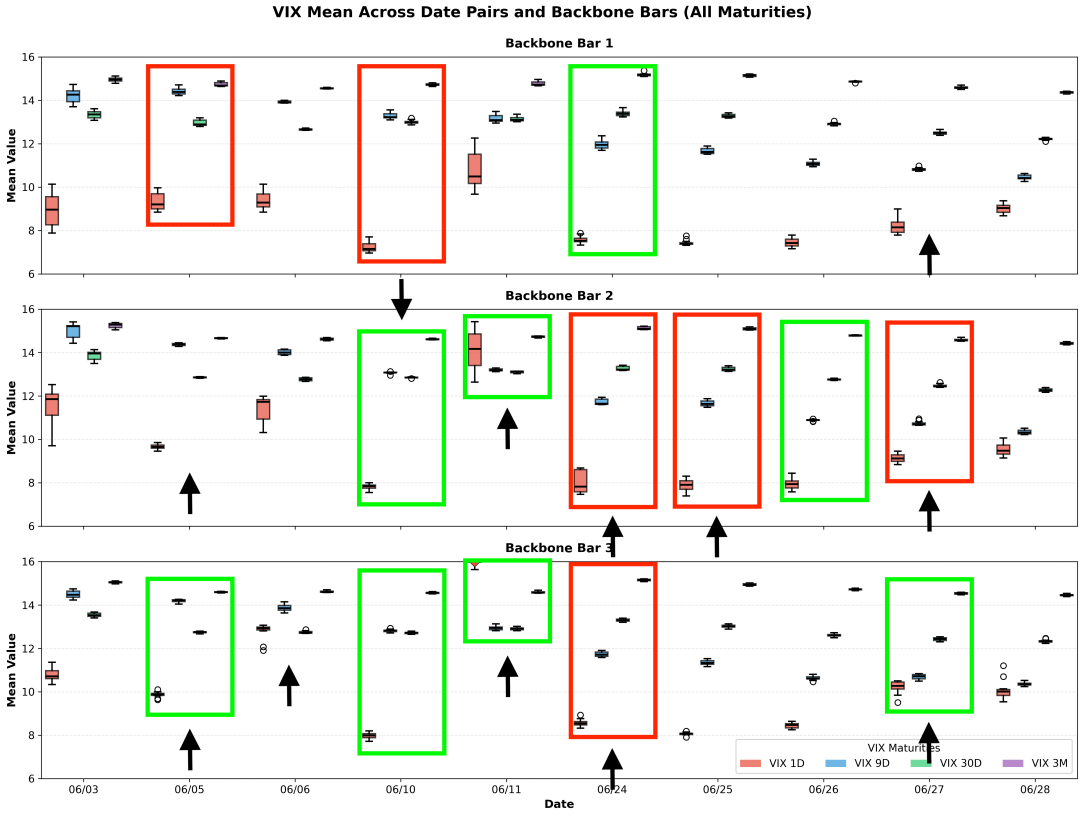


Figure S20: Distribution of the **mean** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

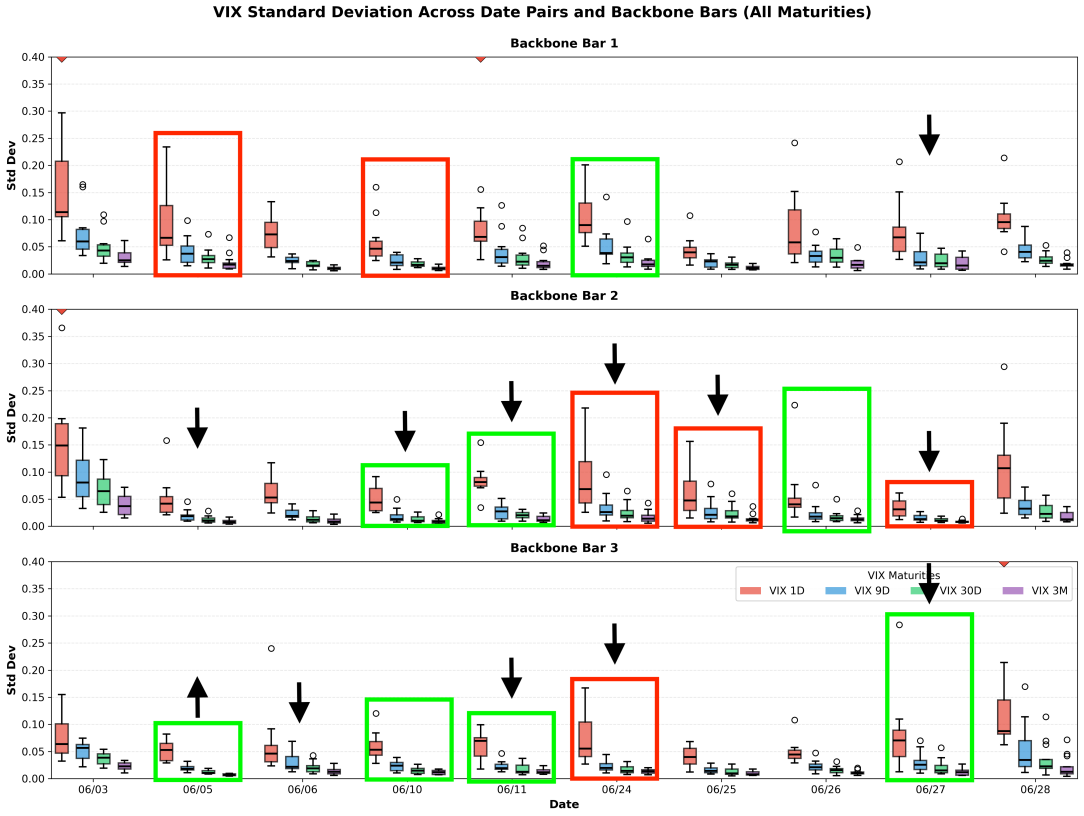


Figure S21: Distribution of the **standard deviation** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

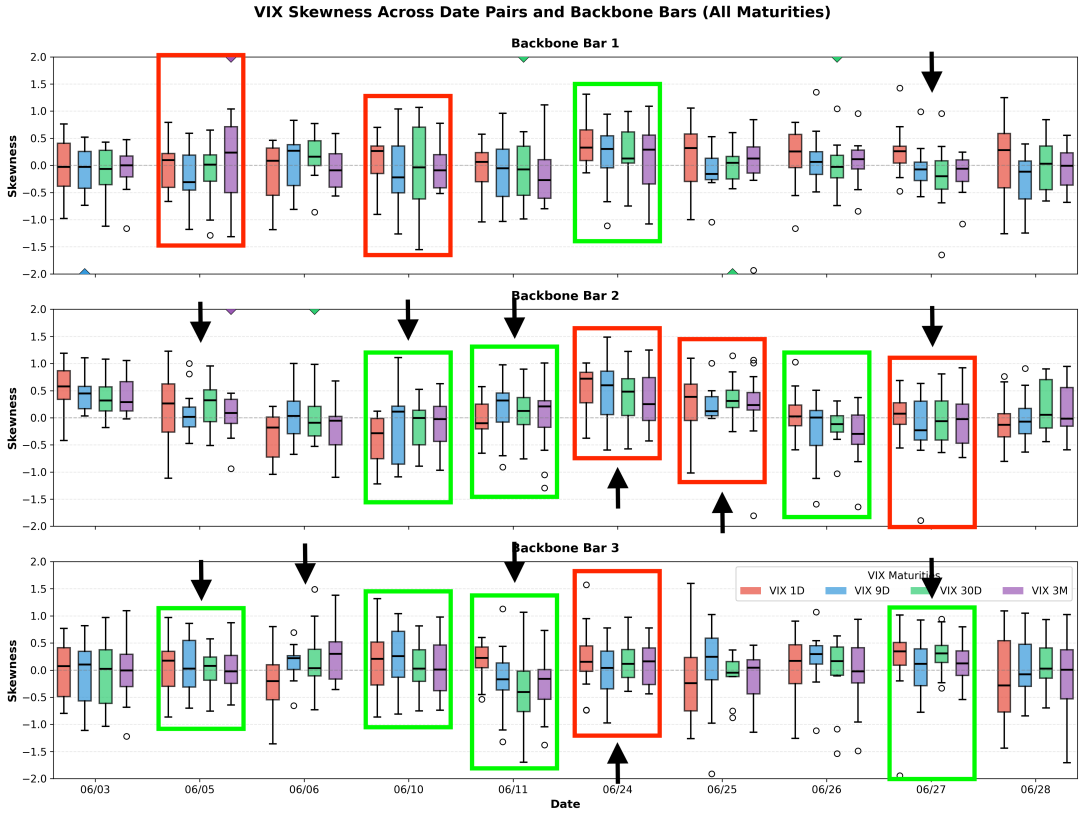


Figure S22: Distribution of the **skewness** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

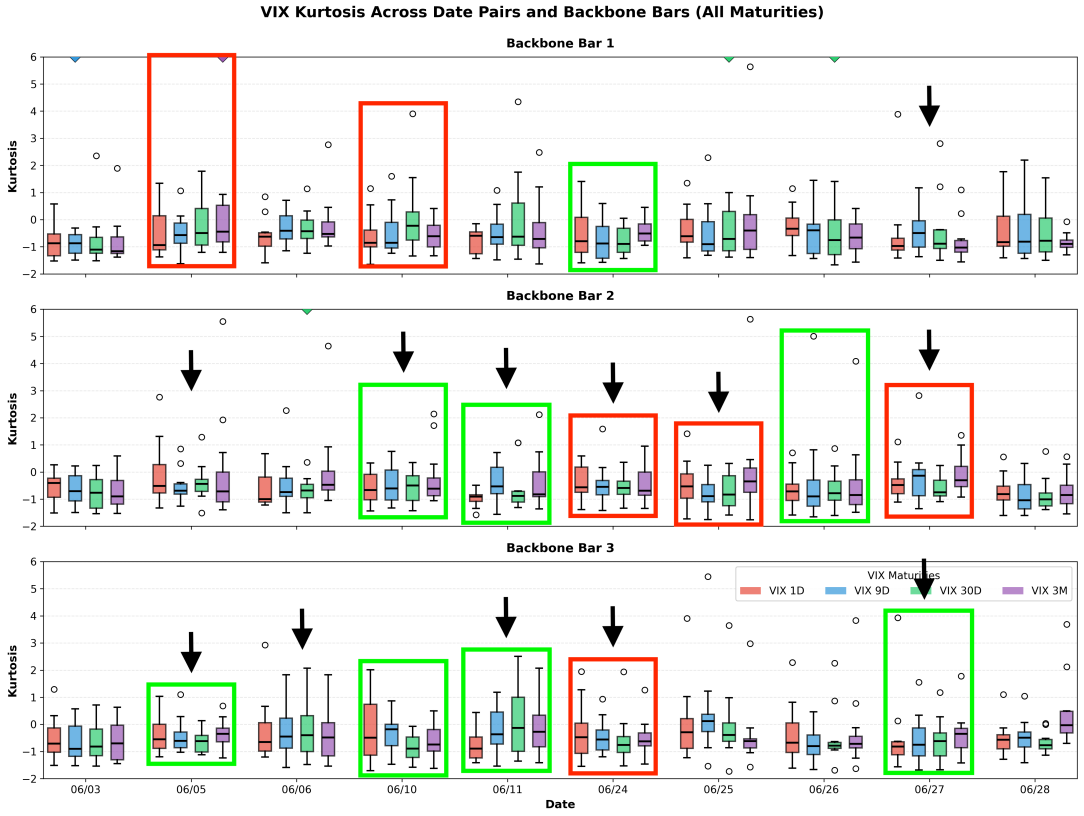


Figure S23: Distribution of the **kurtosis** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure S13 for the interpretation of the markers.

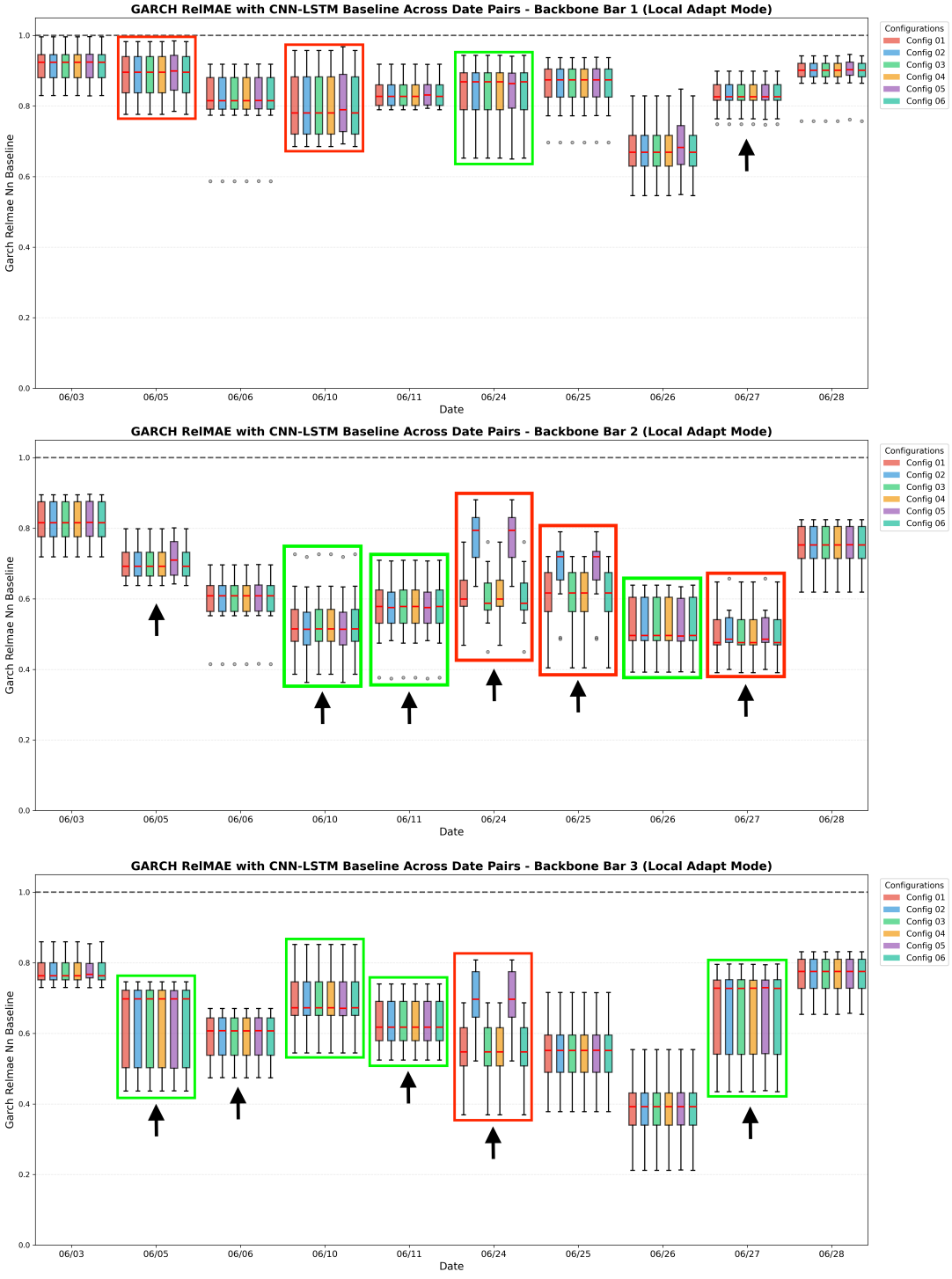


Figure S24: Sequential RelMAE boxplots for the INTC dataset across date pairs in Local Adapt mode using the raw CNN-LSTM imputation as the baseline (dashed line at 1.00). Top: Backbone Bar 1; Middle: Backbone Bar 2; Bottom: Backbone Bar 3. See Figure S13 for the interpretation of the markers (arrows and rectangles).